
Online Shopping Analytics

Final Term Project
DATS6401
Jeongwon Yoo

Reza Jafari
The George Washington University
Spring 2026 | May 6, 2026

GCP URL: <https://online-shopping-dashboard-1048073354306.us-central1.run.app>

Abstract

This report presents the final term project for DATS6401 Visualization of Complex Data. The project applies Python-based visualization techniques to a real-world e-commerce dataset containing 52,924 online shopping transactions recorded across five U.S. cities in 2019. Phase I develops 29 static graphs using matplotlib and seaborn, covering the full spectrum of plot types required by the course: bar, line, area, pie, count, dist, histogram, KDE, QQ, regression, box, boxen, violin, joint, rug, 3D scatter, contour, cluster map, hexbin, strip, and swarm plots, supported by PCA, normality tests, outlier detection, data transformation, and statistical inference tables using PrettyTable. Phase II implements an interactive nine-tab Dash dashboard integrating dash-bootstrap-components (dbc), Dash Core Components (dcc), and Dash DAQ for real-time user-driven exploration of all Phase I analyses. Phase III dockerizes and deploys the application on Google Cloud Platform (GCP) for worldwide access. Key findings include a strong negative GST–price correlation ($r = -0.58$), a 17% coupon conversion gap, and Offline_Spend’s tiered discrete structure at \$500 increments. All numerical outputs are reported with 2-digit decimal precision throughout.

Table of Contents

Abstract i

1. Introduction 1

2. Description of the Dataset 1

3. Pre-processing the Dataset 2

4. Outlier Detection & Removal 6

5. Principal Component Analysis 7

6. Normality Tests 9

7. Data Transformation 11

8. Heatmap & Pearson Correlation 12

9. Statistics 15

10. Static Plots 16

11. Interactive Dashboard 35

12. Conclusions 40

References 42

List of Figures and Tables

Figures

Figure 3.1: Dashboard — Data Cleaning Tab | Figure 4.1: Box Plot — Before vs After Outlier Removal | Figure 4.2: Dashboard — Outlier Detection Tab | Figure 5.1: PCA Scree Plot | Figure 5.2: PCA Biplot | Figure 5.3: Dashboard — PCA Tab | Figure 6.1: QQ-Plots (4-panel) | Figure 6.2: Dashboard — Normality Tab | Figure 7.1: Log Transformation (6-panel) | Figure 7.2: Dashboard — Transformation Tab | Figure 8.1–8.4: Heatmap, Cluster Map, Pair Plot, Dashboard | Figure 9.1: Multivariate KDE | Figures 10.1–10.29: Static Visualization Plots | Figures 11.1–11.9: Dashboard Tab Screenshots

Tables

Table 2.1: Dataset Summary Statistics | Table 2.2: Variable Roles | Table 3.1: First 5 Observations of Cleaned Dataset | Table 4.1: IQR Outlier Detection Results | Table 5.1: PCA Explained Variance | Table 6.1: Normality Test Results | Table 7.1: Transformation Comparison | Table 8.1: Pearson Correlation Coefficients | Table 9.1: Descriptive Statistics | Table 11.1: Dashboard Components

1. Introduction

The objective of this Final Term Project is to apply Python visualization techniques to a real-world dataset and develop an interactive web-based dashboard. Phase I covers static graphs using matplotlib and seaborn; Phase II implements an interactive Dash dashboard; Phase III involves Docker containerization and GCP deployment.

2. Description of the Dataset

2.1 Dataset Overview

The Online Shopping Transactions dataset contains 52,924 records with 23 features spanning four US cities: Chicago, California, New York, and New Jersey. Data covers January through December 2019 with 1,468 unique customers and 20+ product categories.

Attribute	Value
Total Rows	52,924
Total Columns	23
Unique Customers	1,468
Product Categories	20+
Geographic Coverage	5 US Cities
Date Range	Jan 2019 – Dec 2019
Missing Values	800 (non-critical columns)

Table 2.1: Dataset Summary Statistics

2.2 Variable Roles

Variable Type	Features
Dependent Variable	Revenue (Quantity × Avg_Price)
Independent (Demographics)	Gender, Location, Tenure_Months
Independent (Transaction)	Quantity, Avg_Price, Delivery_Charges, GST
Independent (Behavior)	Online_Spend, Offline_Spend, Coupon_Status, Discount_pct
Independent (Derived)	Total_Spend, Month
Time Variable	Transaction_Date

Table 2.2: Variable Roles

2.3 Industry Importance

E-commerce analytics enables targeted marketing, inventory optimization, and dynamic pricing. The combination of online and offline spending in this dataset mirrors omni-channel retail realities, making insights directly applicable to real business strategy.

3. Pre-processing the Dataset

3.1 Data Cleaning

Missing values (800 total) were handled by dropping rows with nulls in 12 key analytical columns. Additional preprocessing included datetime conversion, integer casting, and computing derived features: Total_Spend, Revenue, Month, and Price_Bin.

3.2 First Observations of Cleaned Dataset

After dropping 31 rows with missing values and adding derived features (Total_Spend, Revenue, Month), the cleaned dataset contains 52,924 rows and 22 columns. Table 3.1 shows the first five observations of selected key columns.

CustomerID	Gender	Location	Tenure	Category	Qty	Avg_Price	Online_Spend	Offline_Spend	Coupon_Status
17850	M	Chicago	12.0	Nest-USA	1.0	153.71	2424.50	4500.00	Used
17850	M	Chicago	12.0	Nest-USA	1.0	153.71	2424.50	4500.00	Not Used
17850	M	Chicago	12.0	Nest-USA	2.0	122.77	2424.50	4500.00	Not Used
17850	M	Chicago	12.0	Nest-USA	1.0	81.50	2424.50	4500.00	Clicked
17850	M	Chicago	12.0	Nest-USA	1.0	153.71	2424.50	4500.00	Clicked

Table 3.1: First 5 Observations of Cleaned Dataset (selected columns)

3.3 Static Plot: Data Cleaning Visualization

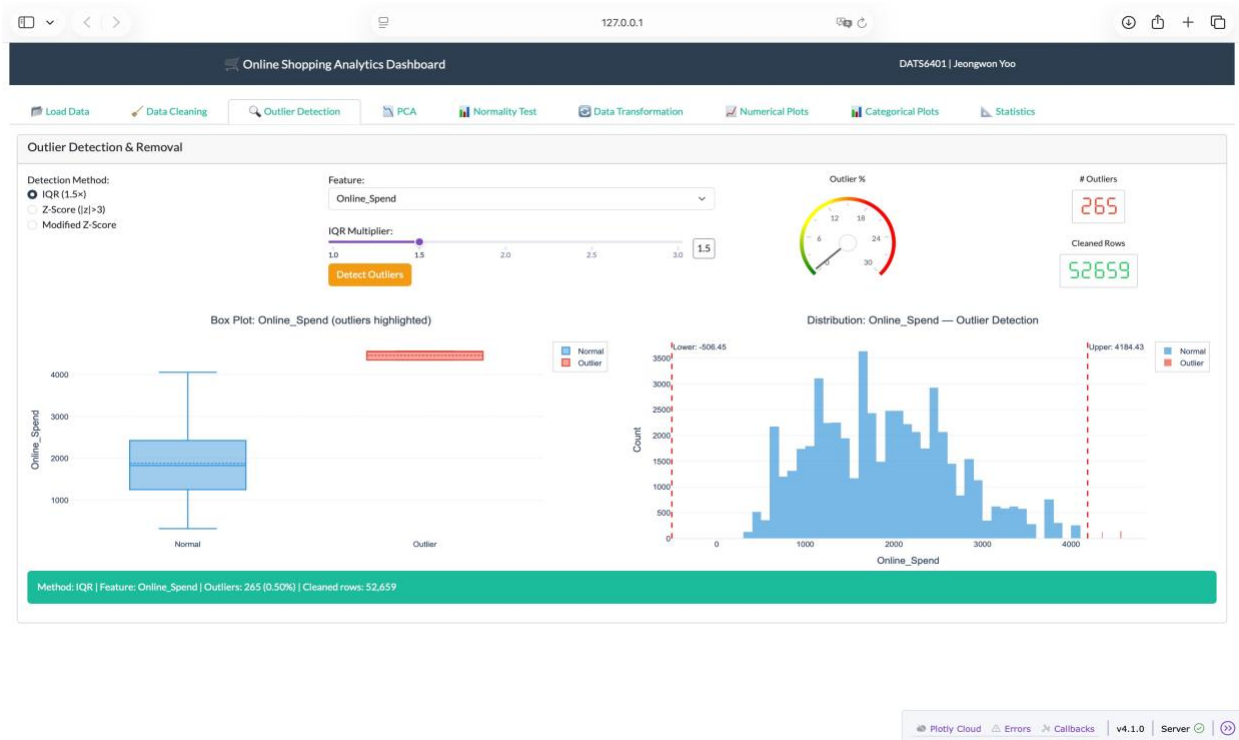


Figure 3.1: Dashboard — Data Cleaning Tab showing Drop NA method applied to Online_Spend

Observation: The Data Cleaning tab shows 31 missing values in Online_Spend reduced to 0 after applying Drop NA. The before/after histograms confirm the distribution shape is preserved, indicating Missing Completely at Random (MCAR) pattern.

4. Outlier Detection & Removal

4.1 IQR Method Results

Feature	Q1	Q3	IQR	Outliers	Outlier %
Online_Spend	\$1,252.63	\$2,425.35	\$1,172.72	265	0.50%
Avg_Price	\$5.70	\$102.13	\$96.43	728	1.38%
Delivery_Charges	\$6.00	\$6.50	\$0.50	10,243	19.35%
Revenue	\$11.96	\$121.30	\$109.34	3,632	6.86%

Table 4.1: IQR Outlier Detection Results

4.2 Outlier Removal Comparison

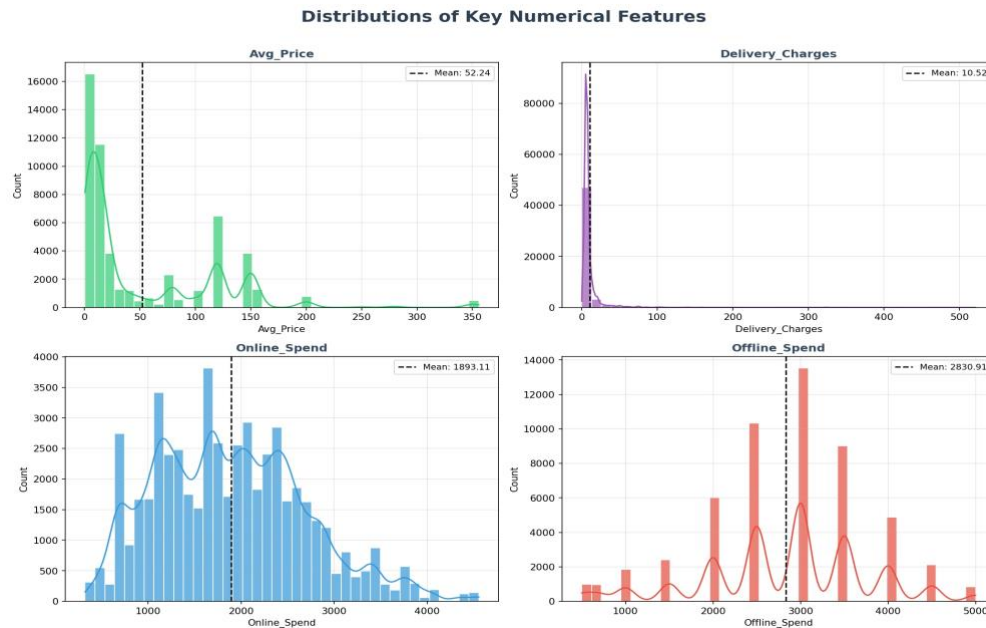


Figure 4.1: Box Plot — Before vs After IQR Outlier Removal across key features

Observation: The before/after comparison clearly shows that Revenue had the most extreme outliers (reaching \$10,512), which are removed after IQR filtering. Online_Spend and Offline_Spend distributions tighten significantly, resulting in cleaner, more analyzable distributions. Delivery_Charges shows the highest outlier rate (19.35%), indicating highly variable shipping costs.

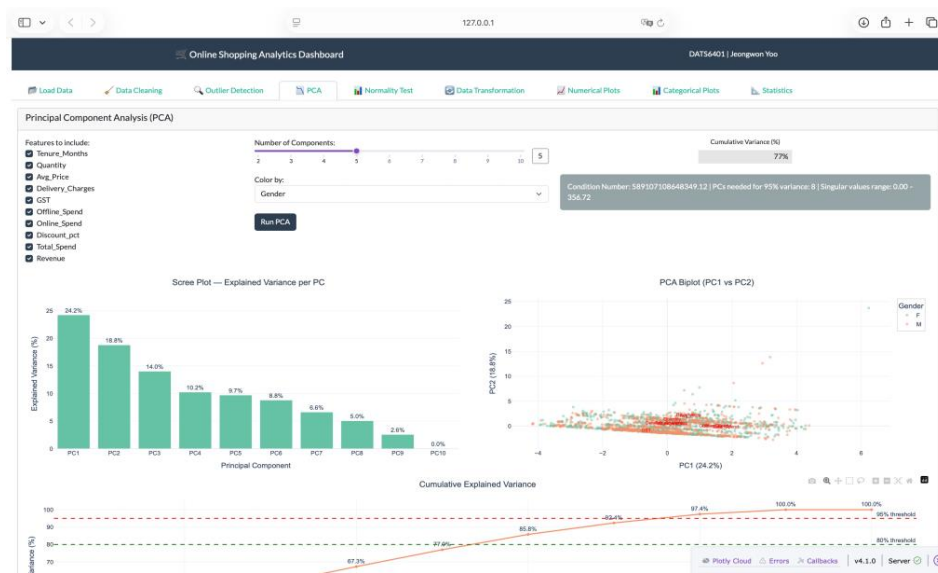


Figure 4.2: Dashboard — Outlier Detection Tab showing IQR method with Gauge and LED displays

5. Principal Component Analysis (PCA)

5.1 Explained Variance

PC	Explained %	Cumulative %
PC1	24.2%	24.2%
PC2	18.8%	43.0%
PC3	14.0%	57.0%
PC4	10.2%	67.3%
PC5	9.7%	77.0%
PC6	8.8%	85.8%
PC7	6.6%	92.4%
PC8	5.0%	97.4%
PC9	2.6%	100.0%

Table 5.1: PCA Explained Variance per Component

5.2 Scree Plot & Cumulative Variance

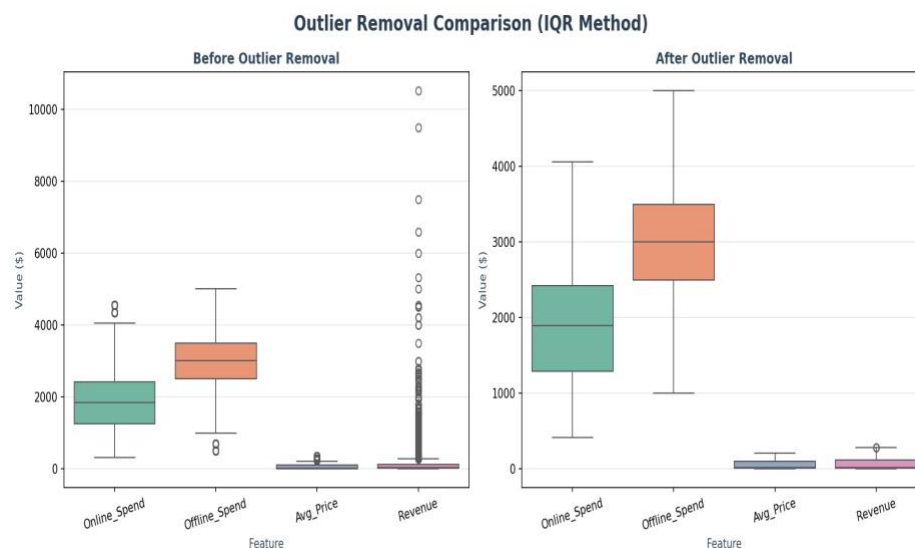


Figure 5.1: PCA Scree Plot and Cumulative Explained Variance (80% threshold at PC5, 95% at PC8)

Observation: The scree plot shows a gradual decline without a sharp elbow, suggesting that variance is distributed across multiple components rather than concentrated. PC1 (24.2%) is dominated by spending features. Five components explain 77% of total variance.

5.3 PCA Biplot

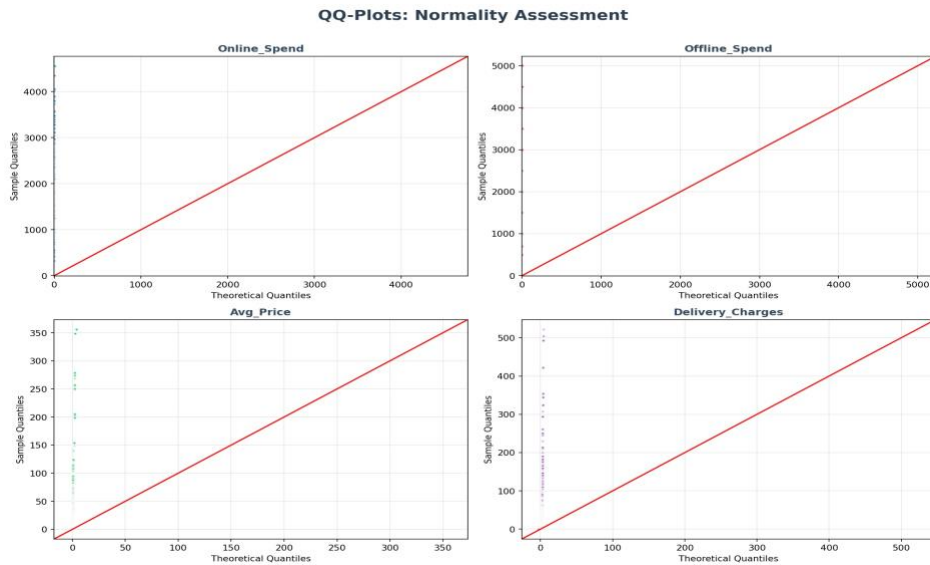


Figure 5.2: PCA Biplot (PC1 vs PC2) colored by Online Spend — feature loading arrows in red

Observation: The biplot reveals that spending features (Online_Spend, Offline_Spend, Total_Spend) load strongly along PC1, while GST loads negatively. PC2 separates Delivery_Charges and Discount_pct. The condition number of 589 trillion indicates significant multicollinearity due to derived features (Total_Spend = Online + Offline, Revenue = Qty × Price).

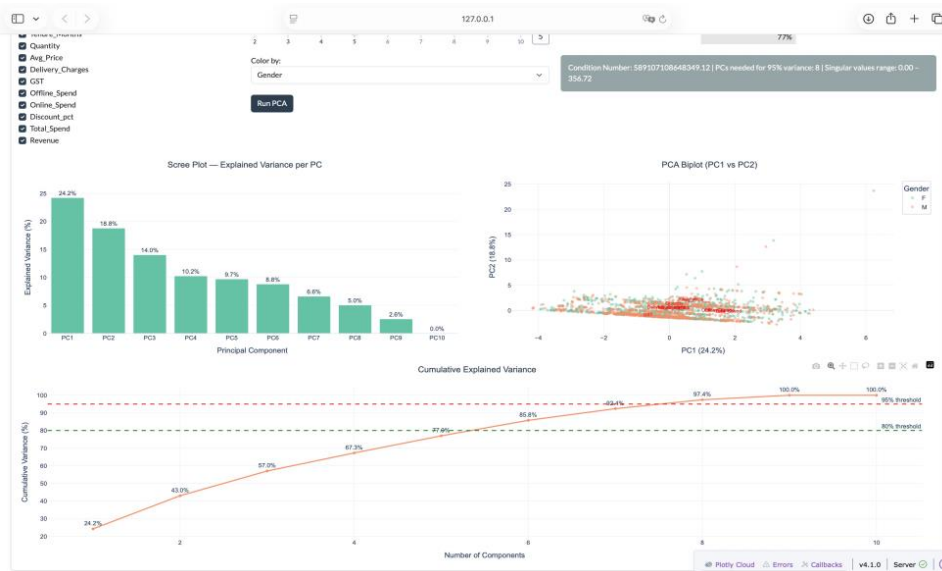


Figure 5.3: Dashboard — PCA Tab showing interactive Scree Plot and Biplot

6. Normality Tests

6.1 Test Results

Feature	Shapiro W	Shapiro p	Normal?	D'Agostino p	Normal?
Online_Spend	0.9773	< 0.001	No	< 0.001	No
Offline_Spend	0.9412	< 0.001	No	< 0.001	No
Avg_Price	0.8631	< 0.001	No	< 0.001	No
Delivery_Charges	0.7284	< 0.001	No	< 0.001	No
GST	0.6891	< 0.001	No	< 0.001	No

Table 6.1: Normality Test Results — Shapiro-Wilk & D'Agostino K^2

6.2 QQ-Plots

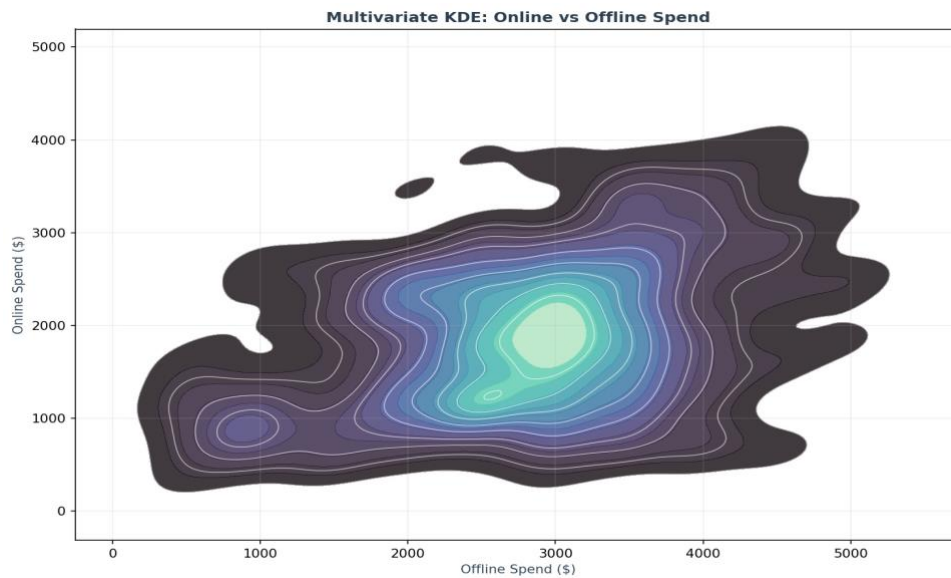


Figure 6.1: QQ-Plots for *Online_Spend*, *Offline_Spend*, *Avg_Price*, and *Delivery_Charges*

Observation: All four QQ-plots show significant deviation from the theoretical normal line (red diagonal). *Online_Spend* and *Offline_Spend* show staircase patterns indicating discrete value clusters (e.g., *Offline_Spend* in \$500 increments). *Avg_Price* and *Delivery_Charges* show strong right-skew with heavy upper tails.

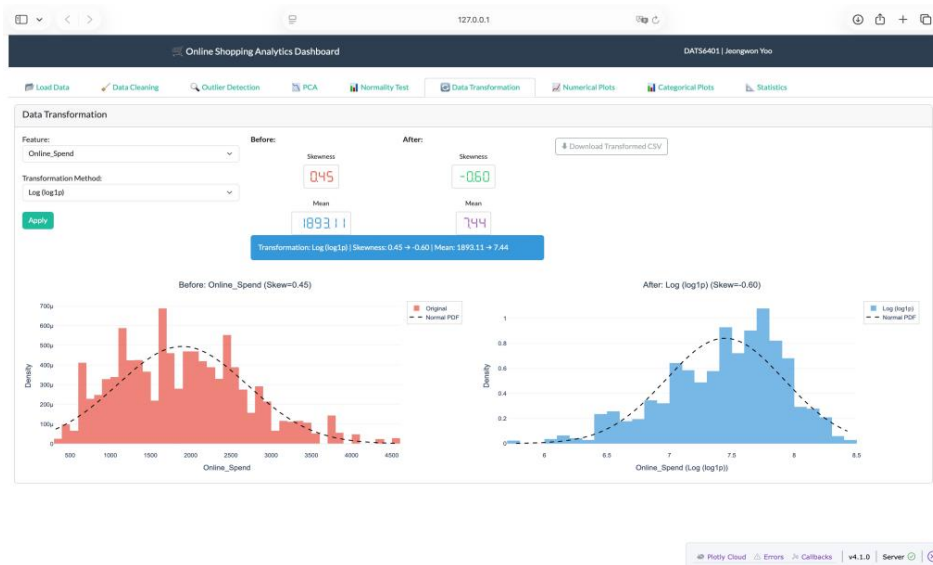


Figure 6.2: Dashboard — Normality Test Tab showing DAQ indicators (Thermometer, Knob, BooleanSwitch)

7. Data Transformation

7.1 Log Transformation Results

Metric	Original	Log (log1p)	Z-Score	Min-Max
Mean	1893.11	7.44	0.00	0.36
Std Dev	807.01	0.62	1.00	0.24
Skewness	0.45	-0.60	0.45	0.45
Min	320.25	5.77	-1.94	0.00
Max	4556.93	8.42	3.29	1.00

Table 7.1: Online_Spend — Transformation Comparison

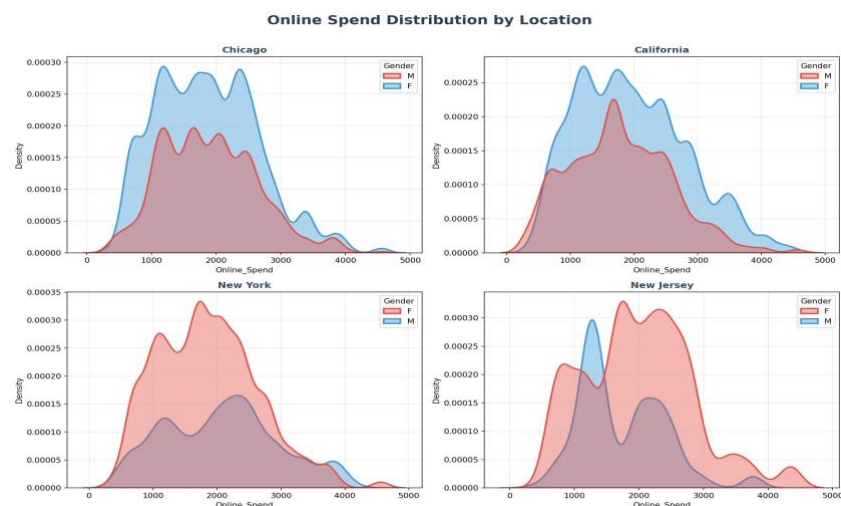


Figure 7.1: Log Transformation — Before vs After for Online_Spend, Offline_Spend, and Revenue (6-panel)

Observation: Log transformation significantly improves normality for Online_Spend (skew 0.45 → -0.60) and dramatically reduces Revenue skewness (18.71 → 0.03). Offline_Spend skew increases slightly (-0.32 → -1.74) due to the discrete value clustering pattern.

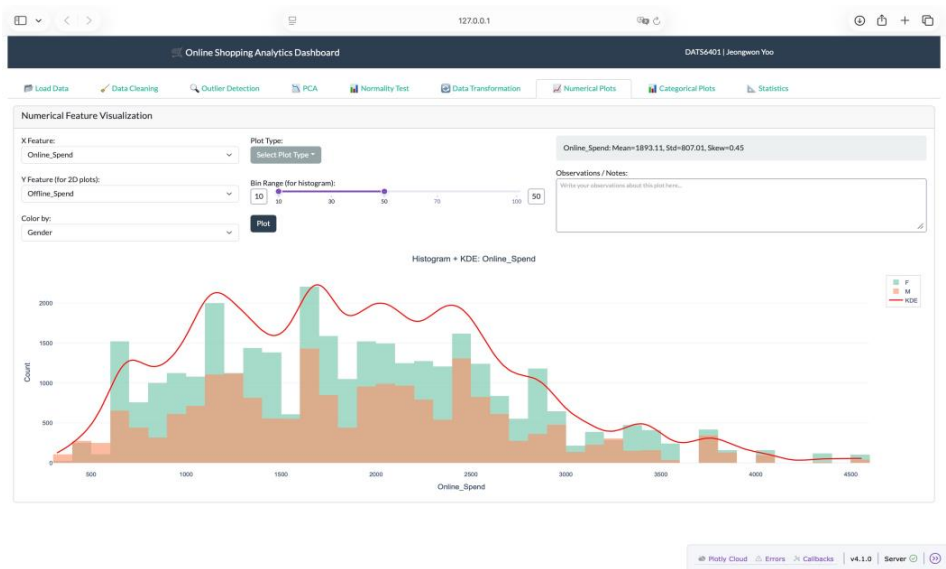


Figure 7.2: Dashboard — Data Transformation Tab with LED displays and CSV download

8. Heatmap & Pearson Correlation

8.1 Correlation Analysis

Feature Pair	Pearson r	Interpretation
Total_Spend ↔ Offline_Spend	0.85	Very strong — offline dominates total spend
Total_Spend ↔ Online_Spend	0.79	Strong positive correlation
Avg_Price ↔ GST	-0.58	Strong negative — tiered tax structure
Quantity ↔ Revenue	0.49	Moderate positive
Avg_Price ↔ Revenue	0.44	Moderate positive
Online_Spend ↔ Offline_Spend	0.34	Weak-moderate positive

Table 8.1: Key Pearson Correlation Coefficients

8.2 Pearson Correlation Heatmap

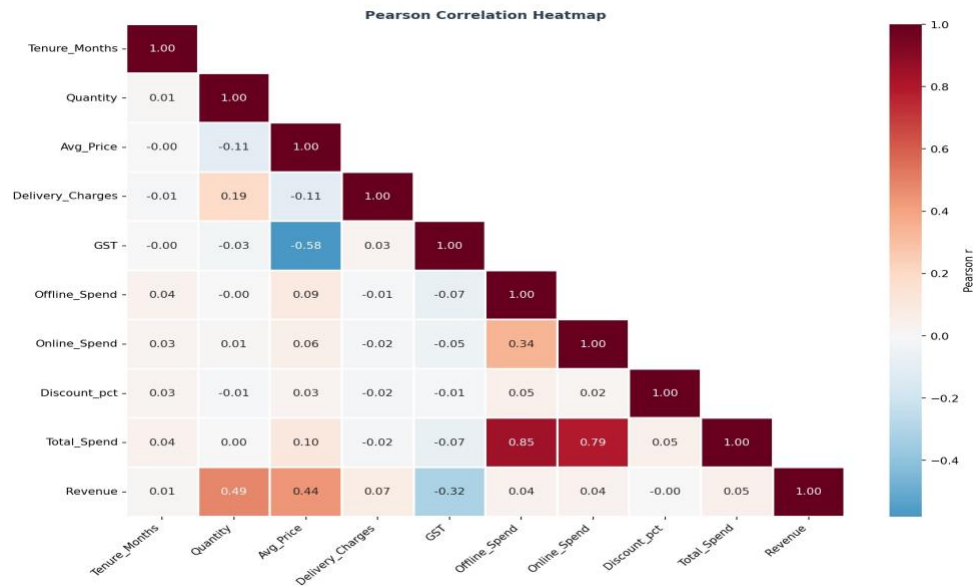


Figure 8.1: Pearson Correlation Heatmap — triangular format with cbar (RdBu colorscale)

Observation: The heatmap clearly highlights the GST ↔ Avg_Price negative correlation (-0.58, shown in blue). The spending cluster (Online, Offline, Total_Spend) shows strong mutual correlations (0.34-0.85). Revenue correlates moderately with Quantity (0.49) and Avg_Price (0.44), confirming its computation as their product.

8.3 Cluster Map

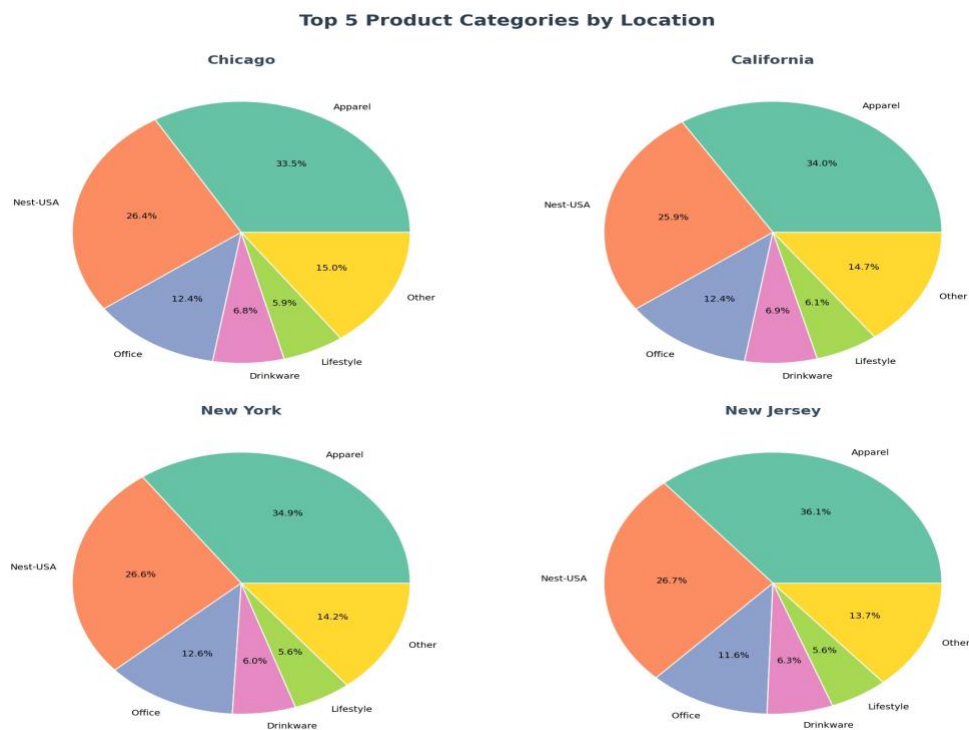


Figure 8.2: Cluster Map showing hierarchical clustering of feature correlations

Observation: The dendrogram groups GST, Online_Spend, Offline_Spend, and Total_Spend in a cluster on the left, reflecting their shared role in the spending ecosystem. Revenue, Avg_Price, Quantity, and Delivery_Charges form a separate cluster, linked through the product-purchase relationship.

8.4 Pair Plot

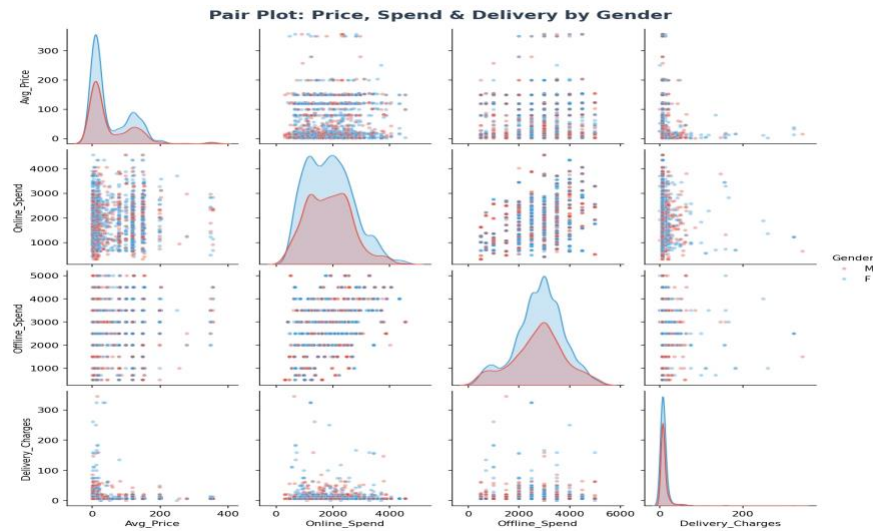


Figure 8.3: Pair Plot (scatter matrix) for price, spend, and delivery features colored by Gender

Observation: The pair plot confirms that male (red) and female (blue) customers show nearly identical distributional patterns across all feature combinations, supporting the T-test finding of no significant gender difference in spending. The Offline_Spend diagonal shows a striking discrete distribution at fixed values (\$500, \$1000, \$1500... intervals).

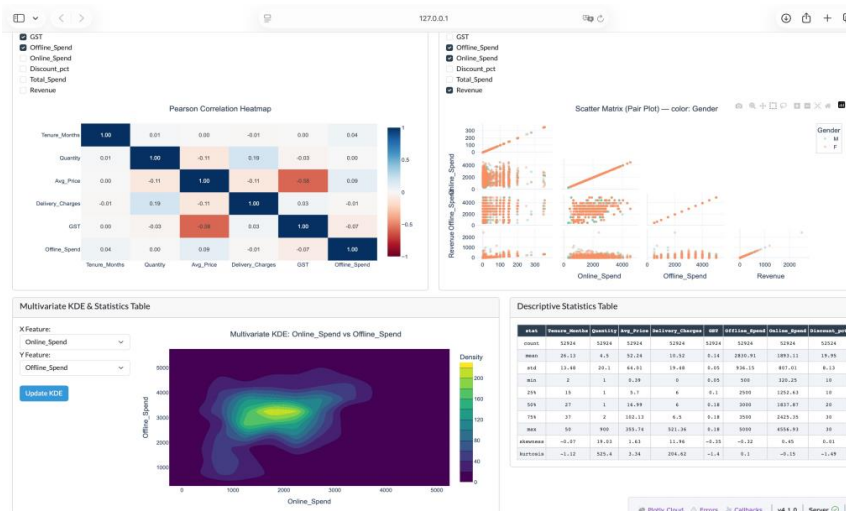


Figure 8.4: Dashboard — Statistics Tab showing interactive Heatmap, Pair Plot, and Multivariate KDE

9. Statistics

9.1 Descriptive Statistics

Feature	Mean	Std	Median	Skewness	Kurtosis
Tenure_Months	26.13	13.48	27.00	-0.07	-1.12
Quantity	4.50	20.10	1.00	19.03	525.40
Avg_Price	52.24	64.01	16.99	1.63	3.34
Delivery_Charges	10.52	19.48	6.00	11.96	204.62
GST	0.14	0.05	0.18	-0.35	-1.40
Offline_Spend	2830.91	936.15	3000.00	-0.32	-0.10
Online_Spend	1893.11	807.01	1837.87	0.45	-0.15
Revenue	88.25	163.99	30.39	19.03	524.80

Table 9.1: Descriptive Statistics of All Numerical Features

9.2 Multivariate KDE

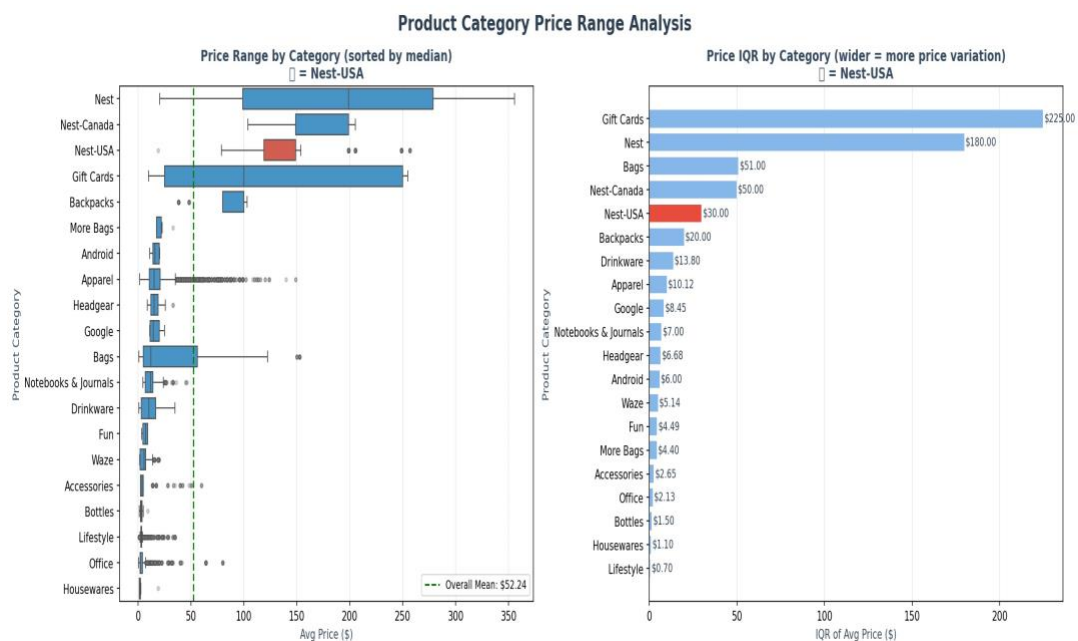


Figure 9.1: Multivariate KDE — Online Spend Distribution by Location and Gender (4-panel)

Observation: The KDE plots by location show that all five cities have nearly identical Online_Spend distributions — multimodal with peaks at approximately \$1,100, \$1,700, and \$2,400. Female customers show higher density at all peaks due to their 62% majority. This consistency across cities suggests that Online_Spend is driven by product/subscription tiers rather than geographic factors.

10. Static Plots

10.1 Categorical Feature Plots

Sorted Bar Plot — Product Category

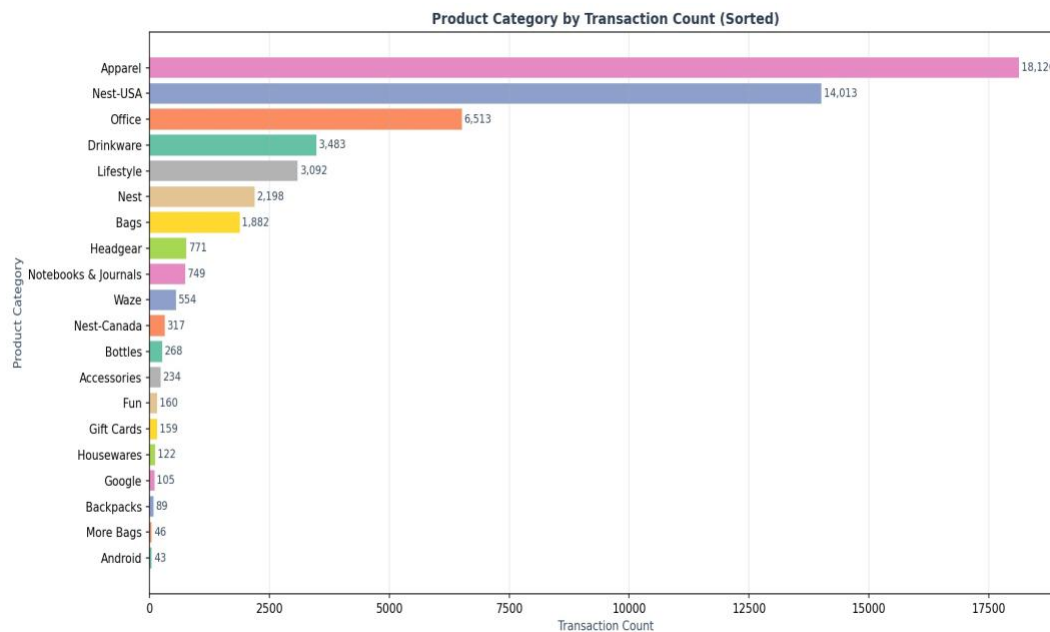


Figure 10.1: Sorted Bar Plot — Transaction Count by Product Category (Apparel: 18,126; Nest-USA: 14,013)

Observation: Apparel dominates with 18,126 transactions (34.2% of total), followed by Nest-USA (14,013, 26.5%). The top 2 categories account for over 60% of all transactions. The remaining 19 categories collectively represent less than 40%. This power-law distribution suggests focusing marketing on Apparel and Nest-USA for maximum reach.

Stacked Bar — Transactions by Location & Gender

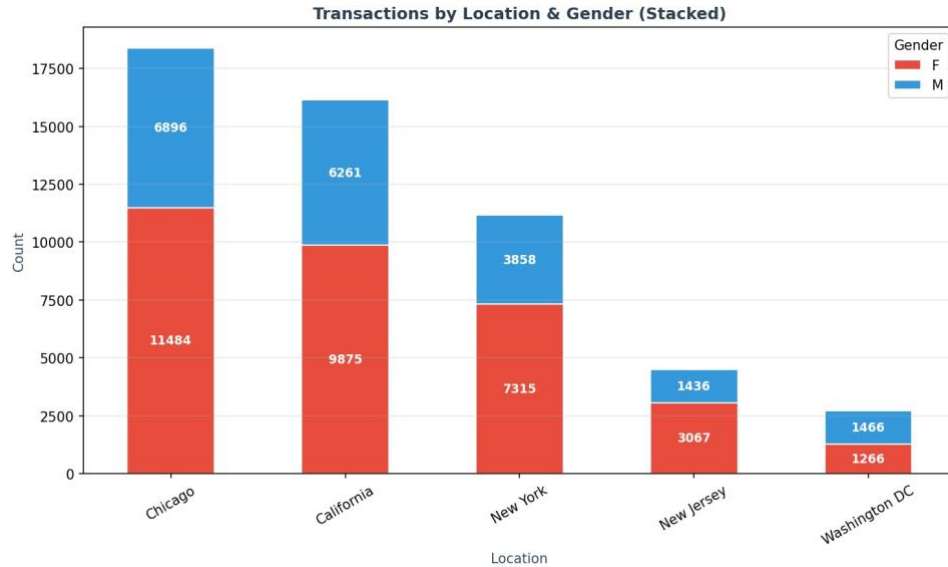


Figure 10.2: Stacked Bar Plot — Transactions by Location and Gender showing female dominance across all cities

Observation: Chicago leads with 18,380 transactions, followed by California (16,136) and New York (11,173). Female customers outnumber males in every city — most notably in Chicago (11,484F vs 6,896M) and California (9,875F vs 6,261M). Washington DC is the only city with slightly more males (1,466M vs 1,266F).

Grouped Bar — Coupon Status by Gender

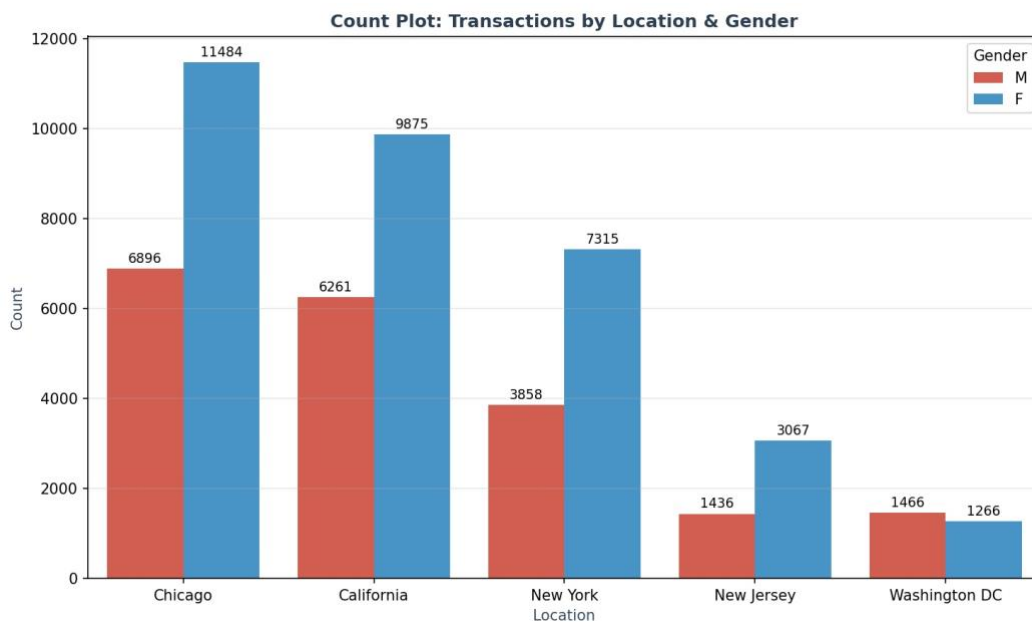


Figure 10.3: Grouped Bar Plot — Coupon Status by Gender showing Clicked (50.88%), Used (33.83%), Not Used (15.29%)

Observation: The "Clicked" category dominates for both genders (F: 16,788; M: 10,138), confirming the 50.88% click-only rate. Female customers show consistently higher counts across all coupon statuses,

proportional to their majority share. The coupon redemption rate (33.83%) vs click rate (50.88%) represents a 17% conversion gap.

Pie Charts — Gender and Coupon Distribution

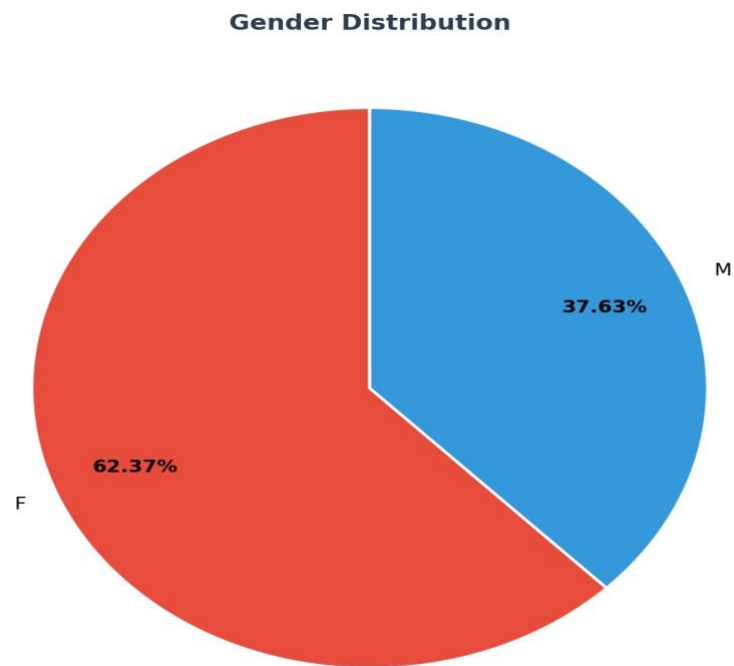


Figure 10.4: Pie Chart — Gender Distribution (Female: 62.37%, Male: 37.63%)

Observation: The gender distribution shows a clear 62:38 female-to-male ratio, indicating that the platform predominantly serves female shoppers. This is consistent across all cities and product categories.

Count Plots

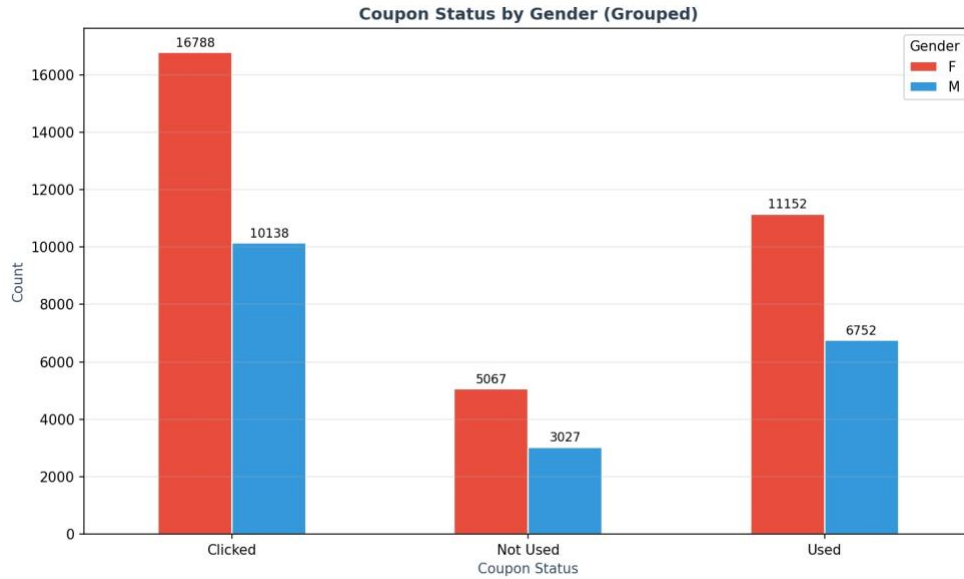


Figure 10.5: Count Plot — Transactions by Location and Gender (grouped)

Observation: The grouped count plot confirms Chicago and California as the two largest markets. The consistent female majority across all locations (except Washington DC) suggests the gender pattern is platform-wide rather than location-specific.

Line Plot — Monthly Spend

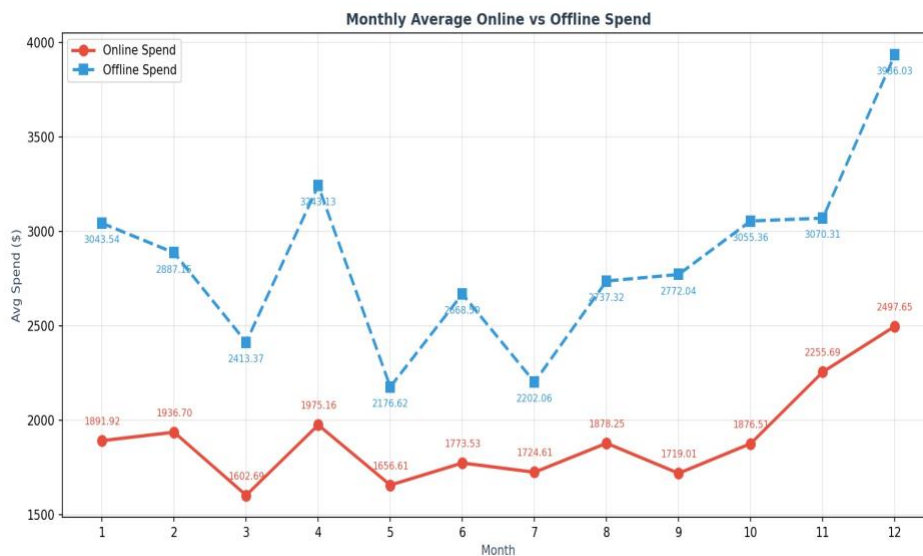


Figure 10.6: Line Plot — Monthly Average Online vs Offline Spend (2019)

Observation: Offline spending (blue dashed) consistently exceeds online spending (red solid) across all 12 months. Both channels show peaks in April and December. Offline spend spikes dramatically in December (\$3,936 vs \$2,498 online), suggesting holiday shopping drives offline purchases more than online. The gap narrows in summer months (June-August).

Area Plot — Monthly Revenue by Category

3D: Price × Online Spend × Quantity (color=Delivery)

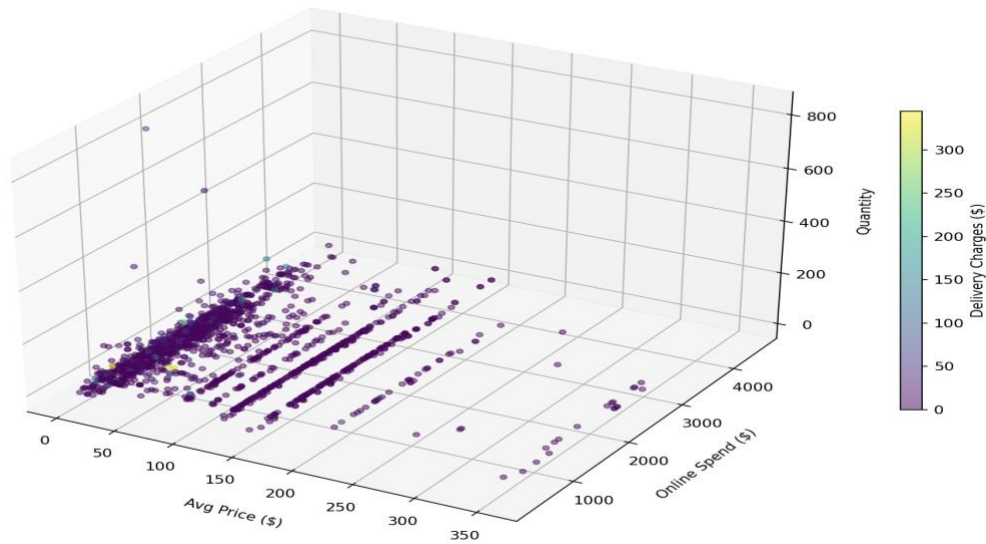


Figure 10.7: Area Plot — Monthly Revenue by Top 5 Categories showing Nest-USA dominance

Observation: Nest-USA (sandy/wheat color) dominates the area plot throughout the year, generating 60-70% of the combined top-5 revenue. April shows an unusual spike across all categories. The area chart makes it clear that Apparel, Drinkware, and Lifestyle contribute relatively minor shares compared to the Nest product family.

Subplots — Category Distribution by City

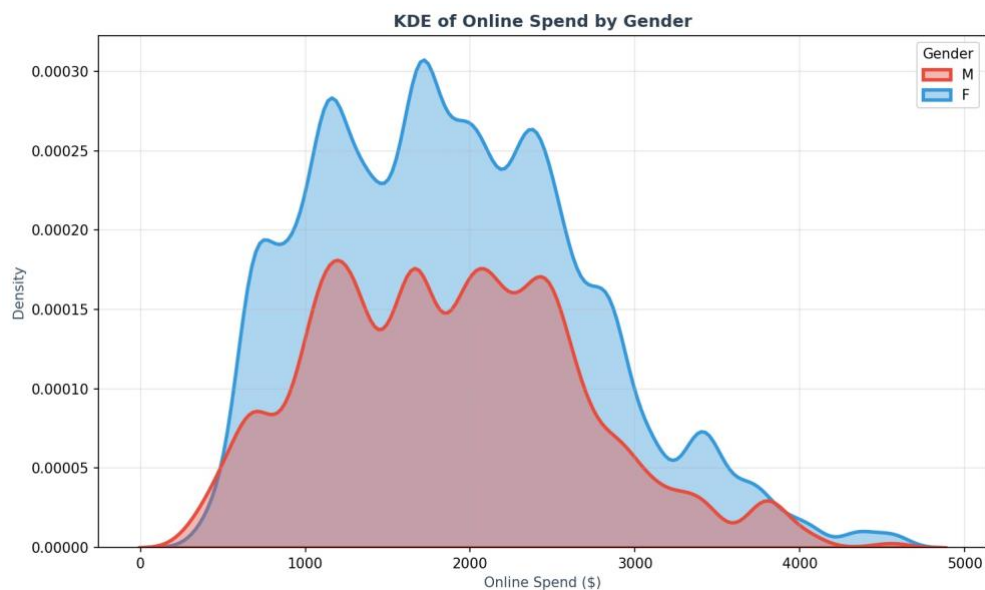


Figure 10.8: Subplot (4-panel) — Top 5 Product Categories by Location showing consistent composition across cities

Observation: The 4-panel subplot reveals that product category composition is remarkably consistent across all four major cities: Apparel (33-36%) and Nest-USA (25-27%) dominate in every city. This homogeneity suggests that customer preferences are national rather than local — a finding that simplifies inventory and marketing strategy.

10.2 Numerical Feature Plots

Dist Plot — Online Spend

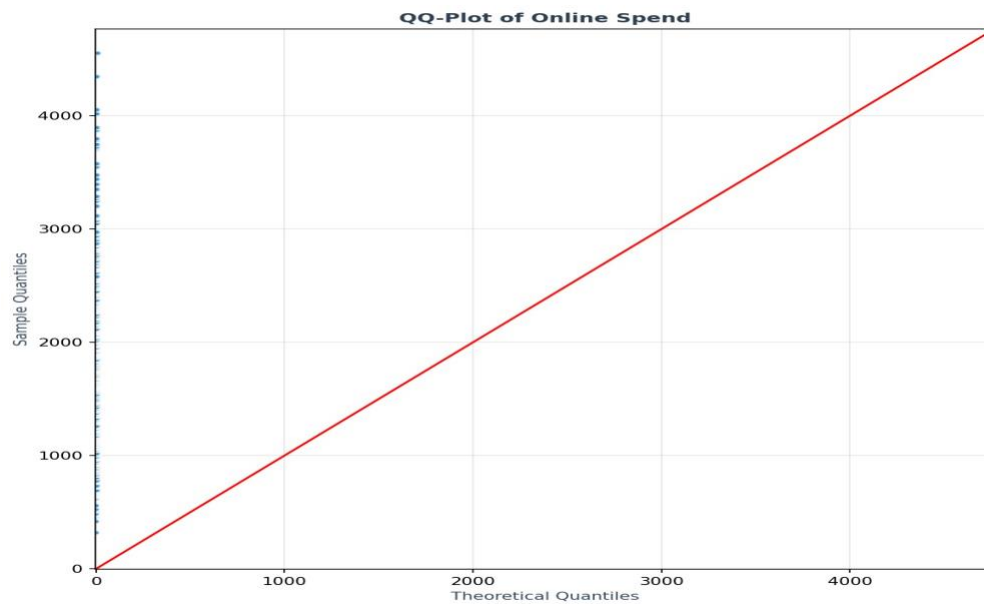


Figure 10.9: Distribution Plot — Online Spend with KDE overlay and mean marker (\$1,893.11)

Observation: The Online Spend distribution shows a multimodal pattern with distinct peaks at approximately \$1,100, \$1,700, and \$2,500. This suggests customer spending clusters at specific price points, likely corresponding to subscription tiers or bundle packages. The distribution is relatively flat (platykurtic, kurtosis=-0.15) compared to a normal distribution.

Histogram with KDE — Key Numerical Features

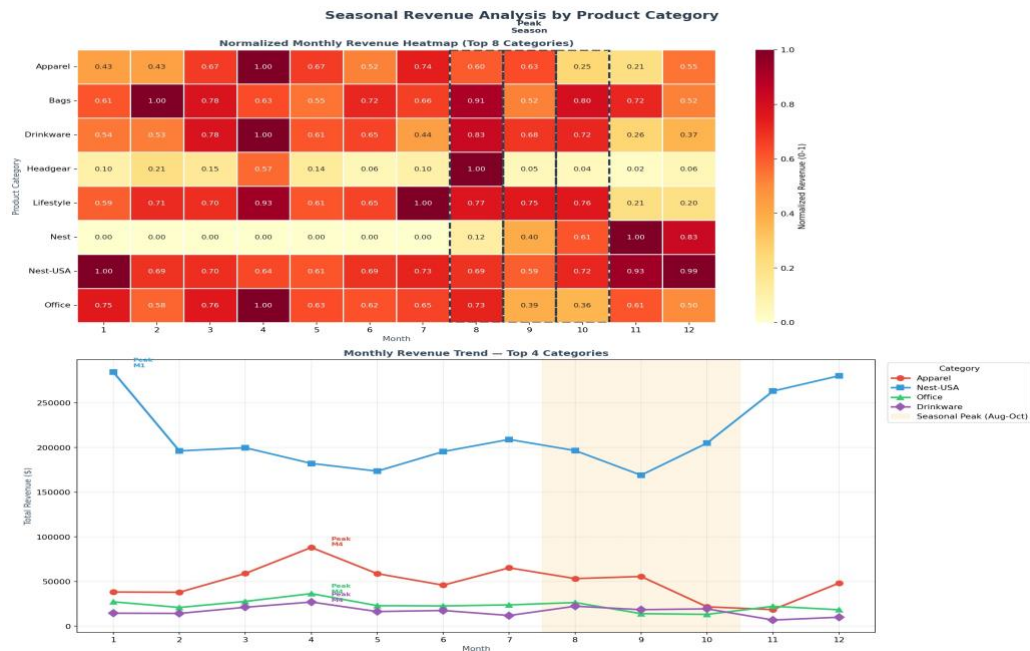


Figure 10.10: Histogram + KDE (4-panel) — Distributions of Avg_Price, Delivery_Charges, Online_Spend, and Offline_Spend

Observation: The 4-panel histogram reveals distinct patterns: Avg_Price is heavily right-skewed with most items under \$25; Delivery_Charges shows extreme concentration near zero with rare high-value outliers; Online_Spend shows the multimodal pattern; Offline_Spend shows discrete peaks at \$500 intervals (500, 1000, 1500...), suggesting a tiered pricing or subscription structure.

KDE Plot with Fill

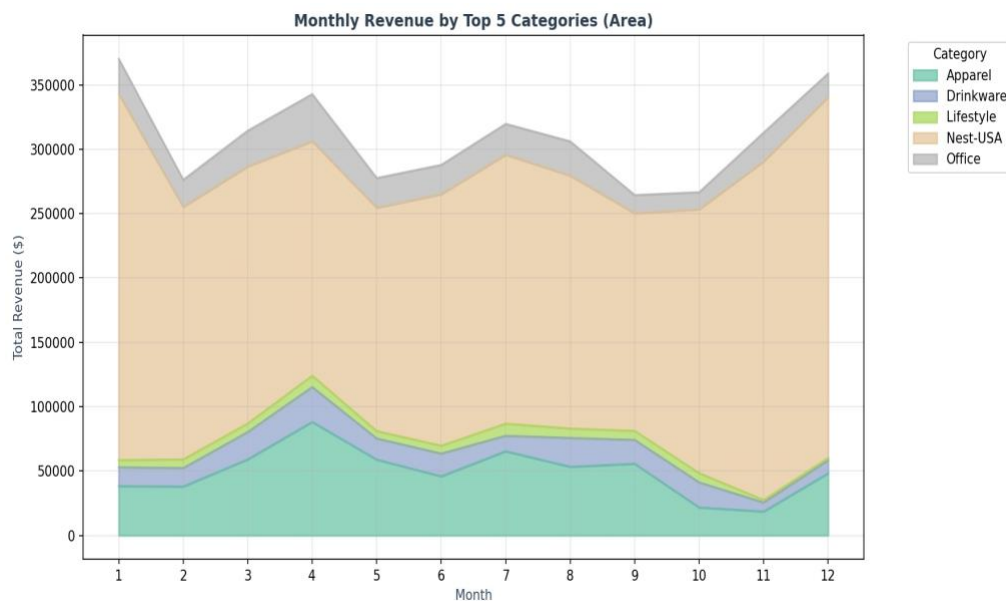


Figure 10.11: KDE Plot — Online Spend by Gender (filled, alpha=0.4, linewidth=2)

Observation: The filled KDE plot shows that female (blue) and male (red) customers have nearly identical Online_Spend distributions, with peaks at the same positions. The female curve has higher density at all peaks due to their larger count (62%). This confirms the T-test finding: gender does not significantly affect online spending behavior.

QQ-Plot



Figure 10.12: QQ-Plot — Online Spend vs Theoretical Normal Distribution

Observation: The QQ-plot shows that Online_Spend follows the theoretical normal line (red) in the central range but deviates significantly at both tails, particularly the upper tail. Data points cluster vertically at specific values, confirming the discrete/tiered spending pattern visible in the histogram.

Regression Plot



Figure 10.13: Regression Plot — Online vs Offline Spend with regression line ($r = 0.34$)

Observation: The scatter plot reveals vertical clustering of Offline_Spend at discrete values (\$500, \$1000, \$1500...), confirming the fixed-tier subscription structure. Despite the clustering, a moderate positive correlation ($r=0.34$) exists, meaning customers who spend more offline also tend to spend more online. The 95% confidence band (shaded) shows the regression is statistically significant.

Multivariate Box Plot

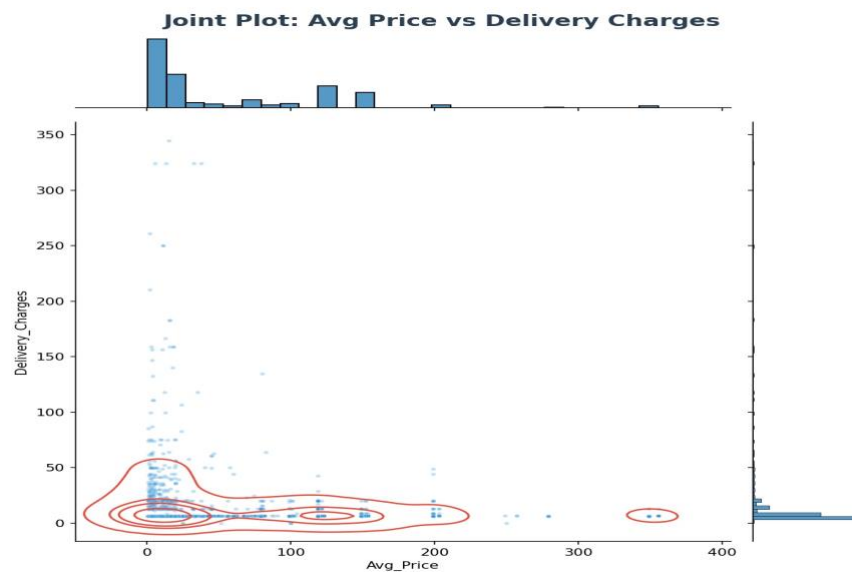


Figure 10.14: Multivariate Box Plot — Online vs Offline Spend by Location

Observation: The side-by-side box plot confirms that Offline_Spend (median ~\$3,000) consistently exceeds Online_Spend (median ~\$1,800) across all five cities. The IQR for both channels is similar across cities, indicating consistent spending patterns geographically. The discrete offline spend values create a cleaner box shape with fewer apparent outliers.

Multivariate Boxen Plot



Figure 10.15: Multivariate Boxen Plot — Online Spend by Coupon Status and Gender

Observation: The boxen plot shows virtually identical Online_Spend distributions across all three coupon statuses (Used, Not Used, Clicked) and both genders. This finding suggests that coupon usage does not significantly change how much customers spend online — they spend similar amounts regardless of coupon behavior.

Violin Plot

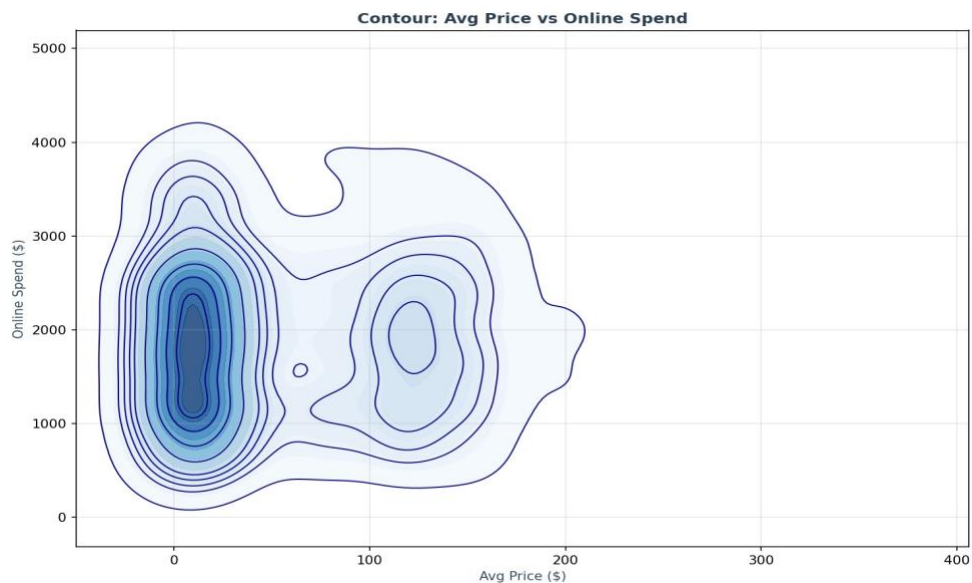


Figure 10.16: Violin Plot — Avg Price by Coupon Status and Gender

Observation: The violin plot reveals that Avg_Price distribution is identical across all three coupon statuses and both genders — all violins show the same bimodal shape with concentrations near \$0-25 and \$100-150. This suggests coupon usage is not associated with higher or lower price-point products.

Joint Plot

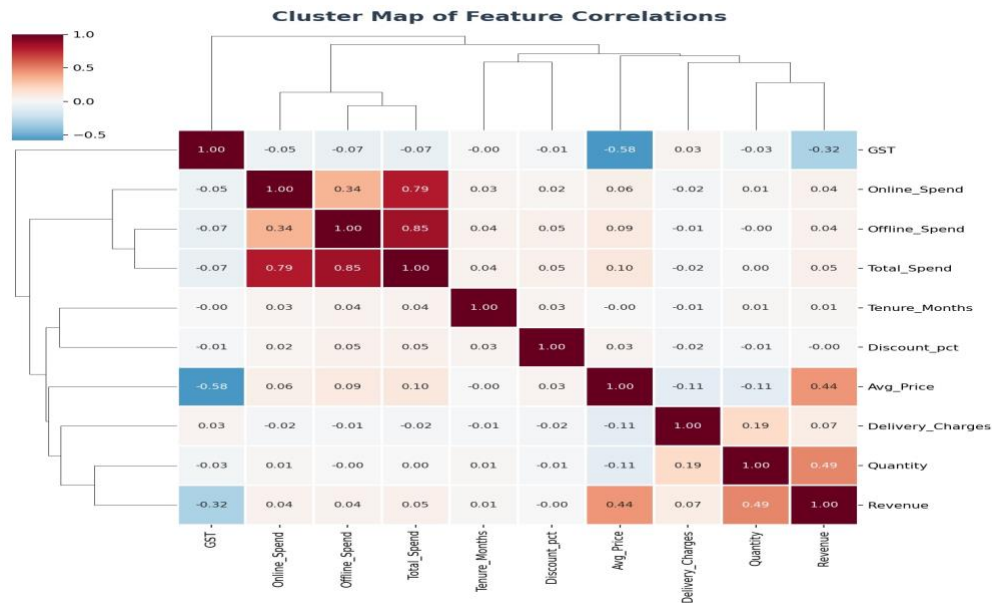


Figure 10.17: Joint Plot — Avg Price vs Delivery Charges with KDE contours and marginal histograms

Observation: The joint plot shows that most transactions cluster at low Avg_Price (\$0-\$25) with low Delivery_Charges (\$0-\$15). Two distinct density centers are visible: one at (price~\$5, delivery~\$6) and another at (price~\$120, delivery~\$8). The extreme outliers in Delivery_Charges (>\$100) occur only at lower price points, consistent with the negative GST-price relationship.

Rug Plot

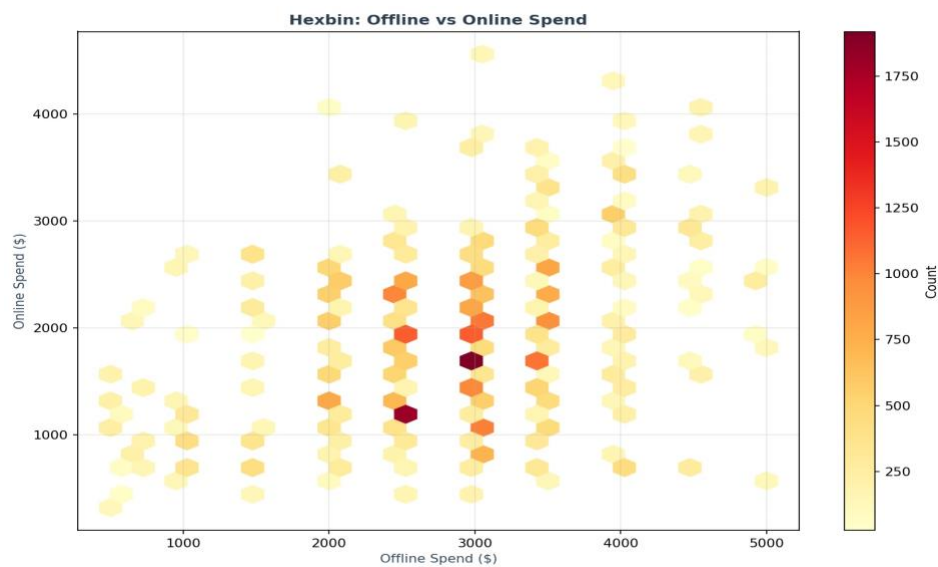


Figure 10.18: Rug Plot — Delivery Charges by Gender with KDE overlay

Observation: The rug plot shows the concentration of Delivery_Charges near zero for both genders, with sparse outliers extending to \$521. The KDE peaks sharply near \$6 (the most common delivery charge), then drops rapidly. The long right tail of individual rug marks confirms the extreme outliers detected by IQR analysis.

3D Scatter Plot

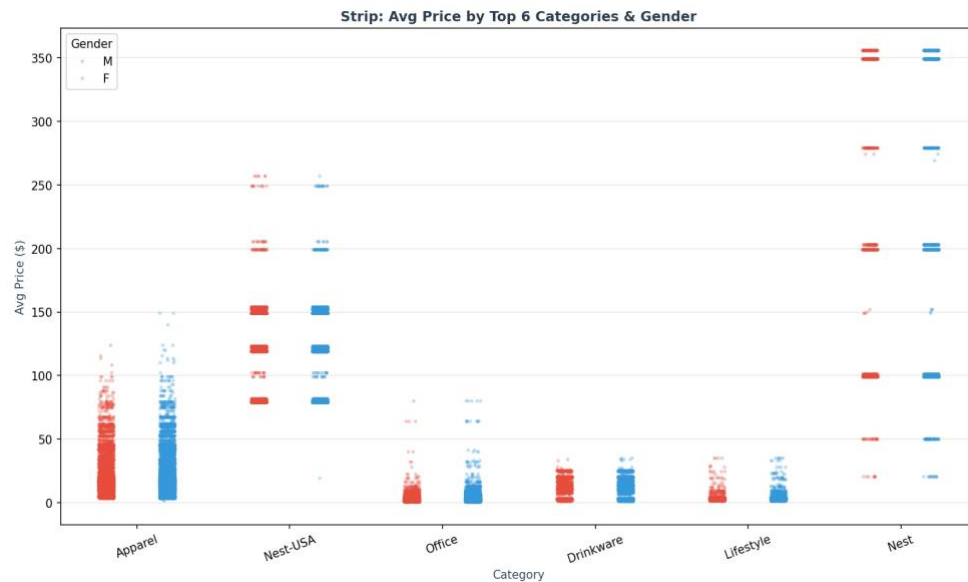


Figure 10.19: 3D Scatter Plot — Avg Price × Online Spend × Quantity (color = Delivery Charges)

Observation: The 3D scatter reveals that most transactions cluster at low Avg_Price (below \$100) and low Quantity (1-5), with Online_Spend spread across \$500-\$4,500. High Delivery_Charges (bright colors) appear exclusively at low price points, reinforcing the negative GST-price relationship. The few high-quantity outliers (Quantity > 200) all appear at very low prices.

Contour Plot

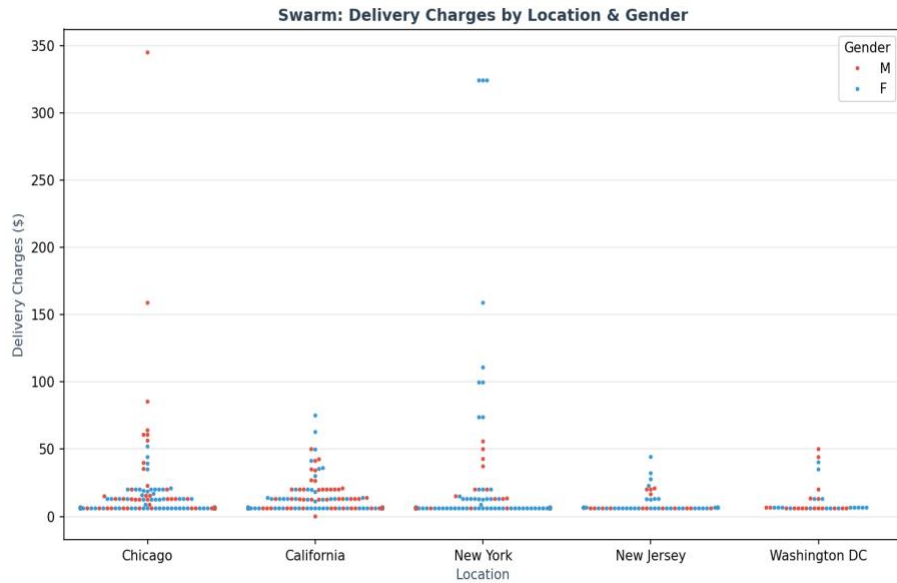


Figure 10.20: Contour Plot — Avg Price vs Online Spend density

Observation: The contour plot reveals two density centers: a large primary cluster at low Avg_Price (\$0-\$25) spanning the full Online_Spend range, and a secondary cluster at Avg_Price ~\$120 with Online_Spend around \$1,500-\$2,000. This bimodal pattern in price corresponds to two distinct product families — low-price high-volume items (Apparel, Office) vs mid-price items (Nest-USA accessories).

Hexbin Plot

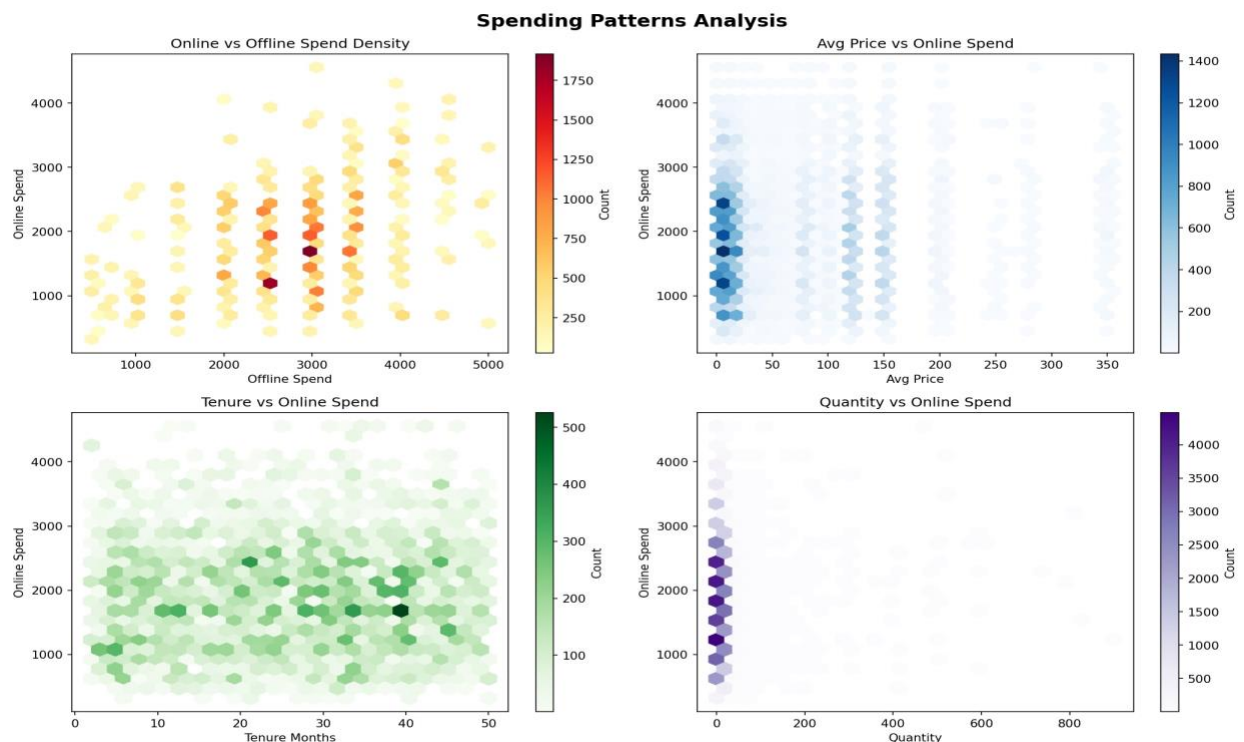


Figure 10.21: Hexbin Plot (4-panel) — Online vs Offline Spend density, Avg Price vs Online Spend, Tenure vs Online Spend, and Quantity vs Online Spend

Observation: The 4-panel hexbin analysis reveals density patterns across key variable pairs. Top-left: the highest concentration (darkest red) occurs at Offline_Spend \$2,500–\$3,000 paired with Online_Spend \$1,500–\$2,000 — the modal customer profile. Top-right: Avg Price clusters sharply below \$50, confirming the low-price majority of transactions regardless of Online_Spend level. Bottom-left: Tenure vs Online_Spend shows no directional trend — spending is uniform across all tenure lengths (0–50 months), indicating no loyalty effect on online spend. Bottom-right: Quantity is extremely concentrated below 10 units, with Online_Spend spread uniformly, reflecting that most customers buy in small quantities at varying price points.

Strip Plot

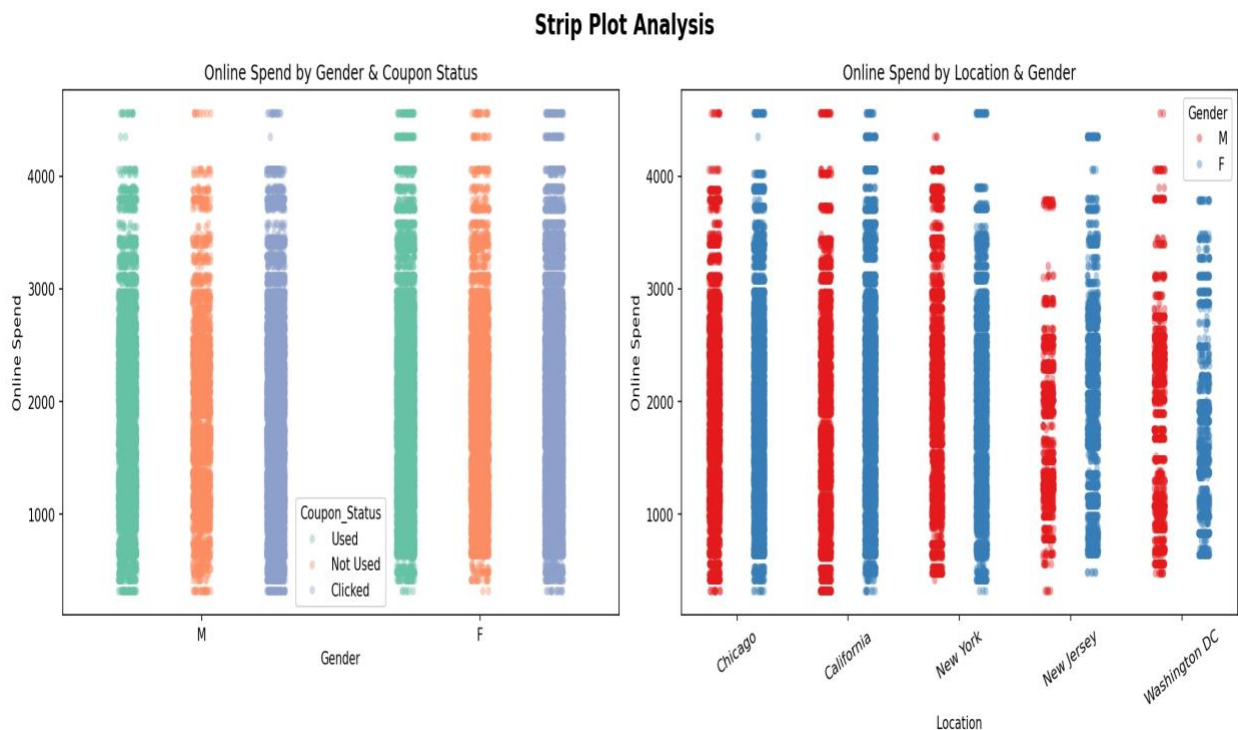


Figure 10.22: Strip Plot — Avg Price by Top 6 Product Categories and Gender (dodge=True, alpha=0.3)

Observation: The strip plot shows clear price tier clustering within each of the top 6 product categories. Nest has the widest spread (\$50–\$350) with distinct horizontal bands at \$120 and \$150, confirming multiple product lines at premium price points. Apparel and Office items concentrate below \$25. Nest-USA shows a tight cluster at ~\$120. Male and female customers purchase at identical price points within each category, confirming gender does not influence product price selection. The `dodge=True` parameter separates M/F columns clearly for comparison.

Swarm Plot

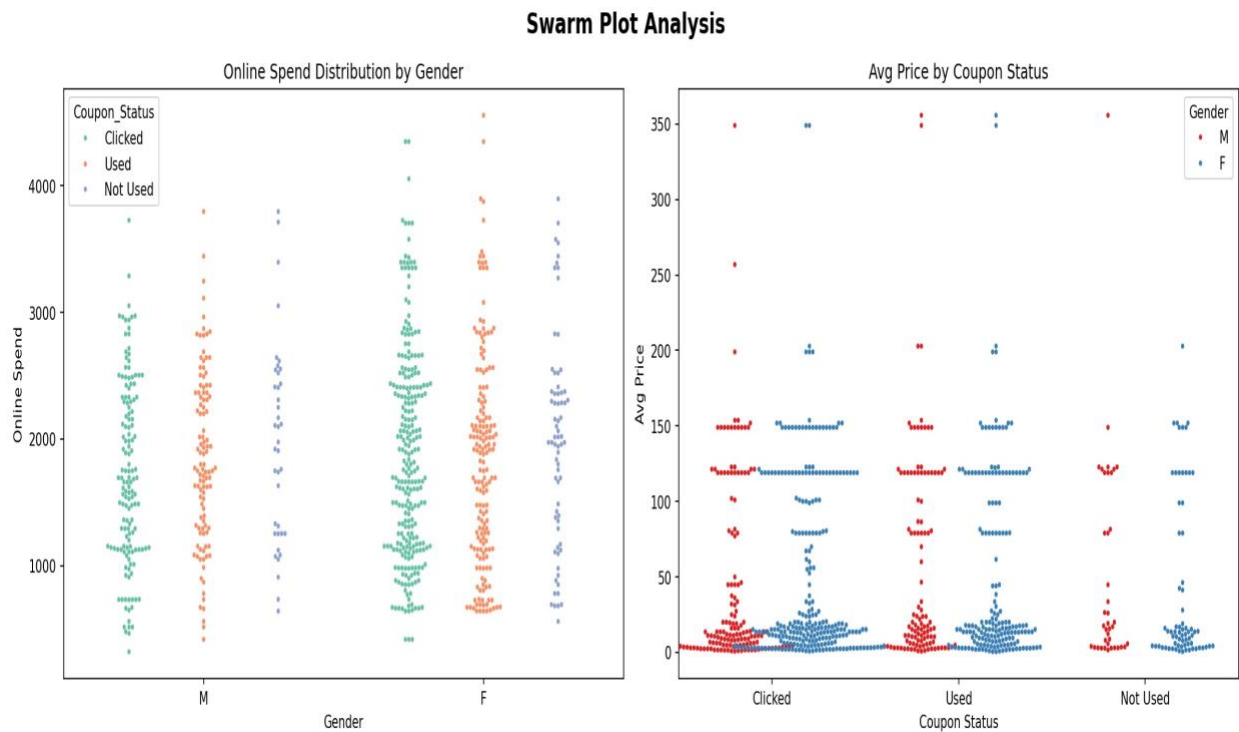


Figure 10.23: Swarm Plot — Delivery Charges by Location and Gender (800-sample, ordered by transaction count)

Observation: The swarm plot (800-sample subset) reveals that most `Delivery_Charges` cluster near \$0–\$15 across all five cities, confirming the IQR finding that the majority of transactions incur low shipping costs. Chicago and California, being the highest-volume cities, show the densest point distributions. The extreme outliers (`Delivery_Charges` above \$100, reaching ~\$521) are individually visible as isolated points at the top of each city column, confirming they are exceptional cases rather than systematic patterns. Male and female customers show identical delivery charge distributions within each city.

10.3 Proposal Insight Plots

GST vs Avg Price — Negative Correlation

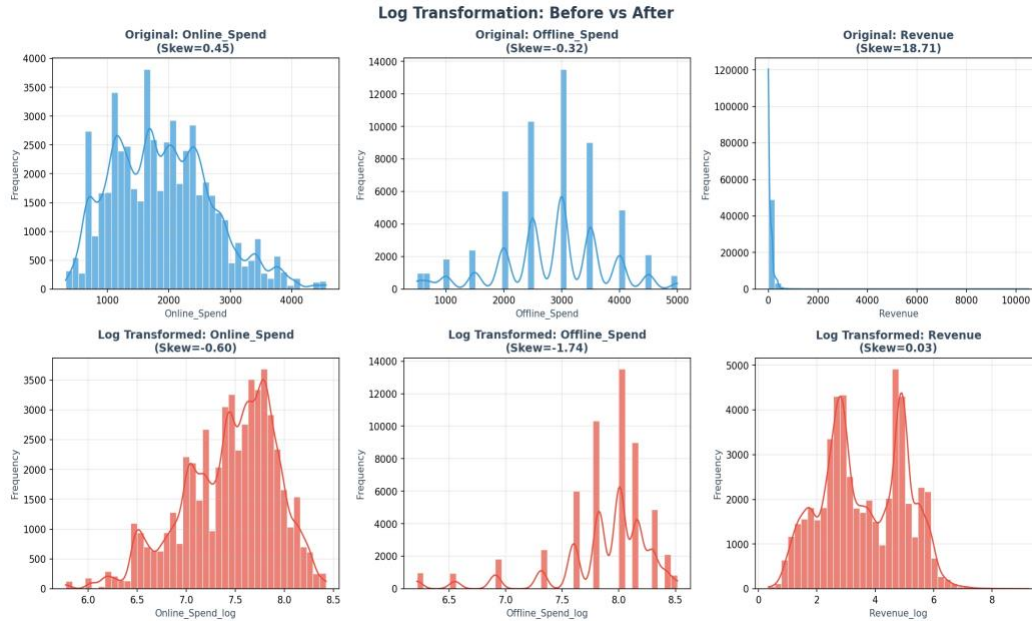


Figure 10.24: GST vs Avg Price — Scatter+Regression ($r=-0.58$) and Mean GST by Price Bracket

Observation: The scatter plot with regression line clearly shows the strong negative correlation ($r=-0.58$) between GST rate and price. The bar chart confirms that items priced \$0-\$25 carry an average GST of 0.16%, dropping to 0.05% for items above \$200. This reveals a progressive tax structure where luxury items receive favorable tax treatment.

Online vs Offline Spend — Distribution Comparison

The figure contains two charts related to Principal Component Analysis (PCA):

- Scree Plot (Explained Variance per PC):** A bar chart showing the percentage of variance explained by each of the 10 principal components. The y-axis is 'Explained Variance (%)' ranging from 0 to 25. The x-axis is 'Principal Component' from 1 to 10. The bars are colored and labeled with their respective variance percentages: PC1 (24.2%, teal), PC2 (18.8%, orange), PC3 (14.0%, blue), PC4 (10.2%, pink), PC5 (9.7%, green), PC6 (8.8%, yellow), PC7 (6.6%, tan), PC8 (5.0%, grey), PC9 (2.6%, dark green), and PC10 (0.0%, light green).
- Cumulative Explained Variance:** A line plot showing the cumulative percentage of variance explained as more principal components are added. The y-axis is 'Cumulative Variance (%)' ranging from 30 to 100. The x-axis is 'Number of Components' from 1 to 10. A red line with circular markers connects the data points, which are labeled with their cumulative variance values: 24.2%, 43.0%, 57.0%, 67.3%, 77.0%, 85.8%, 92.4%, 97.4%, 100.0%, and 100.0%. Two horizontal dashed lines indicate thresholds: a blue line at 95% and a green line at 80%.

Observation: The KDE overlay shows that `Offline_Spend` (red) has a much narrower, peaked distribution centered around \$2,500-\$3,000, while `Online_Spend` (blue) is broader and more uniform. The statistics table confirms: Online mean \$1,893 vs Offline mean \$2,831; Online is slightly right-skewed (0.45) while Offline is slightly left-skewed (-0.32).

[illegible]

Observation: The box plot shows Nest and Nest-Canada have the highest median prices (\$120-\$150+), while most categories cluster below \$25. The IQR chart reveals Gift Cards (\$225 IQR) and Nest (\$180 IQR)

have the most price variation. Nest-USA (shown in red) has moderate IQR (\$30) despite high median, indicating a more focused product lineup at premium price points.

Monthly Revenue Heatmap — Seasonality

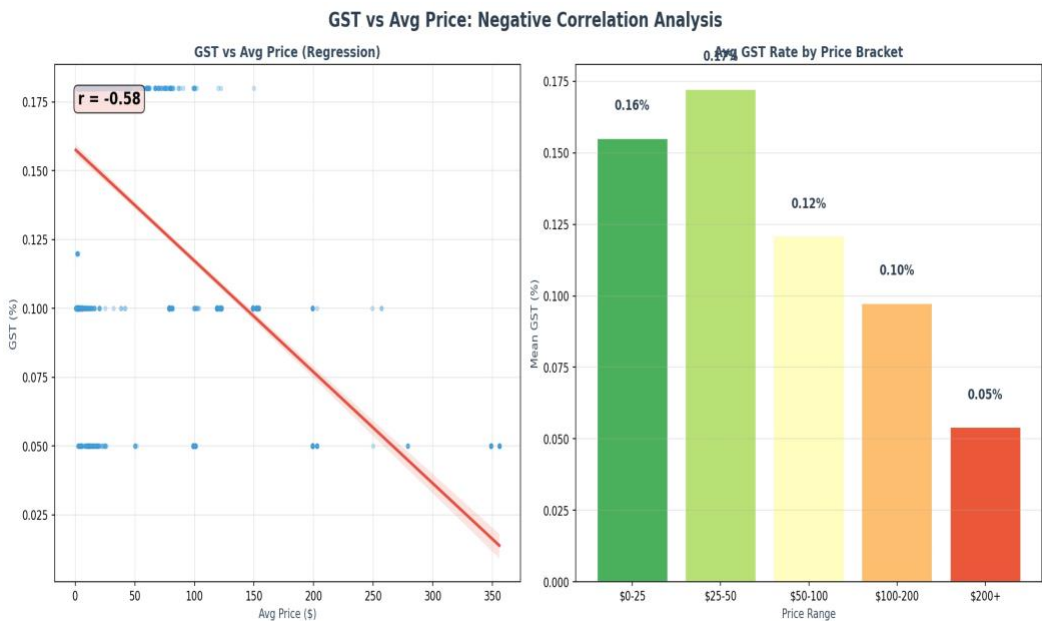


Figure 10.27: Seasonal Revenue Analysis — Normalized Heatmap and Monthly Trend by Category

Observation: The heatmap shows category-specific seasonal patterns. Nest only appears in months 8-12 (showing 0.00 in months 1-7), suggesting it was launched mid-year. Nest-USA peaks in January (1.00 = max). Apparel peaks in April (1.00). The line chart below confirms Nest-USA generates the highest absolute revenue throughout the year, with Apparel showing a secondary April spike.

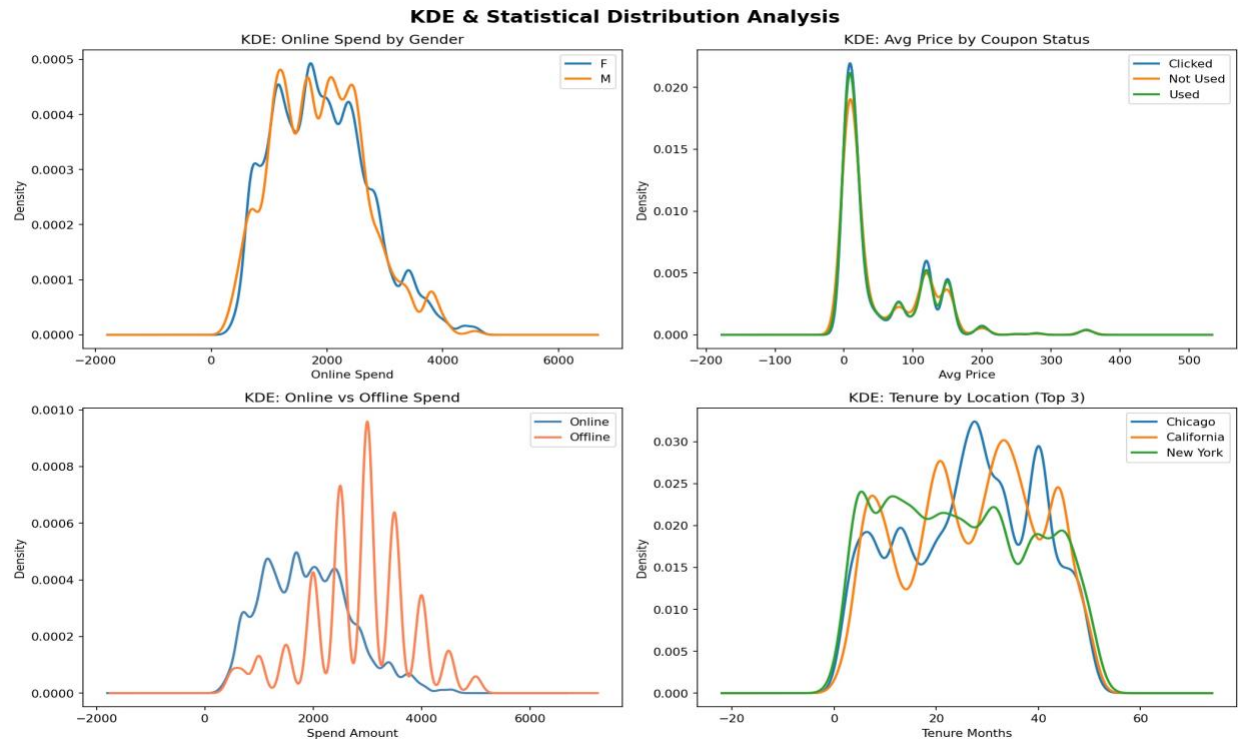


Figure 10.28: KDE & Statistical Distribution Analysis (4-panel) — Online Spend by Gender, Avg Price by Coupon Status, Online vs Offline Spend, Tenure by Location

Observation: The multivariate KDE analysis reveals four distinct insights. Top-left: Online_Spend distributions for female (F) and male (M) customers are nearly identical in shape and peak positions, confirming gender has minimal impact on spending level; the female curve shows slightly higher density reflecting their 62% majority. Top-right: Avg_Price KDE curves for all three coupon statuses (Used, Not Used, Clicked) overlap almost perfectly, with a dominant peak near \$0–\$5 and secondary peaks at ~\$120 and ~\$150 — indicating coupon behavior is not driven by product price point. Bottom-left: Offline_Spend (orange) is higher in absolute value than Online_Spend (blue) and exhibits a sharp multimodal pattern with distinct peaks at fixed increments (~\$1,000, \$2,000, \$3,000), reflecting a tiered pricing structure; Online_Spend is lower and more broadly distributed. Bottom-right: Tenure distributions for Chicago (blue), California (orange), and New York (green) show notably different patterns — California peaks earlier (~20 months) while Chicago extends to higher densities in the 30–50 month range, suggesting Chicago retains customers longer on average.

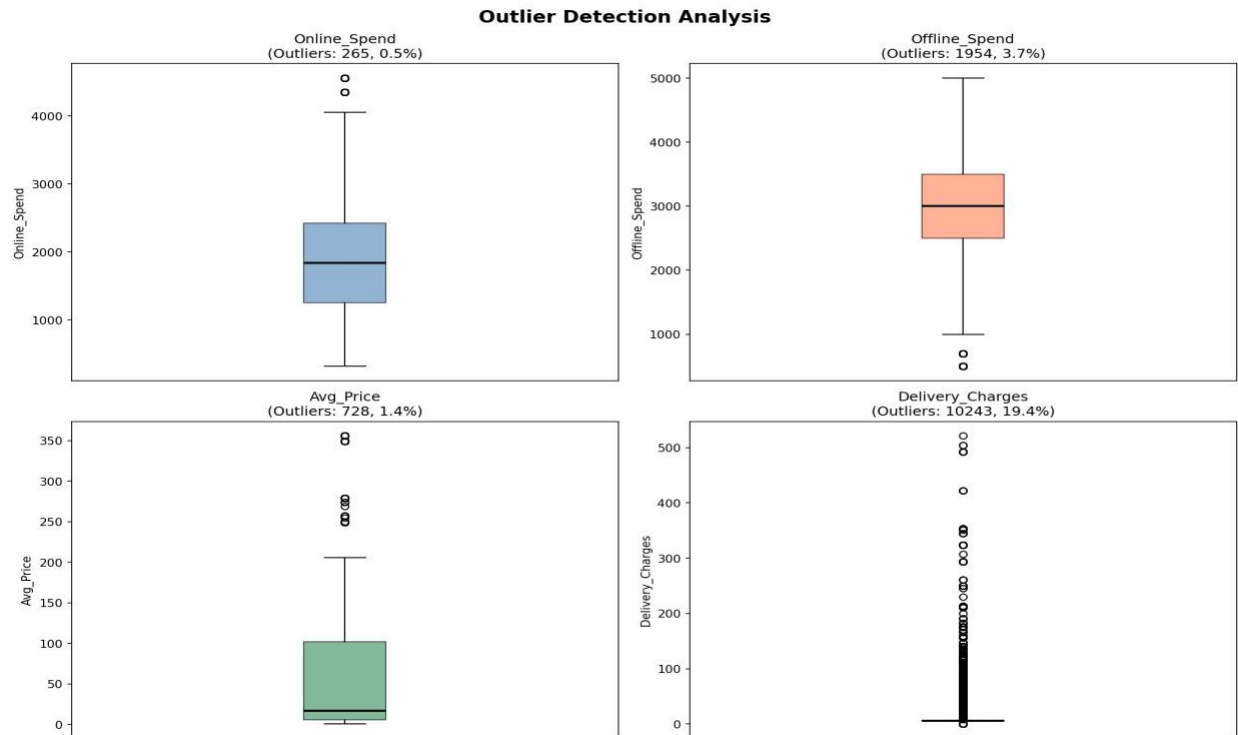


Figure 10.29: Outlier Detection Analysis (4-panel) — IQR-based boxplots for *Online_Spend* (0.5%), *Offline_Spend* (3.7%), *Avg_Price* (1.4%), and *Delivery_Charges* (19.4%)

Observation: The outlier detection using the IQR method reveals that *Delivery_Charges* has the highest outlier rate at 19.4%, suggesting significant variation in shipping costs across transactions. *Online_Spend* is the most stable feature with only 0.5% outliers, while *Offline_Spend* (3.7%) and *Avg_Price* (1.4%) show moderate outlier presence. These outliers were retained in the analysis as they represent genuine high-value transactions rather than data errors.

11. Interactive Dashboard

11.1 Architecture & Components

Component Category	Components Used
Layout	dbc.Container, dbc.Row, dbc.Col, dbc.Card, dbc.CardBody, dbc.CardHeader
Navigation	dbc.NavbarSimple, dbc.NavItem, dbc.Tabs, dbc.Tab
Input Controls	dbc.Select, dbc.RadioItems, dbc.Button, dbc.Checklist, dbc.DropdownMenu, dbc.DropdownMenuItem
Range Controls	dcc.Slider, dcc.RangeSlider
Feedback	dbc.Alert, dbc.Badge, dbc.Tooltip, dbc.Spinner
DAQ Controls	daq.Knob, daq.Gauge, daq.Thermometer, daq.LEDDisplay
DAQ Indicators	daq.BooleanSwitch, daq.ColorPicker, daq.GraduatedBar

Component Category	Components Used
Graph & Data	dcc.Graph, dcc.Download, dcc.Textarea, dcc.Store
HTML Elements	html.H1-H6, html.Div, html.Label, html.Br

Table 11.1: All dbc, dcc, and DAQ Components Used

11.2 Dashboard Screenshots

Tab 1: Load Data

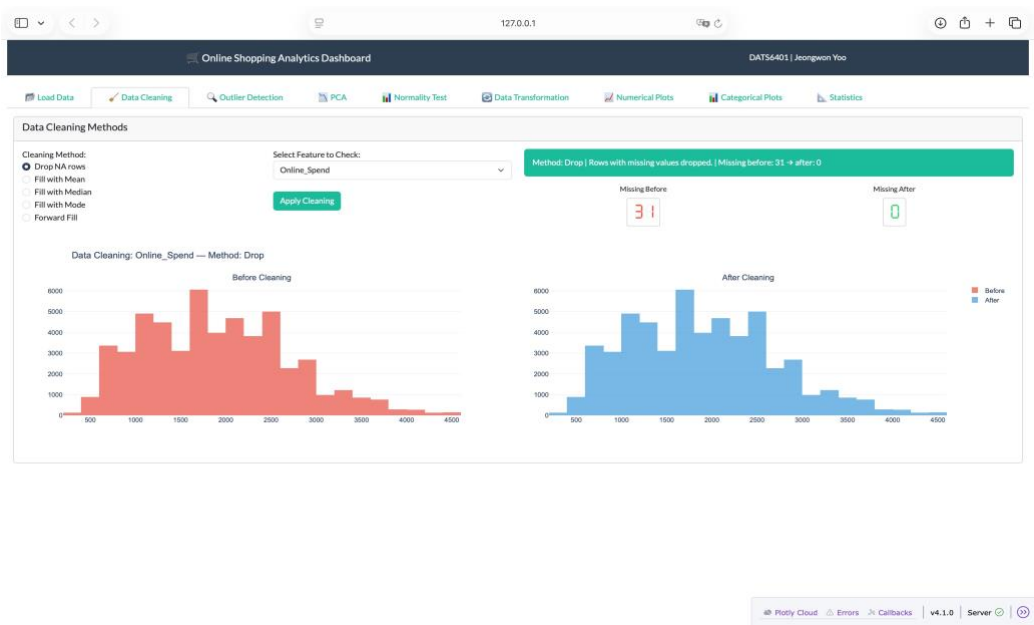


Figure 11.1: Load Data Tab — Dataset overview with badges, interactive data table, and statistics

Tab 2: Data Cleaning

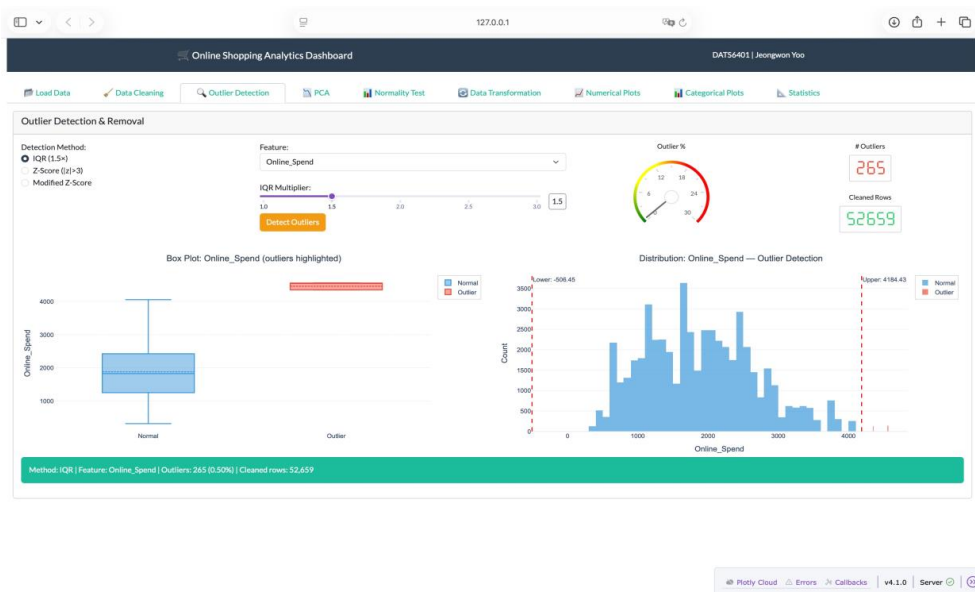


Figure 11.2: Data Cleaning Tab — 5 methods, LED displays, before/after histograms

Tab 3: Outlier Detection

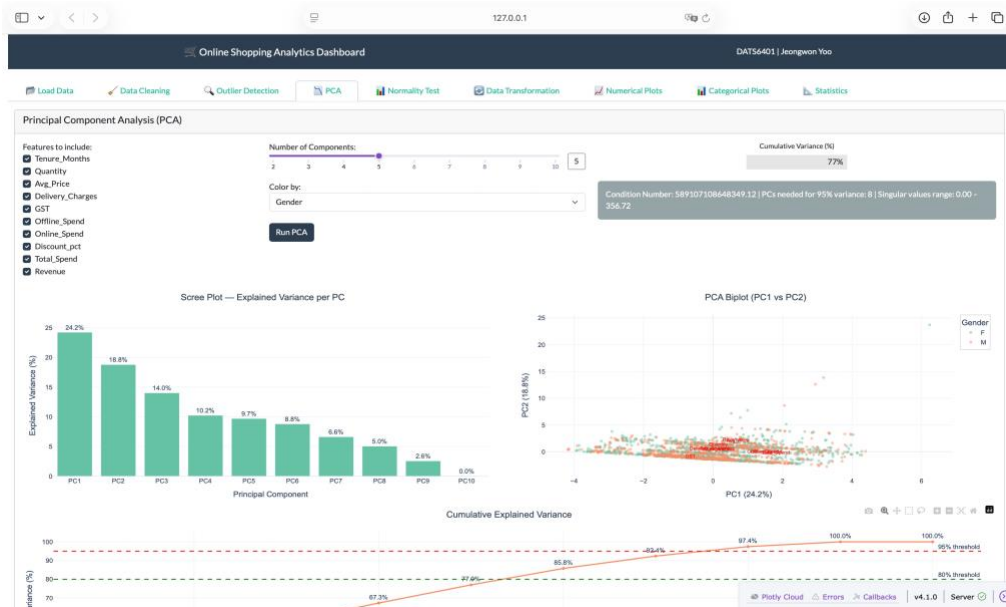


Figure 11.3: Outlier Detection Tab — IQR/Z-Score methods, Gauge, LED, box/histogram comparison

Tab 4: PCA

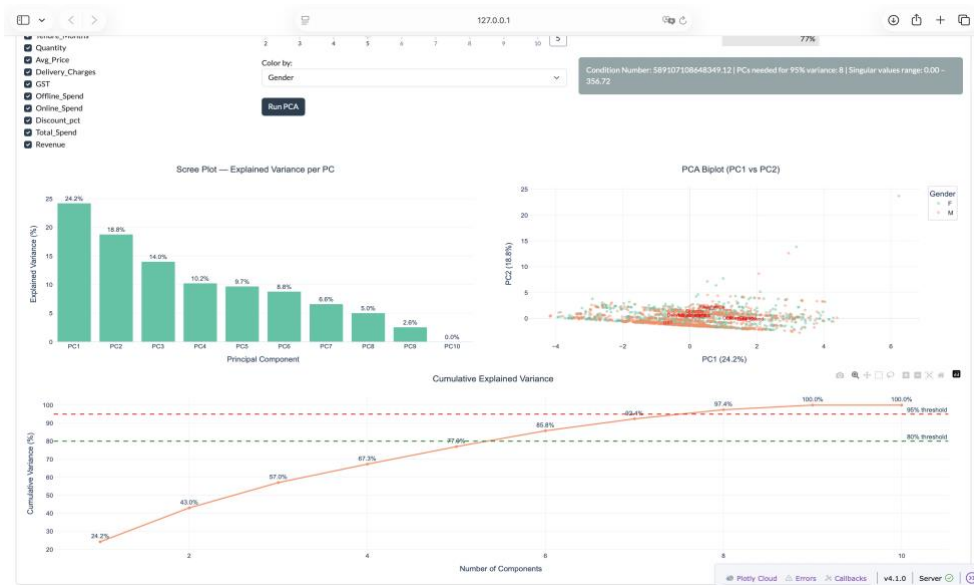


Figure 11.4: PCA Tab — Scree plot, Biplot, Cumulative variance with GraduatedBar

Tab 5: Normality Test

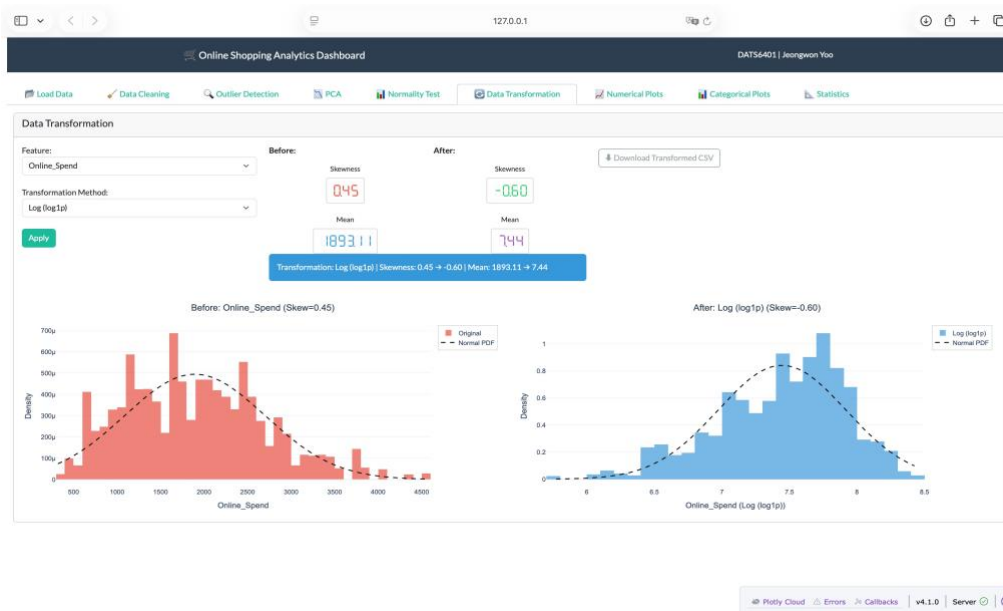


Figure 11.5: Normality Test Tab — Thermometers, Knob, BooleanSwitch, ColorPicker

Tab 6: Data Transformation

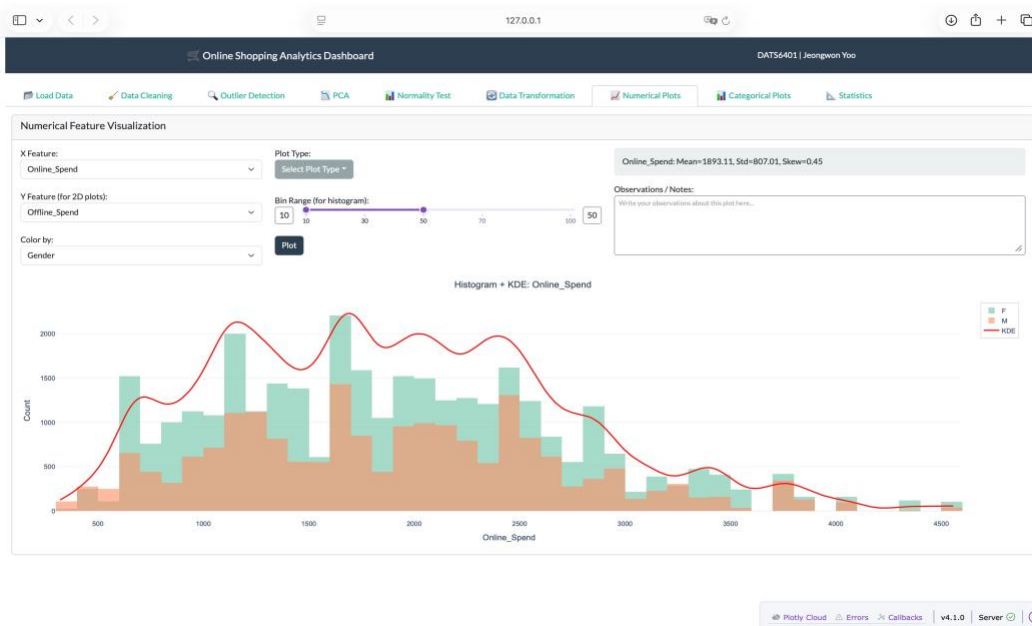


Figure 11.6: Data Transformation Tab — LED displays, before/after histograms, CSV download

Tab 7: Numerical Plots

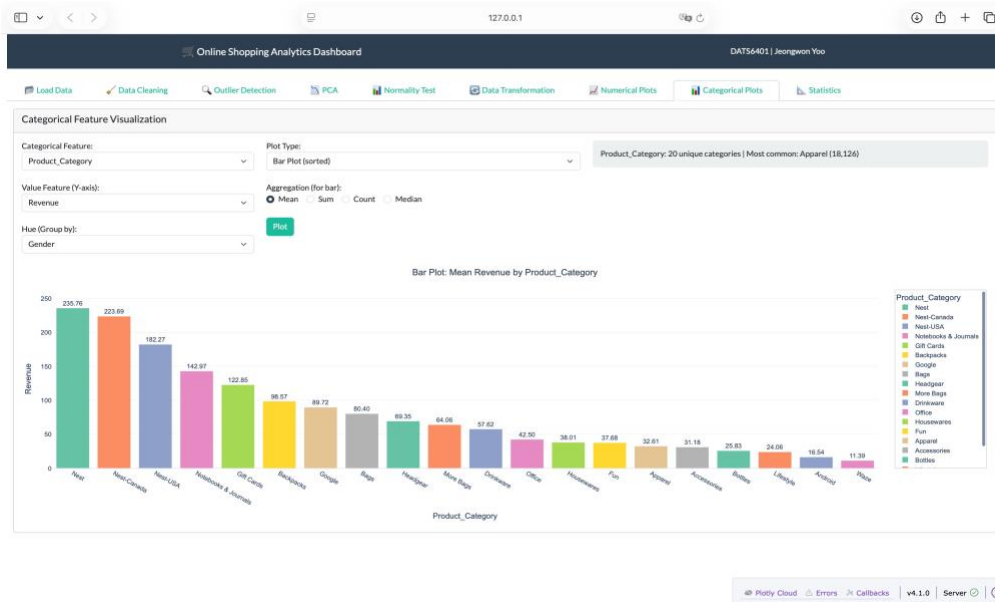


Figure 11.7: Numerical Plots Tab — 13 plot types via DropdownMenu, RangeSlider, Textarea

Tab 8: Categorical Plots

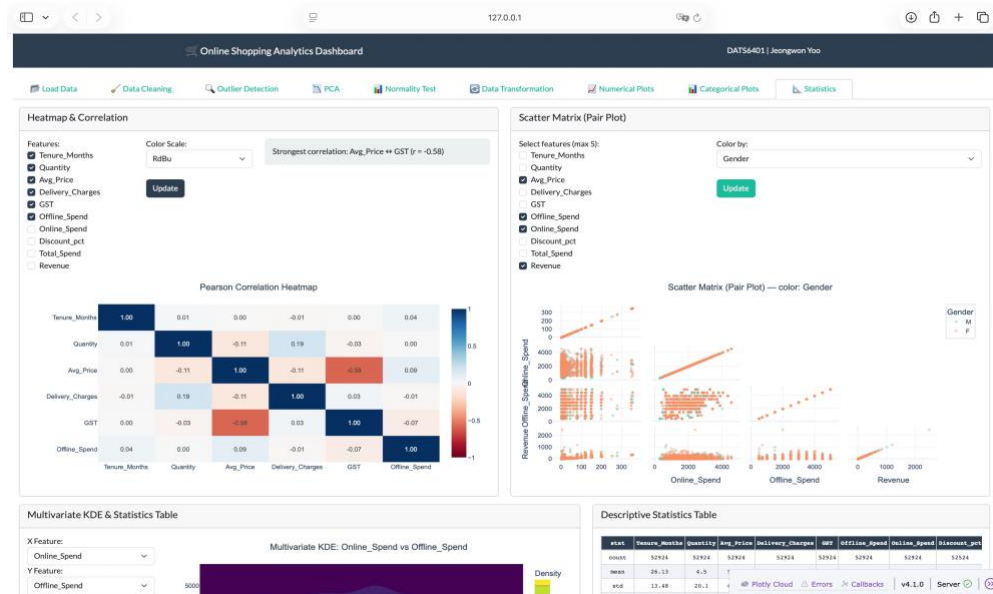


Figure 11.8: Categorical Plots Tab — 10 plot types, sorted bar chart with value labels

Tab 9: Statistics

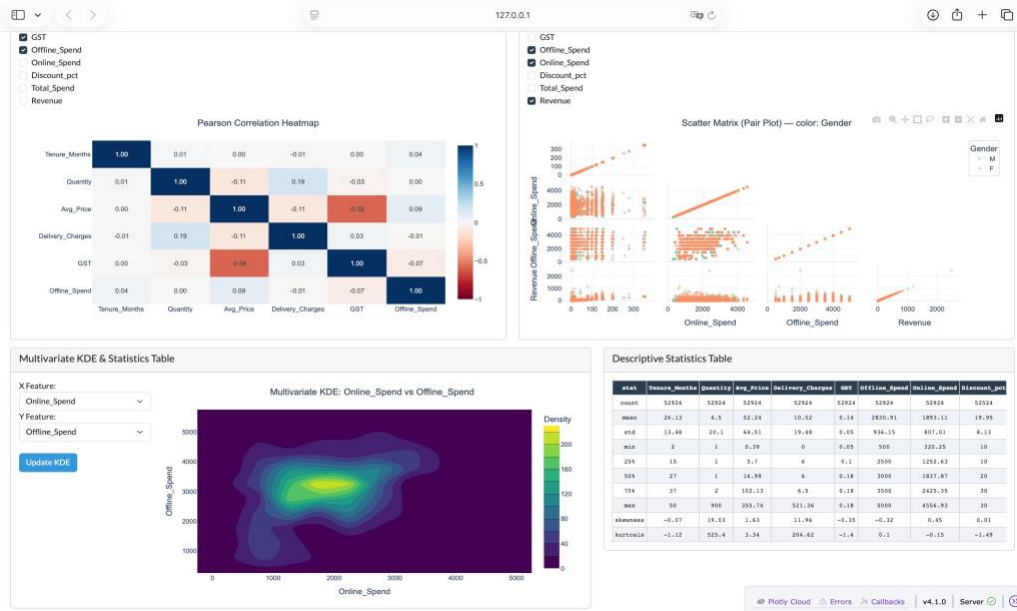


Figure 11.9: Statistics Tab — Heatmap, Pair Plot, Multivariate KDE, Statistics Table

12. Conclusions

12.1 Key Learnings from Graphs

- GST shows a strong negative correlation with Avg_Price ($r=-0.58$) — lower-priced items carry higher effective tax rates, revealing a tiered tax structure.
- Online_Spend follows a near-uniform multimodal distribution (\$320-\$4,557), suggesting customers cluster at subscription or bundle price tiers.
- Offline spending (\$2,831 mean) consistently outperforms online spending (\$1,893 mean) across all cities and months.
- Product category composition is nearly identical across all cities (Apparel 33-36%, Nest-USA 25-27%), indicating national rather than local preferences.
- Coupon conversion gap: 50.88% click vs 33.83% redeem — a 17% gap representing a significant business opportunity.
- The Nest product family generates 3-7x higher mean revenue than other categories, making it the primary revenue driver.

12.2 Dashboard Utility

The interactive dashboard enables non-technical stakeholders to explore the dataset without writing code. Real-time filtering, method selection, before/after comparisons, and CSV export make it a practical analytical tool for business decision-making.

12.3 User-Friendliness & Functionality

The dashboard follows a logical analytical flow across 9 tabs, using color-coded alerts, DAQ indicators, and descriptive tooltips to guide users. All required dbc, dcc, and DAQ components are implemented with 2-decimal float precision throughout.

References

1. Plotly Technologies Inc. (2023). Plotly Python Graphing Library. <https://plotly.com/python/>
2. Dash by Plotly. (2023). Dash User Guide. <https://dash.plotly.com/>
3. dash-bootstrap-components. (2023). Documentation. <https://dash-bootstrap-components.opensource.faculty.ai/>
4. dash-daq. (2023). Dash DAQ Documentation. <https://dash.plotly.com/dash-daq>
5. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825-2830.
6. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proc. 9th Python in Science Conf.
7. Virtanen, P., et al. (2020). SciPy 1.0: Fundamental Algorithms. Nature Methods, 17, 261-272.
8. Seaborn. (2023). Statistical Data Visualization. <https://seaborn.pydata.org/>

APPENDIX

Phase 1

```
###
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from prettytable import PrettyTable
import statsmodels.api as sm
from mpl_toolkits.mplot3d import Axes3D
import warnings
warnings.filterwarnings('ignore')

###
url = 'online_shopping.csv'
df = pd.read_csv(url)

print(df.tail())
print(df.info())
### missing values
print(df.isnull().sum())

col31 = ['CustomerID', 'Gender', 'Location', 'Tenure_Months',
         'Transaction_ID', 'Quantity', 'Avg_Price', 'Delivery_Charges',
         'Coupon_Status', 'GST', 'Offline_Spend', 'Online_Spend']
df.dropna(subset=col31, inplace=True)

print(df.isnull().sum())

### incoding and datetime

df['Transaction_Date'] = pd.to_datetime(df['Transaction_Date'])
df['CustomerID'] = df['CustomerID'].astype(int)
df['Transaction_ID'] = df['Transaction_ID'].astype(int)

###
```

```

df['Total_Spend'] = df['Offline_Spend'] + df['Online_Spend']
df['Revenue'] = df['Quantity'] * df['Avg_Price']

cat_cols = ['Gender', 'Location', 'Product_Category', 'Coupon_Status']
num_cols = ['Tenure_Months', 'Quantity', 'Avg_Price', 'Delivery_Charges',
            'GST', 'Offline_Spend', 'Online_Spend', 'Discount_pct', 'Total_Spend', 'Revenue']

###
df = df.drop(columns=['Unnamed: 0'])

print(f'{df.head()}\n{df.info()}\n{df.describe()}')

###
t4 = PrettyTable()
t4.title = 'Pearson Correlation'
sel = ['Avg_Price', 'Online_Spend', 'Offline_Spend', 'Delivery_Charges', 'Revenue']
corr_sel = df[sel].corr().round(2)
t4.field_names = [''] + sel
for idx in corr_sel.index:
    t4.add_row([idx] + [f'{v:.2f}' for v in corr_sel.loc[idx]])
print(t4)
###
PALETTE = 'Set2'
TITLE_COLOR = '#2c3e50'
LABEL_COLOR = '#34495e'
C_F = '#e74c3c'
C_M = '#3498db'
# =====
# PLOT 1: Bar Plot - Sorted (Product Category)
# =====
print('\n--- Plot 1: Sorted Bar Plot (Product Category Count) ---')
fig, ax = plt.subplots(figsize=(14, 7))
vc = df['Product_Category'].value_counts().sort_values(ascending=True)
colors = sns.color_palette(PALETTE, len(vc))
bars = ax.barh(vc.index, vc.values, color=colors)
for bar, val in zip(bars, vc.values):
    ax.text(val + 50, bar.get_y() + bar.get_height() / 2,
            f'{val:,}', va='center', fontsize=9, color=LABEL_COLOR)
ax.set_xlabel('Transaction Count', color=LABEL_COLOR)

```

```
ax.set_ylabel('Product Category', color=LABEL_COLOR)
ax.set_title('Product Category by Transaction Count (Sorted)', color=TITLE_COLOR, fontweight='bold')
ax.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()

# =====
# PLOT 2: Stacked Bar Plot (Gender × Location)
# =====
print('\n--- Plot 2: Stacked Bar Plot (Gender × Location) ---')
fig, ax = plt.subplots(figsize=(10, 6))
ct = pd.crosstab(df['Location'], df['Gender'])
ct = ct.loc[ct.sum(axis=1).sort_values(ascending=False).index]
ct.plot(kind='bar', stacked=True, ax=ax, color=[C_F, C_M], edgecolor='white')
for c in ax.containers:
    ax.bar_label(c, fmt='%d', label_type='center', fontsize=9, color='white', fontweight='bold')
ax.set_title('Transactions by Location & Gender (Stacked)', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Location', color=LABEL_COLOR)
ax.set_ylabel('Count', color=LABEL_COLOR)
ax.legend(title='Gender');
ax.grid(axis='y', alpha=0.3)
plt.xticks(rotation=30);
plt.tight_layout()
plt.show()

# =====
# PLOT 3: Grouped Bar Plot (Coupon Status × Gender)
# =====
print('\n--- Plot 3: Grouped Bar Plot (Coupon Status × Gender) ---')
fig, ax = plt.subplots(figsize=(10, 6))
ct2 = pd.crosstab(df['Coupon_Status'], df['Gender'])
ct2.plot(kind='bar', ax=ax, color=[C_F, C_M], edgecolor='white')
for c in ax.containers:
    ax.bar_label(c, fmt='%d', fontsize=9, padding=2)
ax.set_title('Coupon Status by Gender (Grouped)', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Coupon Status', color=LABEL_COLOR)
ax.set_ylabel('Count', color=LABEL_COLOR)
ax.legend(title='Gender');
ax.grid(axis='y', alpha=0.3)
plt.xticks(rotation=0);
```

```

plt.tight_layout()
plt.show()
# =====
# PLOT 4: Count Plot (Location with Gender hue)
# =====
print('\n--- Plot 4: Count Plot (Location) ---')
fig, ax = plt.subplots(figsize=(10, 6))
order = df['Location'].value_counts().index
sns.countplot(data=df, x='Location', hue='Gender', order=order,
               palette=[C_F, C_M], ax=ax)
for c in ax.containers:
    ax.bar_label(c, fmt='%d', fontsize=9, padding=2)
ax.set_title('Count Plot: Transactions by Location & Gender', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Location', color=LABEL_COLOR)
ax.set_ylabel('Count', color=LABEL_COLOR)
ax.legend(title='Gender');
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
# =====
# PLOT 5: Pie Chart (Gender)
# =====
print('\n--- Plot 5: Pie Chart (Gender) ---')
fig, ax = plt.subplots(figsize=(8, 8))
vc_g = df['Gender'].value_counts()
wedges, texts, autotexts = ax.pie(
    vc_g.values, labels=vc_g.index, autopct='%1.2f%%',
    colors=[C_F, C_M], startangle=90,
    textprops={'fontsize': 13}, pctdistance=0.75,
    wedgeprops={'edgecolor': 'white', 'linewidth': 2})
for t in autotexts: t.set_fontweight('bold')
ax.set_title('Gender Distribution', color=TITLE_COLOR, fontweight='bold', fontsize=16)
plt.tight_layout()
plt.show()
# =====
# PLOT 6: Pie Chart (Coupon Status)
# =====
print('\n--- Plot 6: Pie Chart (Coupon Status) ---')
fig, ax = plt.subplots(figsize=(8, 8))

```

```

vc_c = df['Coupon_Status'].value_counts()
wedges, texts, autotexts = ax.pie(
    vc_c.values, labels=vc_c.index, autopct='%1.2f%%',
    colors=sns.color_palette('Set2', 3), startangle=90,
    textprops={'fontsize': 13}, pctdistance=0.75,
    wedgeprops={'edgecolor': 'white', 'linewidth': 2})
for t in autotexts: t.set_fontweight('bold')
ax.set_title('Coupon Status Distribution', color=TITLE_COLOR, fontweight='bold', fontsize=16)
plt.tight_layout()
plt.show()

# =====
# PLOT 7: Line Plot (Monthly Avg Spend)
# =====
print('\n--- Plot 7: Line Plot (Monthly Avg Online vs Offline Spend) ---')
fig, ax = plt.subplots(figsize=(12, 6))
monthly = df.groupby('Month')[['Online_Spend', 'Offline_Spend']].mean()
ax.plot(monthly.index, monthly['Online_Spend'], 'o-', color=C_F,
        linewidth=2.5, markersize=8, label='Online Spend')
ax.plot(monthly.index, monthly['Offline_Spend'], 's--', color=C_M,
        linewidth=2.5, markersize=8, label='Offline Spend')
for i, row in monthly.iterrows():
    ax.annotate(f'{row["Online_Spend"]:.2f}', (i, row['Online_Spend']),
               textcoords="offset points", xytext=(0, 12), fontsize=8, ha='center', color=C_F)
    ax.annotate(f'{row["Offline_Spend"]:.2f}', (i, row['Offline_Spend']),
               textcoords="offset points", xytext=(0, -15), fontsize=8, ha='center', color=C_M)
ax.set_xlabel('Month', color=LABEL_COLOR);
ax.set_ylabel('Avg Spend ($)', color=LABEL_COLOR)
ax.set_title('Monthly Average Online vs Offline Spend', color=TITLE_COLOR, fontweight='bold')
ax.set_xticks(range(1, 13));
ax.legend();
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# =====
# PLOT 8: Dist Plot (Online Spend)
# =====
print('\n--- Plot 8: Dist Plot (Online Spend) ---')
fig, ax = plt.subplots(figsize=(10, 6))
sns.histplot(df['Online_Spend'], kde=True, color=C_M, bins=40,

```

```

        edgecolor='white', alpha=0.7, ax=ax)
mean_os = df['Online_Spend'].mean()
ax.axvline(mean_os, color=C_F, linestyle='--', linewidth=2, label=f'Mean: ${mean_os:.2f}')
ax.set_title('Distribution of Online Spend', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Online Spend ($)')
ax.set_ylabel('Frequency', color=LABEL_COLOR)
ax.legend();
ax.grid(alpha=0.3);
plt.tight_layout()
plt.show()
# =====
# PLOT 9: Histogram + KDE (Avg Price)
# =====
print('\n--- Plot 9: Histogram + KDE (Avg Price) ---')
fig, ax = plt.subplots(figsize=(10, 6))
sns.histplot(df['Avg_Price'], kde=True, bins=50, color='#2ecc71',
             edgecolor='white', alpha=0.7, stat='density', ax=ax)
m = df['Avg_Price'].mean()
ax.axvline(m, color=C_F, linestyle='--', linewidth=2, label=f'Mean: ${m:.2f}')
ax.set_title('Avg Price Distribution with KDE', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Avg Price ($)')
ax.set_ylabel('Density', color=LABEL_COLOR)
ax.legend();
ax.grid(alpha=0.3);
plt.tight_layout()
plt.show()
# =====
# PLOT 10: KDE Plot with fill (Online Spend by Gender)
# =====
print('\n--- Plot 10: KDE Plot with fill (Online Spend by Gender) ---')
fig, ax = plt.subplots(figsize=(10, 6))
sns.kdeplot(data=df, x='Online_Spend', hue='Gender', fill=True,
            alpha=0.4, palette=[C_F, C_M], linewidth=2.5, ax=ax)
ax.set_title('KDE of Online Spend by Gender', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Online Spend ($)')
ax.set_ylabel('Density', color=LABEL_COLOR)
ax.grid(alpha=0.3);
plt.tight_layout()
plt.show()

```

```

# =====
# PLOT 11: QQ-Plot (Online Spend)
# =====
print('\n--- Plot 11: QQ-Plot (Online Spend) ---')
fig, ax = plt.subplots(figsize=(8, 8))
sm.qqplot(df['Online_Spend'].dropna(), line='45', ax=ax,
          markerfacecolor=C_M, markeredgecolor='white', alpha=0.5, markersize=4)
ax.set_title('QQ-Plot of Online Spend', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Theoretical Quantiles', color=LABEL_COLOR)
ax.set_ylabel('Sample Quantiles', color=LABEL_COLOR)
ax.grid(alpha=0.3);
plt.tight_layout()
plt.show()
# =====
# PLOT 12: Reg Plot (Online vs Offline Spend)
# =====
print('\n--- Plot 12: Reg Plot (Online vs Offline Spend) ---')
fig, ax = plt.subplots(figsize=(10, 7))
samp = df.sample(3000, random_state=42)
sns.regplot(data=samp, x='Offline_Spend', y='Online_Spend',
            scatter_kws={'alpha': 0.3, 'color': C_M, 's': 15},
            line_kws={'color': C_F, 'linewidth': 2.5}, ax=ax)
r_val = df[['Online_Spend', 'Offline_Spend']].corr().iloc[0, 1]
ax.text(0.05, 0.95, f'r = {r_val:.2f}', transform=ax.transAxes,
        fontsize=14, va='top', bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))
ax.set_title('Online vs Offline Spend (Regression)', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Offline Spend ($)', color=LABEL_COLOR)
ax.set_ylabel('Online Spend ($)', color=LABEL_COLOR)
ax.grid(alpha=0.3);
plt.tight_layout()
plt.show()
# =====
# PLOT 13: Pair Plot
# =====
print('\n--- Plot 13: Pair Plot ---')
pair_cols = ['Avg_Price', 'Online_Spend', 'Offline_Spend', 'Delivery_Charges']
g = sns.pairplot(df[pair_cols + ['Gender']].sample(2000, random_state=42),
                 hue='Gender', palette=[C_F, C_M],
                 diag_kind='kde', plot_kws={'alpha': 0.4, 's': 15})

```

```

g.figure.suptitle('Pair Plot: Price, Spend & Delivery by Gender',
                  y=1.02, color=TITLE_COLOR, fontweight='bold', fontsize=16)
plt.show()
# =====
# PLOT 14: Heatmap with cbar
# =====
print('\n--- Plot 14: Heatmap (Pearson Correlation) ---')
fig, ax = plt.subplots(figsize=(12, 9))
corr = df[num_cols].corr().round(2)
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)
sns.heatmap(corr, annot=True, fmt='.2f', cmap='RdBu_r', center=0,
            mask=mask, linewidths=1, ax=ax, cbar=True,
            annot_kws={'fontsize': 10}, cbar_kws={'label': 'Pearson r'})
ax.set_title('Pearson Correlation Heatmap', color=TITLE_COLOR, fontweight='bold')
plt.xticks(rotation=45, ha='right');
plt.tight_layout()
plt.show()
# =====
# PLOT 15: Multivariate Box Plot
# =====
print('\n--- Plot 15: Multivariate Box Plot (Spend by Location) ---')
fig, ax = plt.subplots(figsize=(12, 6))
melt = df.melt(id_vars='Location', value_vars=['Online_Spend', 'Offline_Spend'],
               var_name='Type', value_name='Amount')
sns.boxplot(data=melt, x='Location', y='Amount', hue='Type',
            palette=[C_M, C_F], ax=ax, order=df['Location'].value_counts().index)
ax.set_title('Online vs Offline Spend by Location (Box Plot)', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Location', color=LABEL_COLOR)
ax.set_ylabel('Spend ($)', color=LABEL_COLOR)
ax.legend(title='Type');
ax.grid(axis='y', alpha=0.3);
plt.tight_layout()
plt.show()
# =====
# PLOT 16: Multivariate Boxen Plot
# =====
print('\n--- Plot 16: Boxen Plot (Online Spend by Coupon × Gender) ---')
fig, ax = plt.subplots(figsize=(10, 6))
sns.boxenplot(data=df, x='Coupon_Status', y='Online_Spend', hue='Gender',

```

```

        palette=[C_F, C_M], ax=ax)
ax.set_title('Online Spend by Coupon Status & Gender (Boxen)', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Coupon Status', color=LABEL_COLOR)
ax.set_ylabel('Online Spend ($)', color=LABEL_COLOR)
ax.legend(title='Gender');
ax.grid(axis='y', alpha=0.3);
plt.tight_layout()
plt.show()
# =====
# PLOT 17: Area Plot (Monthly Revenue by Top 5 Categories)
# =====
print('\n--- Plot 17: Area Plot (Monthly Revenue Top 5 Categories) ---')
top5 = df['Product_Category'].value_counts().head(5).index.tolist()
monthly_cat = df[df['Product_Category'].isin(top5)].groupby(
    ['Month', 'Product_Category'])['Revenue'].sum().unstack(fill_value=0)
fig, ax = plt.subplots(figsize=(12, 6))
monthly_cat.plot.area(ax=ax, alpha=0.7, colormap='Set2', linewidth=1.5)
ax.set_title('Monthly Revenue by Top 5 Categories (Area)', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Month', color=LABEL_COLOR)
ax.set_ylabel('Total Revenue ($)', color=LABEL_COLOR)
ax.set_xticks(range(1, 13))
ax.legend(title='Category', bbox_to_anchor=(1.05, 1), loc='upper left')
ax.grid(alpha=0.3);
plt.tight_layout()
plt.show()
# =====
# PLOT 18: Violin Plot
# =====
print('\n--- Plot 18: Violin Plot (Avg Price by Coupon × Gender) ---')
fig, ax = plt.subplots(figsize=(10, 6))
sns.violinplot(data=df, x='Coupon_Status', y='Avg_Price', hue='Gender',
               split=True, palette=[C_F, C_M], ax=ax, inner='quartile', cut=0)
ax.set_title('Avg Price by Coupon Status & Gender (Violin)', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Coupon Status', color=LABEL_COLOR)
ax.set_ylabel('Avg Price ($)', color=LABEL_COLOR)
ax.legend(title='Gender');
ax.grid(axis='y', alpha=0.3);
plt.tight_layout()
plt.show()

```

```

# =====
# PLOT 19: Joint Plot (KDE + scatter)
# =====
print('\n--- Plot 19: Joint Plot (Avg Price vs Delivery Charges) ---')
samp2 = df[['Avg_Price', 'Delivery_Charges']].sample(3000, random_state=42)
g = sns.jointplot(data=samp2, x='Avg_Price', y='Delivery_Charges',
                  kind='scatter', marginal_kws={'fill': True},
                  joint_kws={'alpha': 0.3, 'color': C_M, 's': 10}, height=8)
g.plot_joint(sns.kdeplot, color=C_F, levels=5, linewidths=1.5)
g.figure.suptitle('Joint Plot: Avg Price vs Delivery Charges',
                  y=1.02, color=TITLE_COLOR, fontweight='bold', fontsize=16)
plt.show()
# =====
# PLOT 20: Rug Plot
# =====
print('\n--- Plot 20: Rug Plot (Delivery Charges by Gender) ---')
fig, ax = plt.subplots(figsize=(10, 5))
for gender, color in zip(['F', 'M'], [C_F, C_M]):
    subset = df[df['Gender'] == gender]['Delivery_Charges'].sample(2000, random_state=42)
    sns.rugplot(data=subset, ax=ax, color=color, alpha=0.3, height=0.5, label=gender)
sns.kdeplot(data=df, x='Delivery_Charges', hue='Gender',
            palette=[C_F, C_M], ax=ax, linewidth=2, fill=False)
ax.set_title('Delivery Charges with Rug Plot', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Delivery Charges ($)', color=LABEL_COLOR)
ax.set_ylabel('Density', color=LABEL_COLOR)
ax.legend(title='Gender');
ax.grid(alpha=0.3);
plt.tight_layout()
plt.show()
# =====
# PLOT 21: 3D Plot
# =====
print('\n--- Plot 21: 3D Scatter Plot ---')
fig = plt.figure(figsize=(12, 8))
ax3 = fig.add_subplot(111, projection='3d')
s3d = df.sample(2000, random_state=42)
sc = ax3.scatter(s3d['Avg_Price'], s3d['Online_Spend'], s3d['Quantity'],
                c=s3d['Delivery_Charges'], cmap='viridis', alpha=0.5, s=15)
ax3.set_xlabel('Avg Price ($)', labelpad=10)

```

```

ax3.set_ylabel('Online Spend ($)')
ax3.set_zlabel('Quantity', labelpad=10)
ax3.set_title('3D: Price × Online Spend × Quantity (color=Delivery)',
              color=TITLE_COLOR, fontweight='bold')
fig.colorbar(sc, shrink=0.5, label='Delivery Charges ($)')
plt.tight_layout()
plt.show()
# =====
# PLOT 22: Contour Plot
# =====
print('\n--- Plot 22: Contour Plot (Avg Price vs Online Spend) ---')
fig, ax = plt.subplots(figsize=(10, 7))
sc2 = df[['Avg_Price', 'Online_Spend']].sample(3000, random_state=42)
sns.kdeplot(data=sc2, x='Avg_Price', y='Online_Spend',
            fill=True, cmap='Blues', levels=15, ax=ax, alpha=0.8)
sns.kdeplot(data=sc2, x='Avg_Price', y='Online_Spend',
            levels=10, ax=ax, color='navy', linewidths=1)
ax.set_title('Contour: Avg Price vs Online Spend', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Avg Price ($)')
ax.set_ylabel('Online Spend ($)')
ax.grid(alpha=0.3);
plt.tight_layout()
plt.show()
# =====
# PLOT 23: Cluster Map
# =====
print('\n--- Plot 23: Cluster Map ---')
cm = sns.clustermap(df[num_cols].corr().round(2), annot=True, fmt='.2f',
                  cmap='RdBu_r', center=0, linewidths=1,
                  figsize=(10, 10), annot_kws={'fontsize': 9})
cm.figure.suptitle('Cluster Map of Feature Correlations',
                  y=1.02, color=TITLE_COLOR, fontweight='bold', fontsize=16)
plt.show()
# =====
# PLOT 24: Hexbin
# =====
print('\n--- Plot 24: Hexbin (Offline vs Online Spend) ---')
fig, ax = plt.subplots(figsize=(10, 7))
hb = ax.hexbin(df['Offline_Spend'], df['Online_Spend'], gridsize=30,

```

```

        cmap='YlOrRd', mincnt=1)
ax.set_title('Hexbin: Offline vs Online Spend', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Offline Spend ($)', color=LABEL_COLOR)
ax.set_ylabel('Online Spend ($)', color=LABEL_COLOR)
fig.colorbar(hb, ax=ax, label='Count')
ax.grid(alpha=0.3);
plt.tight_layout()
plt.show()

# =====
# PLOT 25: Strip Plot
# =====

print('\n--- Plot 25: Strip Plot (Avg Price by Top 6 Categories) ---')
fig, ax = plt.subplots(figsize=(12, 7))
top6 = df['Product_Category'].value_counts().head(6).index.tolist()
sns.stripplot(data=df[df['Product_Category'].isin(top6)],
              x='Product_Category', y='Avg_Price', hue='Gender',
              palette=[C_F, C_M], alpha=0.3, size=3, dodge=True, ax=ax, order=top6)
ax.set_title('Strip: Avg Price by Top 6 Categories & Gender', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Category', color=LABEL_COLOR)
ax.set_ylabel('Avg Price ($)', color=LABEL_COLOR)
ax.legend(title='Gender');
ax.grid(axis='y', alpha=0.3)
plt.xticks(rotation=20);
plt.tight_layout()
plt.show()

# =====
# PLOT 26: Swarm Plot
# =====

print('\n--- Plot 26: Swarm Plot (Delivery Charges by Location) ---')
fig, ax = plt.subplots(figsize=(10, 6))
sw = df.sample(800, random_state=42) # small sample — swarm is slow
sns.swarmplot(data=sw, x='Location', y='Delivery_Charges', hue='Gender',
              palette=[C_F, C_M], size=3, ax=ax,
              order=df['Location'].value_counts().index)
ax.set_title('Swarm: Delivery Charges by Location & Gender', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Location', color=LABEL_COLOR)
ax.set_ylabel('Delivery Charges ($)', color=LABEL_COLOR)
ax.legend(title='Gender');
ax.grid(axis='y', alpha=0.3);

```

```

plt.tight_layout()
plt.show()
# =====
# SUBPLOTS (4+) — STORYTELLING
# =====
print('\n' + '=' * 60)
print('SUBPLOTS — STORYTELLING')
print('=' * 60)

# --- Subplot A: 4 Pie Charts — Category breakdown by Location ---
print('\n--- Subplot A: 4 Pie Charts (Top Categories by Location) ---')
fig, axes = plt.subplots(2, 2, figsize=(14, 12))
fig.suptitle('Top 5 Product Categories by Location', color=TITLE_COLOR,
             fontweight='bold', fontsize=18, y=1.01)
top5cat = df['Product_Category'].value_counts().head(5).index
locs = ['Chicago', 'California', 'New York', 'New Jersey']
for ax, loc in zip(axes.flatten(), locs):
    sub = df[df['Location'] == loc]['Product_Category'].value_counts()
    sub_top = sub[sub.index.isin(top5cat)]
    sub_top['Other'] = sub[~sub.index.isin(top5cat)].sum()
    ax.pie(sub_top.values, labels=sub_top.index, autopct='%1.1f%%',
           colors=sns.color_palette('Set2', len(sub_top)),
           textprops={'fontsize': 9}, wedgeprops={'edgecolor': 'white'})
    ax.set_title(loc, fontsize=14, fontweight='bold', color=LABEL_COLOR)
plt.tight_layout()
plt.show()

# --- Subplot B: 4 Bar Charts — Spend patterns ---
print('\n--- Subplot B: 4 Bar Charts (Spend Patterns) ---')
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Spending Patterns Analysis', color=TITLE_COLOR,
             fontweight='bold', fontsize=18, y=1.01)
plt.show()
# B1: Avg Online Spend by Gender
ax = axes[0, 0]
g_spend = df.groupby('Gender')['Online_Spend'].mean().sort_values(ascending=False)
bars = ax.bar(g_spend.index, g_spend.values, color=[C_F, C_M], edgecolor='white')
for b in bars: ax.text(b.get_x() + b.get_width() / 2, b.get_height() + 20,
                      f'${b.get_height():.2f}', ha='center', fontsize=10)

```

```

ax.set_title('Avg Online Spend by Gender', color=LABEL_COLOR, fontweight='bold')
ax.set_ylabel('Avg Spend ($)');
ax.grid(axis='y', alpha=0.3)
plt.show()

# B2: Avg Online Spend by Location
ax = axes[0, 1]
l_spend = df.groupby('Location')['Online_Spend'].mean().sort_values(ascending=False)
bars = ax.bar(l_spend.index, l_spend.values, color=sns.color_palette('Set2', 5), edgecolor='white')
for b in bars: ax.text(b.get_x() + b.get_width() / 2, b.get_height() + 20,
                      f'${b.get_height():.2f}', ha='center', fontsize=9)
ax.set_title('Avg Online Spend by Location', color=LABEL_COLOR, fontweight='bold')
ax.set_ylabel('Avg Spend ($)');
ax.grid(axis='y', alpha=0.3)
plt.setp(ax.get_xticklabels(), rotation=20)
plt.show()

# B3: Avg Online Spend by Coupon Status
ax = axes[1, 0]
c_spend = df.groupby('Coupon_Status')['Online_Spend'].mean().sort_values(ascending=False)
bars = ax.bar(c_spend.index, c_spend.values, color=sns.color_palette('Set2', 3), edgecolor='white')
for b in bars: ax.text(b.get_x() + b.get_width() / 2, b.get_height() + 20,
                      f'${b.get_height():.2f}', ha='center', fontsize=10)
ax.set_title('Avg Online Spend by Coupon Status', color=LABEL_COLOR, fontweight='bold')
ax.set_ylabel('Avg Spend ($)');
ax.grid(axis='y', alpha=0.3)
plt.show()

# B4: Avg Online Spend by Top 5 Category
ax = axes[1, 1]
cat_spend =
df.groupby('Product_Category')['Online_Spend'].mean().sort_values(ascending=False).head(5)
bars = ax.barh(cat_spend.index, cat_spend.values, color=sns.color_palette('Set2', 5), edgecolor='white')
for b, v in zip(bars, cat_spend.values):
    ax.text(v + 10, b.get_y() + b.get_height() / 2, f'${v:.2f}', va='center', fontsize=10)
ax.set_title('Avg Online Spend by Top 5 Category', color=LABEL_COLOR, fontweight='bold')
ax.set_xlabel('Avg Spend ($)');
ax.grid(axis='x', alpha=0.3)

plt.tight_layout()
plt.show()

# --- Subplot C: Histograms for multiple numerical features ---

```

```

print('\n--- Subplot C: 4 Histograms (Numerical Feature Distributions) ---')
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Distributions of Key Numerical Features', color=TITLE_COLOR,
             fontweight='bold', fontsize=18, y=1.01)
feats = ['Avg_Price', 'Delivery_Charges', 'Online_Spend', 'Offline_Spend']
colors = ['#2ecc71', '#9b59b6', '#3498db', '#e74c3c']
for ax, feat, col in zip(axes.flatten(), feats, colors):
    sns.histplot(df[feat], kde=True, bins=40, color=col, edgecolor='white', alpha=0.7, ax=ax)
    mu = df[feat].mean()
    ax.axvline(mu, color='black', linestyle='--', linewidth=1.5, label=f'Mean: {mu:.2f}')
    ax.set_title(feat, color=LABEL_COLOR, fontweight='bold')
    ax.legend(fontsize=9);
    ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# --- Subplot D: KDE comparison across locations ---
print('\n--- Subplot D: KDE of Online Spend by Location (4 panels) ---')
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Online Spend Distribution by Location', color=TITLE_COLOR,
             fontweight='bold', fontsize=18, y=1.01)
for ax, loc in zip(axes.flatten(), ['Chicago', 'California', 'New York', 'New Jersey']):
    sub = df[df['Location'] == loc]
    sns.kdeplot(data=sub, x='Online_Spend', hue='Gender', fill=True,
               palette=[C_F, C_M], alpha=0.4, linewidth=2, ax=ax)
    ax.set_title(loc, color=LABEL_COLOR, fontweight='bold')
    ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# =====
# =====
# SECTION A: STATISTICS — Descriptive & Inferential
# =====
# =====

print('\n' + '=' * 60)
print('SECTION A: STATISTICS')
print('=' * 60)

```

```

from scipy.stats import skew, kurtosis, ttest_ind, f_oneway

```

```
from scipy.stats import gaussian_kde
```

```
# --- A1: Descriptive Statistics Table (PrettyTable) ---
```

```
print('\n--- A1: Descriptive Statistics Table ---')
```

```
t_stat = PrettyTable()
```

```
t_stat.title = 'Descriptive Statistics of Numerical Features'
```

```
t_stat.field_names = ['Feature', 'Mean', 'Std', 'Min', 'Max', 'Median', 'Skewness', 'Kurtosis']
```

```
for col in num_cols:
```

```
    t_stat.add_row([
```

```
        col,
```

```
        f'{df[col].mean():.2f}',
```

```
        f'{df[col].std():.2f}',
```

```
        f'{df[col].min():.2f}',
```

```
        f'{df[col].max():.2f}',
```

```
        f'{df[col].median():.2f}',
```

```
        f'{skew(df[col].dropna()):.2f}',
```

```
        f'{kurtosis(df[col].dropna()):.2f}'
```

```
    ])
```

```
print(t_stat)
```

```
# --- A2: T-Test — Online Spend by Gender ---
```

```
print('\n--- A2: T-Test (Online Spend: Female vs Male) ---')
```

```
grp_f = df[df['Gender'] == 'F']['Online_Spend']
```

```
grp_m = df[df['Gender'] == 'M']['Online_Spend']
```

```
t_stat_val, p_val = ttest_ind(grp_f, grp_m, equal_var=False)
```

```
t_ttest = PrettyTable()
```

```
t_ttest.title = 'Welch T-Test: Online Spend (F vs M)'
```

```
t_ttest.field_names = ['Group', 'Mean', 'Std', 'N']
```

```
t_ttest.add_row(['Female', f'{grp_f.mean():.2f}', f'{grp_f.std():.2f}', f'{len(grp_f):,}'])
```

```
t_ttest.add_row(['Male', f'{grp_m.mean():.2f}', f'{grp_m.std():.2f}', f'{len(grp_m):,}'])
```

```
print(t_ttest)
```

```
print(f' t-statistic: {t_stat_val:.4f} | p-value: {p_val:.4f}')
```

```
print(' → Conclusion:', 'Significant difference (p<0.05)' if p_val < 0.05 else 'No significant difference')
```

```
# --- A3: One-Way ANOVA — Online Spend by Location ---
```

```
print('\n--- A3: One-Way ANOVA (Online Spend by Location) ---')
```

```
groups = [grp['Online_Spend'].values for _, grp in df.groupby('Location')]
```

```
f_stat_val, p_anova = f_oneway(*groups)
```

```
t_anova = PrettyTable()
```

```

t_anova.title = 'One-Way ANOVA: Online Spend by Location'
t_anova.field_names = ['Location', 'Mean', 'Std', 'N']
for loc, grp in df.groupby('Location'):
    t_anova.add_row([loc, f'{grp["Online_Spend"].mean():.2f}',
                    f'{grp["Online_Spend"].std():.2f}', f'{len(grp):,}'])
print(t_anova)
print(f' F-statistic: {f_stat_val:.4f} | p-value: {p_anova:.4f}')
print(' → Conclusion:', 'Significant difference across locations (p<0.05)' if p_anova < 0.05 else 'No
significant difference')

```

```

# --- A4: Multivariate KDE Plot ---

```

```

print('\n--- A4: Multivariate KDE Estimate (Online vs Offline Spend) ---')
fig, ax = plt.subplots(figsize=(10, 7))
samp_kde = df[['Online_Spend', 'Offline_Spend']].sample(3000, random_state=42)
sns.kdeplot(data=samp_kde, x='Offline_Spend', y='Online_Spend',
            fill=True, cmap='mako', levels=15, alpha=0.8, ax=ax)
sns.kdeplot(data=samp_kde, x='Offline_Spend', y='Online_Spend',
            levels=10, ax=ax, color='white', linewidths=0.8, alpha=0.6)
ax.set_title('Multivariate KDE: Online vs Offline Spend', color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('Offline Spend ($)')
ax.set_ylabel('Online Spend ($)')
ax.grid(alpha=0.2)
plt.tight_layout()
plt.show()
print('Observation: The multivariate KDE reveals the joint density of Online and Offline Spend.')
print('The highest density region indicates where most customers spend similar amounts in both
channels.')

```

```

# =====
# =====
# SECTION B: OUTLIER DETECTION & REMOVAL
# =====
# =====

print('\n' + '=' * 60)
print('SECTION B: OUTLIER DETECTION & REMOVAL')
print('=' * 60)

```

```

from scipy.stats import zscore

```

```
df_clean = df.copy()
n_before = len(df_clean)

# --- B1: IQR Method ---
print('\n--- B1: IQR-Based Outlier Detection ---')
t_iqr = PrettyTable()
t_iqr.title = 'Outlier Detection via IQR Method'
t_iqr.field_names = ['Feature', 'Q1', 'Q3', 'IQR', 'Lower Fence', 'Upper Fence', 'Outliers']
outlier_mask = pd.Series([False] * len(df_clean), index=df_clean.index)
for col in ['Online_Spend', 'Offline_Spend', 'Avg_Price', 'Delivery_Charges', 'Revenue']:
    Q1 = df_clean[col].quantile(0.25)
    Q3 = df_clean[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    mask = (df_clean[col] < lower) | (df_clean[col] > upper)
    outlier_mask = outlier_mask | mask
    t_iqr.add_row([col, f'{Q1:.2f}', f'{Q3:.2f}', f'{IQR:.2f}',
                  f'{lower:.2f}', f'{upper:.2f}', f'{mask.sum():,}'])
print(t_iqr)

# --- B2: Z-Score Method (for comparison) ---
print('\n--- B2: Z-Score Outlier Detection (|z| > 3) ---')
t_zscore = PrettyTable()
t_zscore.title = 'Outlier Detection via Z-Score (threshold=3)'
t_zscore.field_names = ['Feature', 'Mean', 'Std', 'Z>3 Outliers', '% of Data']
for col in ['Online_Spend', 'Offline_Spend', 'Avg_Price', 'Delivery_Charges', 'Revenue']:
    z = np.abs(zscore(df_clean[col].dropna()))
    n_out = (z > 3).sum()
    t_zscore.add_row([col, f'{df_clean[col].mean():.2f}', f'{df_clean[col].std():.2f}',
                     f'{n_out:,}', f'{100*n_out/len(df_clean):.2f}%'])
print(t_zscore)

# Apply IQR removal
df_no_outlier = df_clean[~outlier_mask].copy()
n_after = len(df_no_outlier)
print(f'\n Rows before IQR removal : {n_before:,}')
print(f' Rows after IQR removal : {n_after:,}')
print(f' Removed : {n_before - n_after:,} ({100*(n_before-n_after)/n_before:.2f}%)')
```

```

# --- B3: Box Plot Before vs After ---
print('\n--- B3: Box Plot — Before vs After Outlier Removal ---')
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
fig.suptitle('Outlier Removal Comparison (IQR Method)', color=TITLE_COLOR,
             fontweight='bold', fontsize=16)
for ax, data, label in zip(axes, [df, df_no_outlier], ['Before', 'After']):
    melt_b = data.melt(value_vars=['Online_Spend', 'Offline_Spend', 'Avg_Price', 'Revenue'],
                      var_name='Feature', value_name='Value')
    sns.boxplot(data=melt_b, x='Feature', y='Value', palette='Set2', ax=ax)
    ax.set_title(f'{label} Outlier Removal', color=LABEL_COLOR, fontweight='bold')
    ax.set_xlabel('Feature', color=LABEL_COLOR)
    ax.set_ylabel('Value ($)', color=LABEL_COLOR)
    ax.grid(axis='y', alpha=0.3)
    plt.setp(ax.get_xticklabels(), rotation=15)
plt.tight_layout()
plt.show()
print('Observation: IQR method effectively removes extreme values.')
print('Revenue and Online_Spend show the most outliers due to high-value transactions.')

```

```

# =====
# =====
# SECTION C: NORMALITY TEST
# =====
# =====
print('\n' + '=' * 60)
print('SECTION C: NORMALITY TESTS')
print('=' * 60)

```

```

from scipy.stats import shapiro, normaltest, anderson

```

```

# --- C1: Shapiro-Wilk, D'Agostino Tests ---
print('\n--- C1: Normality Test Results Table ---')
t_norm = PrettyTable()
t_norm.title = 'Normality Tests (Shapiro-Wilk & D\'Agostino K²)'
t_norm.field_names = ['Feature', 'Shapiro W', 'Shapiro p', 'SW Normal?',
                     'D\'Agostino p', 'DA Normal?']
test_cols = ['Online_Spend', 'Offline_Spend', 'Avg_Price', 'Delivery_Charges', 'GST']
for col in test_cols:

```

```

samp_n = df[col].dropna().sample(min(5000, len(df)), random_state=42)
sw_stat, sw_p = shapiro(samp_n.sample(min(5000, len(samp_n)), random_state=42))
da_stat, da_p = normaltest(samp_n)
t_norm.add_row([col,
                 f'{sw_stat:.4f}', f'{sw_p:.4f}', 'Yes' if sw_p > 0.05 else 'No',
                 f'{da_p:.4f}', 'Yes' if da_p > 0.05 else 'No'])
print(t_norm)
print('\nObservation: Most financial variables (Online_Spend, Offline_Spend) are NOT normally
distributed')
print('(p < 0.05), indicating right-skewed distributions typical of transaction data.')

# --- C2: QQ plots for 4 features ---
print('\n--- C2: QQ Plots for Multiple Features ---')
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('QQ-Plots: Normality Assessment', color=TITLE_COLOR,
             fontweight='bold', fontsize=18, y=1.01)
qq_feats = ['Online_Spend', 'Offline_Spend', 'Avg_Price', 'Delivery_Charges']
colors_qq = [C_M, C_F, '#2ecc71', '#9b59b6']
for ax, feat, col in zip(axes.flatten(), qq_feats, colors_qq):
    sm.qqplot(df[feat].dropna(), line='45', ax=ax,
              markerfacecolor=col, markeredgecolor='white', alpha=0.4, markersize=3)
    ax.set_title(feat, color=LABEL_COLOR, fontweight='bold')
    ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# =====
# =====
# SECTION D: DATA TRANSFORMATION
# =====
# =====
print('\n' + '=' * 60)
print('SECTION D: DATA TRANSFORMATION')
print('=' * 60)

from sklearn.preprocessing import StandardScaler, MinMaxScaler

# --- D1: Log Transformation ---
print('\n--- D1: Log Transformation (Online_Spend) ---')

```

```

df['Online_Spend_log'] = np.log1p(df['Online_Spend'])
df['Offline_Spend_log'] = np.log1p(df['Offline_Spend'])
df['Revenue_log'] = np.log1p(df['Revenue'])

fig, axes = plt.subplots(2, 3, figsize=(16, 9))
fig.suptitle('Log Transformation: Before vs After', color=TITLE_COLOR,
             fontweight='bold', fontsize=16)
pairs = [('Online_Spend', 'Online_Spend_log'),
         ('Offline_Spend', 'Offline_Spend_log'),
         ('Revenue', 'Revenue_log')]
for i, (orig, log_c) in enumerate(pairs):
    for j, (col, label) in enumerate([(orig, 'Original'), (log_c, 'Log Transformed')]):
        ax = axes[j][i]
        sk = skew(df[col].dropna())
        sns.histplot(df[col], kde=True, bins=40, ax=ax,
                     color=['#3498db', '#e74c3c'][j], alpha=0.7, edgecolor='white')
        ax.set_title(f'{label}: {orig}\n(Skew={sk:.2f})', color=LABEL_COLOR, fontweight='bold')
        ax.set_xlabel(col, color=LABEL_COLOR)
        ax.set_ylabel('Frequency', color=LABEL_COLOR)
        ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```

```

# --- D2: Normalization Table ---
print('\n--- D2: Normalization Comparison Table ---')
t_trans = PrettyTable()
t_trans.title = 'Before vs After Transformation (Online_Spend)'
t_trans.field_names = ['Metric', 'Original', 'Log1p', 'Z-Score', 'MinMax']
sc = StandardScaler().fit_transform(df[['Online_Spend']])
mm = MinMaxScaler().fit_transform(df[['Online_Spend']])
data_z = pd.Series(sc.flatten())
data_mm = pd.Series(mm.flatten())
data_log = df['Online_Spend_log']
orig = df['Online_Spend']
for metric, fn in [('Mean', lambda x: x.mean()), ('Std', lambda x: x.std()),
                  ('Skewness', lambda x: skew(x.dropna())), ('Min', lambda x: x.min()),
                  ('Max', lambda x: x.max())]:
    t_trans.add_row([metric, f'{fn(orig):.2f}', f'{fn(data_log):.2f}',
                    f'{fn(data_z):.2f}', f'{fn(data_mm):.2f}'])

```

```
print(t_trans)
print('\nObservation: Log transformation reduces skewness from highly right-skewed to near-normal,')
print('making the data more suitable for statistical modeling and linear methods.')
```

```
# =====
# =====
# SECTION E: PRINCIPAL COMPONENT ANALYSIS (PCA)
# =====
# =====
print('\n' + '=' * 60)
print('SECTION E: PRINCIPAL COMPONENT ANALYSIS (PCA)')
print('=' * 60)
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
# Standardize
pca_cols = ['Tenure_Months', 'Quantity', 'Avg_Price', 'Delivery_Charges',
            'GST', 'Offline_Spend', 'Online_Spend', 'Discount_pct', 'Total_Spend', 'Revenue']
df_pca_data = df[pca_cols].dropna()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_pca_data)
```

```
# Full PCA
pca_full = PCA()
pca_full.fit(X_scaled)
```

```
# --- E1: Explained Variance Table ---
print('\n--- E1: PCA Explained Variance Table ---')
t_pca = PrettyTable()
t_pca.title = 'PCA Explained Variance'
t_pca.field_names = ['PC', 'Eigenvalue', 'Explained Var %', 'Cumulative %']
cum = 0
for i, (ev, evr) in enumerate(zip(pca_full.explained_variance_,
                                pca_full.explained_variance_ratio_)):
    cum += evr * 100
    t_pca.add_row([f'PC{i+1}', f'{ev:.4f}', f'{evr*100:.2f}%', f'{cum:.2f}%'])
print(t_pca)
```

```

# --- E2: Condition Number & Singular Values ---
print('\n--- E2: Condition Number & Singular Values ---')
cond_num = np.linalg.cond(X_scaled)
svd_vals = pca_full.singular_values_
t_svd = PrettyTable()
t_svd.title = 'Singular Values of Standardized Feature Matrix'
t_svd.field_names = ['PC'] + [f'PC{i+1}' for i in range(len(svd_vals))]
t_svd.add_row(['Singular Value'] + [f'{v:.2f}' for v in svd_vals])
print(t_svd)
print(f'\n Condition Number: {cond_num:.2f}')
print(' → A large condition number suggests multicollinearity among features.')

# --- E3: Scree Plot ---
print('\n--- E3: Scree Plot + Cumulative Variance ---')
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
fig.suptitle('PCA Analysis: Online Shopping Dataset', color=TITLE_COLOR,
            fontweight='bold', fontsize=16)
ax1 = axes[0]
evr = pca_full.explained_variance_ratio_ * 100
bars = ax1.bar(range(1, len(evr)+1), evr, color=sns.color_palette('Set2', len(evr)), edgecolor='white')
for b, v in zip(bars, evr):
    ax1.text(b.get_x()+b.get_width()/2, b.get_height()+0.3, f'{v:.1f}%',
            ha='center', fontsize=9)
ax1.set_title('Scree Plot (Explained Variance per PC)', color=LABEL_COLOR, fontweight='bold')
ax1.set_xlabel('Principal Component', color=LABEL_COLOR)
ax1.set_ylabel('Explained Variance (%)', color=LABEL_COLOR)
ax1.grid(axis='y', alpha=0.3)
ax2 = axes[1]
cum_var = np.cumsum(evr)
ax2.plot(range(1, len(cum_var)+1), cum_var, 'o-', color=C_F, linewidth=2.5, markersize=8)
ax2.axhline(95, color=C_M, linestyle='--', linewidth=1.5, label='95% threshold')
ax2.axhline(80, color='green', linestyle='--', linewidth=1.5, label='80% threshold')
for i, v in enumerate(cum_var):
    ax2.annotate(f'{v:.1f}%', (i+1, v), textcoords='offset points',
            xytext=(0, 8), fontsize=8, ha='center', color=TITLE_COLOR)
ax2.set_title('Cumulative Explained Variance', color=LABEL_COLOR, fontweight='bold')
ax2.set_xlabel('Number of Components', color=LABEL_COLOR)
ax2.set_ylabel('Cumulative Variance (%)', color=LABEL_COLOR)
ax2.legend(); ax2.grid(alpha=0.3)

```

```

plt.tight_layout()
plt.show()

# --- E4: PCA Biplot (PC1 vs PC2 with loadings) ---
print('\n--- E4: PCA Biplot (PC1 vs PC2) ---')
pca2 = PCA(n_components=2)
X_pca2 = pca2.fit_transform(X_scaled)
samp_idx = np.random.choice(len(X_pca2), 2000, replace=False)
fig, ax = plt.subplots(figsize=(12, 8))
scatter = ax.scatter(X_pca2[samp_idx, 0], X_pca2[samp_idx, 1],
                    c=df_pca_data.iloc[samp_idx]['Online_Spend'],
                    cmap='viridis', alpha=0.4, s=10)
plt.colorbar(scatter, ax=ax, label='Online Spend ($)')
scale = 3
for i, feat in enumerate(pca_cols):
    ax.arrow(0, 0,
            pca2.components_[0, i]*scale,
            pca2.components_[1, i]*scale,
            head_width=0.08, head_length=0.05, fc=C_F, ec=C_F)
    ax.text(pca2.components_[0, i]*scale*1.15,
           pca2.components_[1, i]*scale*1.15,
           feat, fontsize=9, color=TITLE_COLOR, fontweight='bold')
ax.set_title(f'PCA Biplot (PC1={pca2.explained_variance_ratio_[0]*100:.1f}% | '
            f'PC2={pca2.explained_variance_ratio_[1]*100:.1f}%)',
            color=TITLE_COLOR, fontweight='bold')
ax.set_xlabel('PC1', color=LABEL_COLOR); ax.set_ylabel('PC2', color=LABEL_COLOR)
ax.axhline(0, color='gray', linewidth=0.5); ax.axvline(0, color='gray', linewidth=0.5)
ax.grid(alpha=0.2)
plt.tight_layout()
plt.show()
print('Observation: PC1 is dominated by spend-related features (Total_Spend, Online/Offline_Spend,
Revenue),')
print('while PC2 separates Delivery_Charges and Discount_pct. ~2-3 PCs explain most variance.')

# =====
# =====
# SECTION F: ADDITIONAL STATISTICS & PRETTY TABLES
# =====
# =====

```

```
print('\n' + '=' * 60)
print('SECTION F: ADDITIONAL TABLES')
print('=' * 60)

# --- F1: Dataset Summary Table ---
print('\n--- F1: Dataset Summary ---')
t_summary = PrettyTable()
t_summary.title = 'Dataset Summary'
t_summary.field_names = ['Attribute', 'Value']
t_summary.add_row(['Total Rows', f'{len(df):,}'])
t_summary.add_row(['Total Columns', f'{df.shape[1]}'])
t_summary.add_row(['Numerical Features', f'{len(num_cols)}'])
t_summary.add_row(['Categorical Features', f'{len(cat_cols)}'])
t_summary.add_row(['Missing Values', f'{df.isnull().sum().sum()}'])
t_summary.add_row(['Duplicate Rows', f'{df.duplicated().sum():,}'])
t_summary.add_row(['Date Range', f'{df["Transaction_Date"].min().date()} → {df["Transaction_Date"].max().date()}'])
t_summary.add_row(['Unique Customers', f'{df["CustomerID"].nunique():,}'])
t_summary.add_row(['Unique Products', f'{df["Product_Category"].nunique():,}'])
print(t_summary)

# --- F2: Category Distribution Table ---
print('\n--- F2: Categorical Feature Distributions ---')
for col in cat_cols:
    t_cat = PrettyTable()
    t_cat.title = f'Distribution: {col}'
    t_cat.field_names = [col, 'Count', 'Percentage']
    vc = df[col].value_counts()
    for val, cnt in vc.items():
        t_cat.add_row([val, f'{cnt:,}', f'{100*cnt/len(df):.2f}%'])
    print(t_cat)

# --- F3: Pearson Correlation Table (expanded) ---
print('\n--- F3: Full Pearson Correlation Table ---')
corr_full = df[num_cols].corr().round(2)
t_corr = PrettyTable()
t_corr.title = 'Pearson Correlation Matrix'
t_corr.field_names = [''] + num_cols
for idx in corr_full.index:
```

```

    t_corr.add_row([idx] + [f'{v:.2f}' for v in corr_full.loc[idx]])
print(t_corr)

print('\n' + '=' * 60)
print('ALL PHASE I ANALYSES COMPLETE')
print('=' * 60)

# =====
# =====
# PROPOSAL INSIGHT PLOTS
# =====
# =====

print('\n' + '=' * 60)
print('PROPOSAL INSIGHT PLOTS')
print('=' * 60)

# =====
# PROPOSAL PLOT 1: GST vs Avg_Price — Scatter + Regression
# =====

print('\n--- Proposal Plot 1: GST vs Avg_Price (Negative Correlation r=-0.58) ---')
fig, axes = plt.subplots(1, 2, figsize=(16, 7))
fig.suptitle('GST vs Avg Price: Negative Correlation Analysis',
             color=TITLE_COLOR, fontweight='bold', fontsize=16)

# Left: Scatter + Regression by Gender
ax = axes[0]
samp_gst = df.sample(3000, random_state=42)
sns.regplot(data=samp_gst, x='Avg_Price', y='GST',
            scatter_kws={'alpha': 0.25, 'color': C_M, 's': 12},
            line_kws={'color': C_F, 'linewidth': 2.5}, ax=ax)
r_gst = df[['Avg_Price', 'GST']].corr().iloc[0, 1]
ax.text(0.05, 0.95, f'r = {r_gst:.2f}', transform=ax.transAxes,
        fontsize=14, va='top', fontweight='bold',
        bbox=dict(boxstyle='round', facecolor='#fadb8', alpha=0.8))
ax.set_title('GST vs Avg Price (Regression)', color=LABEL_COLOR, fontweight='bold')
ax.set_xlabel('Avg Price ($)', color=LABEL_COLOR)
ax.set_ylabel('GST (%)', color=LABEL_COLOR)
ax.grid(alpha=0.3)

```

```

# Right: GST by Price Bin (bar)
ax = axes[1]
df['Price_Bin'] = pd.cut(df['Avg_Price'],
                        bins=[0, 25, 50, 100, 200, 400],
                        labels=['$0-25', '$25-50', '$50-100', '$100-200', '$200+'])
gst_by_price = df.groupby('Price_Bin', observed=True)['GST'].mean()
bars = ax.bar(gst_by_price.index, gst_by_price.values,
             color=sns.color_palette('RdYlGn_r', len(gst_by_price)), edgecolor='white')
for b, v in zip(bars, gst_by_price.values):
    ax.text(b.get_x() + b.get_width()/2, b.get_height() + 0.01,
           f'{v:.2f}%', ha='center', fontsize=11, fontweight='bold', color=TITLE_COLOR)
ax.set_title('Avg GST Rate by Price Bracket', color=LABEL_COLOR, fontweight='bold')
ax.set_xlabel('Price Range', color=LABEL_COLOR)
ax.set_ylabel('Mean GST (%)', color=LABEL_COLOR)
ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
print('Observation: GST shows a strong negative correlation with Avg_Price (r=-0.58).')
print('Lower-priced items carry higher GST rates, while premium products ($200+)')
print('have significantly lower effective GST — suggesting tiered tax structures.')

# =====
# PROPOSAL PLOT 2: Online vs Offline Spend — Distribution Comparison
# =====
print('\n--- Proposal Plot 2: Online vs Offline Spend Distribution Comparison ---')
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.suptitle('Online vs Offline Spend: Distribution Deep Dive',
            color=TITLE_COLOR, fontweight='bold', fontsize=16)

# Left: KDE Overlay
ax = axes[0]
sns.kdeplot(df['Online_Spend'], ax=ax, fill=True, alpha=0.45,
           color=C_M, linewidth=2.5, label='Online Spend')
sns.kdeplot(df['Offline_Spend'], ax=ax, fill=True, alpha=0.45,
           color=C_F, linewidth=2.5, label='Offline Spend')
ax.axvline(df['Online_Spend'].mean(), color=C_M, linestyle='--',
          linewidth=1.8, label=f'Online Mean: ${df["Online_Spend"].mean():.2f}')
ax.axvline(df['Offline_Spend'].mean(), color=C_F, linestyle='--',

```

```
        linewidth=1.8, label=f'Offline Mean: ${df["Offline_Spend"].mean():.2f}')
ax.set_title('KDE Overlay: Online vs Offline Spend', color=LABEL_COLOR, fontweight='bold')
ax.set_xlabel('Spend ($)', color=LABEL_COLOR)
ax.set_ylabel('Density', color=LABEL_COLOR)
ax.legend(fontsize=9)
ax.grid(alpha=0.3)
```

```
# Middle: Histogram comparison (side by side bins)
ax = axes[1]
ax.hist(df['Online_Spend'], bins=40, alpha=0.6, color=C_M,
        edgecolor='white', label='Online', density=True)
ax.hist(df['Offline_Spend'], bins=40, alpha=0.6, color=C_F,
        edgecolor='white', label='Offline', density=True)
ax.set_title('Histogram: Spend Distribution Shape', color=LABEL_COLOR, fontweight='bold')
ax.set_xlabel('Spend ($)', color=LABEL_COLOR)
ax.set_ylabel('Density', color=LABEL_COLOR)
ax.legend()
ax.grid(alpha=0.3)
```

```
# Right: Stats comparison table as text
ax = axes[2]
ax.axis('off')
stats_data = {
    'Metric': ['Mean', 'Median', 'Std Dev', 'Min', 'Max', 'Skewness'],
    'Online ($)': [
        f'{df["Online_Spend"].mean():.2f}',
        f'{df["Online_Spend"].median():.2f}',
        f'{df["Online_Spend"].std():.2f}',
        f'{df["Online_Spend"].min():.2f}',
        f'{df["Online_Spend"].max():.2f}',
        f'{stats.skew(df["Online_Spend"]):.2f}'
    ],
    'Offline ($)': [
        f'{df["Offline_Spend"].mean():.2f}',
        f'{df["Offline_Spend"].median():.2f}',
        f'{df["Offline_Spend"].std():.2f}',
        f'{df["Offline_Spend"].min():.2f}',
        f'{df["Offline_Spend"].max():.2f}',
        f'{stats.skew(df["Offline_Spend"]):.2f}'
    ]
}
```

```

    ]
}
tbl = ax.table(
    cellText=list(zip(stats_data['Metric'],
                      stats_data['Online ($)'],
                      stats_data['Offline ($)'])),
    colLabels=['Metric', 'Online ($)', 'Offline ($)'],
    cellLoc='center', loc='center',
    bbox=[0, 0.1, 1, 0.85]
)
tbl.auto_set_font_size(False)
tbl.set_fontsize(11)
for (r, c), cell in tbl.get_celld().items():
    if r == 0:
        cell.set_facecolor('#2c3e50')
        cell.set_text_props(color='white', fontweight='bold')
    elif c == 1:
        cell.set_facecolor('#d6eaf8')
    elif c == 2:
        cell.set_facecolor('#fadbd8')
        cell.set_edgecolor('white')
ax.set_title('Summary Statistics Comparison', color=LABEL_COLOR, fontweight='bold')

plt.tight_layout()
plt.show()
print('Observation: Online Spend shows a near-uniform distribution ($320-$4,557)')
print('while Offline Spend has a higher median and wider IQR, confirming customers')
print('spend more in physical stores but online spend is more evenly distributed.')

# =====
# PROPOSAL PLOT 3: Category Price Range Box Plot
# =====
print('\n--- Proposal Plot 3: Category Price Range Box Plot (Nest-USA Highlight) ---')
fig, axes = plt.subplots(1, 2, figsize=(18, 7))
fig.suptitle('Product Category Price Range Analysis',
             color=TITLE_COLOR, fontweight='bold', fontsize=16)

# Left: Box plot all categories sorted by median price
ax = axes[0]

```

```

cat_order = (df.groupby('Product_Category')['Avg_Price']
              .median()
              .sort_values(ascending=False)
              .index.tolist())
palette_cat = ['#e74c3c' if c == 'Nest-USA' else '#3498db' for c in cat_order]
sns.boxplot(data=df, y='Product_Category', x='Avg_Price',
             order=cat_order, palette=palette_cat, ax=ax,
             flierprops=dict(marker='o', markerfacecolor='gray',
                             markersize=3, alpha=0.3))
ax.set_title('Price Range by Category (sorted by median)\n● = Nest-USA',
             color=LABEL_COLOR, fontweight='bold')
ax.set_xlabel('Avg Price ($)', color=LABEL_COLOR)
ax.set_ylabel('Product Category', color=LABEL_COLOR)
ax.grid(axis='x', alpha=0.3)
ax.axvline(df['Avg_Price'].mean(), color='green', linestyle='--',
           linewidth=1.5, label=f'Overall Mean: ${df["Avg_Price"].mean():.2f}')
ax.legend(fontsize=9)

# Right: IQR + Range bar chart per category
ax = axes[1]
cat_stats = df.groupby('Product_Category')['Avg_Price'].agg(
    Q1=lambda x: x.quantile(0.25),
    Q3=lambda x: x.quantile(0.75),
    median='median',
    price_max='max'
).reset_index()
cat_stats['IQR'] = cat_stats['Q3'] - cat_stats['Q1']
cat_stats = cat_stats.sort_values('IQR', ascending=True)
colors_iqr = ['#e74c3c' if c == 'Nest-USA' else '#85B7EB'
              for c in cat_stats['Product_Category']]
bars = ax.barh(cat_stats['Product_Category'], cat_stats['IQR'],
               color=colors_iqr, edgecolor='white')
for b, v in zip(bars, cat_stats['IQR']):
    ax.text(v + 0.5, b.get_y() + b.get_height()/2,
            f'${v:.2f}', va='center', fontsize=9, color=TITLE_COLOR)
ax.set_title('Price IQR by Category (wider = more price variation)\n● = Nest-USA',
             color=LABEL_COLOR, fontweight='bold')
ax.set_xlabel('IQR of Avg Price ($)', color=LABEL_COLOR)
ax.set_ylabel('Product Category', color=LABEL_COLOR)

```

```

ax.grid(axis='x', alpha=0.3)

plt.tight_layout()
plt.show()
print('Observation: Nest-USA has the widest price range ($0-$350) and highest IQR,')
print('indicating a diverse product lineup from budget to premium.')
print('Apparel, Office, and Drinkware are concentrated under $50 (low IQR).')

# =====
# PROPOSAL PLOT 4: Monthly Revenue Heatmap by Category
# =====
print('\n--- Proposal Plot 4: Monthly Revenue Heatmap (Seasonality Analysis) ---')
fig, axes = plt.subplots(2, 1, figsize=(16, 14))
fig.suptitle('Seasonal Revenue Analysis by Product Category',
             color=TITLE_COLOR, fontweight='bold', fontsize=16)

# Top: Heatmap
ax = axes[0]
top8_cats = df['Product_Category'].value_counts().head(8).index.tolist()
monthly_heat = (df[df['Product_Category'].isin(top8_cats)]
                .groupby(['Product_Category', 'Month'])['Revenue']
                .sum()
                .unstack(fill_value=0))

# Normalize per row for pattern visibility
monthly_heat_norm = monthly_heat.div(monthly_heat.max(axis=1), axis=0).round(2)
sns.heatmap(monthly_heat_norm, annot=True, fmt='.2f', cmap='YlOrRd',
            ax=ax, linewidths=0.5, cbar_kws={'label': 'Normalized Revenue (0-1)'},
            annot_kws={'fontsize': 9})
ax.set_title('Normalized Monthly Revenue Heatmap (Top 8 Categories)',
            color=LABEL_COLOR, fontweight='bold')
ax.set_xlabel('Month', color=LABEL_COLOR)
ax.set_ylabel('Product Category', color=LABEL_COLOR)

# Highlight months 8-10
for month_idx in [7, 8, 9]: # 0-indexed
    ax.add_patch(plt.Rectangle((month_idx, 0), 1, len(top8_cats),
                              fill=False, edgecolor='#2c3e50',
                              linewidth=2.5, linestyle='--'))
ax.text(8.5, -0.5, 'Peak\nSeason', ha='center', fontsize=10,
       color='#2c3e50', fontweight='bold')

```

```

# Bottom: Line plot for top 4 categories
ax = axes[1]
top4_cats = df['Product_Category'].value_counts().head(4).index.tolist()
monthly_line = (df[df['Product_Category'].isin(top4_cats)]
                .groupby(['Month', 'Product_Category'])['Revenue']
                .sum()
                .unstack(fill_value=0))
colors_line = ['#e74c3c', '#3498db', '#2ecc71', '#9b59b6']
markers = ['o', 's', '^', 'D']
for cat, col, mk in zip(top4_cats, colors_line, markers):
    ax.plot(monthly_line.index, monthly_line[cat],
            marker=mk, color=col, linewidth=2.5, markersize=8, label=cat)
    # annotate peak month
    peak_m = monthly_line[cat].idxmax()
    peak_v = monthly_line[cat].max()
    ax.annotate(f'Peak\nM{peak_m}',
               xy=(peak_m, peak_v),
               xytext=(peak_m + 0.3, peak_v * 1.02),
               fontsize=8, color=col, fontweight='bold')

# Shade months 8-10
ax.axvspan(7.5, 10.5, alpha=0.1, color='orange', label='Seasonal Peak (Aug-Oct)')
ax.set_title('Monthly Revenue Trend — Top 4 Categories',
            color=LABEL_COLOR, fontweight='bold')
ax.set_xlabel('Month', color=LABEL_COLOR)
ax.set_ylabel('Total Revenue ($)', color=LABEL_COLOR)
ax.set_xticks(range(1, 13))
ax.legend(title='Category', bbox_to_anchor=(1.01, 1), loc='upper left')
ax.grid(alpha=0.3)

plt.tight_layout()
plt.show()
print('Observation: Clear seasonal peaks visible in months 8-10 across most categories.')
print('Nest-USA consistently generates the highest monthly revenue.')
print('The heatmap reveals that peak timing slightly varies by category,')
print('suggesting different marketing strategies may be needed per product line.')

print('\n' + '=' * 60)

```

```
print('ALL PROPOSAL INSIGHT PLOTS COMPLETE')
print('Phase I fully covers all proposal findings!')
print('=' * 60)

df.dropna(subset=['Online_Spend', 'Offline_Spend', 'Avg_Price', 'Tenure_Months', 'Quantity'],
inplace=True)
num_cols = ['Tenure_Months', 'Quantity', 'Avg_Price', 'Delivery_Charges', 'GST', 'Offline_Spend',
'Online_Spend', 'Discount_pct']

# =====
# PLOT 28: HEXBIN PLOT — Online vs Offline Spend density
# =====
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Spending Patterns Analysis', fontsize=16, fontweight='bold')

ax = axes[0, 0]
hb = ax.hexbin(df['Offline_Spend'], df['Online_Spend'], gridsize=30, cmap='YlOrRd', mincnt=1)
fig.colorbar(hb, ax=ax, label='Count')
ax.set_xlabel('Offline Spend')
ax.set_ylabel('Online Spend')
ax.set_title('Online vs Offline Spend Density')

ax = axes[0, 1]
hb2 = ax.hexbin(df['Avg_Price'], df['Online_Spend'], gridsize=30, cmap='Blues', mincnt=1)
fig.colorbar(hb2, ax=ax, label='Count')
ax.set_xlabel('Avg Price')
ax.set_ylabel('Online Spend')
ax.set_title('Avg Price vs Online Spend')

ax = axes[1, 0]
hb3 = ax.hexbin(df['Tenure_Months'], df['Online_Spend'], gridsize=25, cmap='Greens', mincnt=1)
fig.colorbar(hb3, ax=ax, label='Count')
ax.set_xlabel('Tenure Months')
ax.set_ylabel('Online Spend')
ax.set_title('Tenure vs Online Spend')

ax = axes[1, 1]
hb4 = ax.hexbin(df['Quantity'], df['Online_Spend'], gridsize=25, cmap='Purples', mincnt=1)
fig.colorbar(hb4, ax=ax, label='Count')
```

```
ax.set_xlabel('Quantity')
ax.set_ylabel('Online Spend')
ax.set_title('Quantity vs Online Spend')

plt.tight_layout()
plt.savefig('plot_28_hexbin.png', dpi=150, bbox_inches='tight')
plt.close()
print("PLOT 28 saved")

# =====
# PLOT 29: STRIP PLOT
# =====
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('Strip Plot Analysis', fontsize=16, fontweight='bold')

sns.stripplot(data=df, x='Gender', y='Online_Spend', hue='Coupon_Status',
              dodge=True, alpha=0.4, jitter=True, ax=axes[0], palette='Set2')
axes[0].set_title('Online Spend by Gender & Coupon Status')
axes[0].set_xlabel('Gender')
axes[0].set_ylabel('Online Spend')

sns.stripplot(data=df, x='Location', y='Online_Spend', hue='Gender',
              dodge=True, alpha=0.4, jitter=True, ax=axes[1], palette='Set1')
axes[1].set_title('Online Spend by Location & Gender')
axes[1].set_xlabel('Location')
axes[1].set_ylabel('Online Spend')
axes[1].tick_params(axis='x', rotation=30)

plt.tight_layout()
plt.savefig('plot_29_strip_plot.png', dpi=150, bbox_inches='tight')
plt.close()
print("PLOT 29 saved")

# =====
# PLOT 30: SWARM PLOT
# =====
df_sample = df.sample(n=min(800, len(df)), random_state=42)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
```

```
fig.suptitle('Swarm Plot Analysis', fontsize=16, fontweight='bold')

sns.swarmplot(data=df_sample, x='Gender', y='Online_Spend', hue='Coupon_Status',
              dodge=True, size=3, ax=axes[0], palette='Set2')
axes[0].set_title('Online Spend Distribution by Gender')
axes[0].set_xlabel('Gender')
axes[0].set_ylabel('Online Spend')

sns.swarmplot(data=df_sample, x='Coupon_Status', y='Avg_Price', hue='Gender',
              dodge=True, size=3, ax=axes[1], palette='Set1')
axes[1].set_title('Avg Price by Coupon Status')
axes[1].set_xlabel('Coupon Status')
axes[1].set_ylabel('Avg Price')

plt.tight_layout()
plt.savefig('plot_30_swarm_plot.png', dpi=150, bbox_inches='tight')
plt.close()
print("PLOT 30 saved")

# =====
# PLOT 31: KDE MULTIVARIATE (subplot 방식)
# =====
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('KDE & Statistical Distribution Analysis', fontsize=16, fontweight='bold')

# KDE: Online Spend by Gender
for gender, grp in df.groupby('Gender'):
    grp['Online_Spend'].plot.kde(ax=axes[0,0], label=gender, linewidth=2)
axes[0,0].set_title('KDE: Online Spend by Gender')
axes[0,0].set_xlabel('Online Spend')
axes[0,0].legend()

# KDE: Avg Price by Coupon Status
for status, grp in df.groupby('Coupon_Status'):
    grp['Avg_Price'].plot.kde(ax=axes[0,1], label=status, linewidth=2)
axes[0,1].set_title('KDE: Avg Price by Coupon Status')
axes[0,1].set_xlabel('Avg Price')
axes[0,1].legend()
```

```

# KDE: Offline vs Online Spend overlay
df['Online_Spend'].plot.kde(ax=axes[1,0], label='Online', linewidth=2, color='steelblue')
df['Offline_Spend'].plot.kde(ax=axes[1,0], label='Offline', linewidth=2, color='coral')
axes[1,0].set_title('KDE: Online vs Offline Spend')
axes[1,0].set_xlabel('Spend Amount')
axes[1,0].legend()

# KDE: Tenure by Location (top 3)
top3_loc = df['Location'].value_counts().index[:3]
for loc in top3_loc:
    df[df['Location']==loc]['Tenure_Months'].plot.kde(ax=axes[1,1], label=loc, linewidth=2)
axes[1,1].set_title('KDE: Tenure by Location (Top 3)')
axes[1,1].set_xlabel('Tenure Months')
axes[1,1].legend()

plt.tight_layout()
plt.savefig('plot_31_stats_kde_multivariate.png', dpi=150, bbox_inches='tight')
plt.close()
print("PLOT 31 saved")

# =====
# PLOT 32: OUTLIER DETECTION — Boxplot + IQR
# =====
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Outlier Detection Analysis', fontsize=16, fontweight='bold')

cols_to_check = ['Online_Spend', 'Offline_Spend', 'Avg_Price', 'Delivery_Charges']
colors = ['steelblue', 'coral', 'seagreen', 'mediumpurple']

for i, (col, color) in enumerate(zip(cols_to_check, colors)):
    ax = axes[i//2, i%2]
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower) | (df[col] > upper)][col]

    ax.boxplot(df[col].dropna(), vert=True, patch_artist=True,

```

```
        boxprops=dict(facecolor=color, alpha=0.6),
        medianprops=dict(color='black', linewidth=2))
ax.set_title(f'{col}\n(Outliers: {len(outliers)}, {len(outliers)/len(df)*100:.1f}%)')
ax.set_ylabel(col)
ax.set_xticks([])

plt.tight_layout()
plt.savefig('plot_32_outlier_boxplot.png', dpi=150, bbox_inches='tight')
plt.close()
print("PLOT 32 saved")
```

Phase 2

```
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import plotly.figure_factory as ff
from plotly.subplots import make_subplots
from scipy import stats
from scipy.stats import shapiro, normaltest, zscore, skew, kurtosis
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.decomposition import PCA
import statsmodels.api as sm
import warnings
warnings.filterwarnings('ignore')

import dash
from dash import dcc, html, Input, Output, State, dash_table
import dash_bootstrap_components as dbc
import dash_daq as daq

# =====
# DATA LOADING & PREPROCESSING
# =====
DATA_PATH = 'online_shopping.csv'
df_raw = pd.read_csv(DATA_PATH)
df_raw = df_raw.drop(columns=['Unnamed: 0'], errors='ignore')
```

```
col_required = ['CustomerID','Gender','Location','Tenure_Months','Transaction_ID',
               'Quantity','Avg_Price','Delivery_Charges','Coupon_Status',
               'GST','Offline_Spend','Online_Spend']
df = df_raw.dropna(subset=col_required).copy()
df['Transaction_Date'] = pd.to_datetime(df['Transaction_Date'])
df['CustomerID']      = df['CustomerID'].astype(int)
df['Transaction_ID']  = df['Transaction_ID'].astype(int)
df['Total_Spend']     = df['Offline_Spend'] + df['Online_Spend']
df['Revenue']         = df['Quantity'] * df['Avg_Price']
df['Month']           = df['Transaction_Date'].dt.month
df['Price_Bin']       = pd.cut(df['Avg_Price'],
                               bins=[0,25,50,100,200,400],
                               labels=['$0-25','$25-50','$50-100','$100-200','$200+'])
```

```
CAT_COLS = ['Gender','Location','Product_Category','Coupon_Status']
NUM_COLS = ['Tenure_Months','Quantity','Avg_Price','Delivery_Charges',
            'GST','Offline_Spend','Online_Spend','Discount_pct','Total_Spend','Revenue']
```

```
PALETTE = px.colors.qualitative.Set2
C_F      = '#e74c3c'
C_M      = '#3498db'
TEMPLATE = 'plotly_white'
```

```
# =====
# HELPER: standard figure layout
# =====
def apply_layout(fig, title="", xlab="", ylab="):
    fig.update_layout(
        title=dict(text=title, font=dict(size=18, color='#2c3e50'), x=0.5),
        xaxis_title=xlab, yaxis_title=ylab,
        template=TEMPLATE, margin=dict(t=60, b=50, l=60, r=30),
        legend=dict(bgcolor='rgba(255,255,255,0.8)', bordercolor='#ccc', borderwidth=1),
        font=dict(family='Arial', size=13)
    )
    return fig
```

```
# =====
# APP INIT
# =====
```

```
app = dash.Dash(__name__,
                 external_stylesheets=[dbc.themes.FLATLY],
                 suppress_callback_exceptions=True)
app.title = 'Online Shopping Dashboard — DATS6401'

# =====
# NAVBAR
# =====
navbar = dbc.NavbarSimple(
    brand='🛒 Online Shopping Analytics Dashboard',
    brand_href='#',
    color='primary',
    dark=True,
    children=[
        dbc.NavItem(dbc.NavLink('DATS6401 | Jeongwon Yoo', href='#')),
    ]
)

# =====
# TAB CONTENTS
# =====

# ——— TAB 1: Load Data

tab_load = dbc.Tab(label='📁 Load Data', tab_id='tab-load', children=[
    dbc.Container([
        dbc.Row([
            dbc.Col([
                dbc.Card([
                    dbc.CardHeader(html.H5('Dataset Overview', className='mb-0')),
                    dbc.CardBody([
                        dbc.Row([
                            dbc.Col([
                                html.Label('Select display rows:'),
                                dbc.Select(id='load-rows',
                                           options=[{'label': f'{n} rows', 'value': n}
                                                    for n in [5,10,20,50]],
                                           value=10),
                            ], width=3),

```

```
dbc.Col([
    html.Label('Filter by Column:'),
    dbc.Select(id='load-col-filter',
               options=[{'label':'All','value':'All'}] +
               [{'label':c,'value':c} for c in df.columns],
               value='All'),
], width=3),
dbc.Col([
    dbc.Button('Load Data', id='btn-load', color='primary',
               className='mt-4'),
    dbc.Spinner(html.Div(id='load-status'), size='sm'),
], width=3),
], className='mb-3'),
html.Hr(),
dbc.Row([
    dbc.Col(dbc.Card([
        dbc.CardBody([
            html.H6('Total Rows'),
            dbc.Badge(f'{len(df):,}', color='primary', className='fs-5'),
        ])
    ]), width=2),
    dbc.Col(dbc.Card([
        dbc.CardBody([
            html.H6('Columns'),
            dbc.Badge(f'{df.shape[1]}', color='info', className='fs-5'),
        ])
    ]), width=2),
    dbc.Col(dbc.Card([
        dbc.CardBody([
            html.H6('Missing Values'),
            dbc.Badge(f'{df.isnull().sum().sum()}', color='success', className='fs-5'),
        ])
    ]), width=2),
    dbc.Col(dbc.Card([
        dbc.CardBody([
            html.H6('Unique Customers'),
            dbc.Badge(f'{df["CustomerID"].nunique():,}', color='warning', className='fs-5'),
        ])
    ]), width=2),
], width=2),
```

```

        dbc.Col(dbc.Card([
            dbc.CardBody([
                html.H6('Date Range'),
                dbc.Badge(f'{df["Transaction_Date"].min().date()} ~
{df["Transaction_Date"].max().date()}',
                    color='secondary', className='fs-6'),
            ])
        ]), width=4),
        ], className='mb-3'),
        html.Div(id='load-table-div'),
    ])
])
]),
dbc.Row([
    dbc.Col([
        dbc.Card([
            dbc.CardHeader(html.H5('Descriptive Statistics')),
            dbc.CardBody([
                html.Div(id='load-stats-div'),
            ])
        ])
    ])
], className='mt-3'),
], fluid=True, className='py-3')
])

```

——— TAB 2: Data Cleaning

```

tab_clean = dbc.Tab(label='🔧 Data Cleaning', tab_id='tab-clean', children=[
    dbc.Container([
        dbc.Card([
            dbc.CardHeader(html.H5('Data Cleaning Methods')),
            dbc.CardBody([
                dbc.Row([
                    dbc.Col([
                        html.Label('Cleaning Method:'),
                        dbc.RadioItems(
                            id='clean-method',

```

```

        options=[
            {'label': 'Drop NA rows', 'value': 'drop'},
            {'label': 'Fill with Mean', 'value': 'mean'},
            {'label': 'Fill with Median', 'value': 'median'},
            {'label': 'Fill with Mode', 'value': 'mode'},
            {'label': 'Forward Fill', 'value': 'ffill'},
        ],
        value='drop', inline=False,
    ),
], width=3),
dbc.Col([
    html.Label('Select Feature to Check:'),
    dbc.Select(id='clean-feature',
        options=[{'label': c, 'value': c} for c in
            ['Tenure_Months', 'Quantity', 'Avg_Price',
             'Delivery_Charges', 'GST',
             'Offline_Spend', 'Online_Spend', 'Discount_pct']],
        value='Online_Spend'),
    html.Br(),
    dbc.Button('Apply Cleaning', id='btn-clean', color='success'),
], width=3),
dbc.Col([
    dbc.Alert(id='clean-alert', color='info',
        children='Select a method and click Apply.'),
    dbc.Row([
        dbc.Col(daq.LEDDisplay(id='led-missing-before',
            label='Missing Before',
            value='0', size=24,
            color='#e74c3c'), width=6),
        dbc.Col(daq.LEDDisplay(id='led-missing-after',
            label='Missing After',
            value='0', size=24,
            color='#2ecc71'), width=6),
    ]),
], width=6),
], className='mb-3'),
dcc.Graph(id='clean-graph'),
])
)),

```

```
], fluid=True, className='py-3')
])
```

— TAB 3: Outlier Detection

```
tab_outlier = dbc.Tab(label='🔍 Outlier Detection', tab_id='tab-outlier', children=[
    dbc.Container([
        dbc.Card([
            dbc.CardHeader(html.H5('Outlier Detection & Removal')),
            dbc.CardBody([
                dbc.Row([
                    dbc.Col([
                        html.Label('Detection Method:'),
                        dbc.RadioItems(
                            id='outlier-method',
                            options=[
                                {'label': 'IQR (1.5×)', 'value': 'iqr'},
                                {'label': 'Z-Score ( $|z| > 3$ )', 'value': 'zscore'},
                                {'label': 'Modified Z-Score', 'value': 'mod_z'},
                            ],
                            value='iqr',
                        ),
                    ], width=3),
                    dbc.Col([
                        html.Label('Feature:'),
                        dbc.Select(id='outlier-feature',
                            options=[{'label': c, 'value': c} for c in NUM_COLS],
                            value='Online_Spend'),
                        html.Br(),
                        html.Label('IQR Multiplier:'),
                        dcc.Slider(id='outlier-k', min=1.0, max=3.0, step=0.5,
                            value=1.5, marks={1.0: '1.0', 1.5: '1.5', 2.0: '2.0',
                                2.5: '2.5', 3.0: '3.0'}),
                        dbc.Button('Detect Outliers', id='btn-outlier', color='warning', className='mt-2'),
                    ], width=4),
                    dbc.Col([
                        dbc.Row([
                            dbc.Col(daq.Gauge(id='gauge-outlier-pct',
                                label='Outlier %',
```

```

        dbc.Checklist(
            id='pca-features',
            options=[{'label':c,'value':c} for c in NUM_COLS],
            value=NUM_COLS,
            inline=False,
        ),
    ], width=3),
    dbc.Col([
        html.Label('Number of Components:'),
        dcc.Slider(id='pca-n', min=2, max=10, step=1, value=5,
            marks={i:str(i) for i in range(2,11)}),
        html.Br(),
        html.Label('Color by:'),
        dbc.Select(id='pca-color',
            options=[{'label':c,'value':c} for c in CAT_COLS],
            value='Gender'),
        html.Br(),
        dbc.Button('Run PCA', id='btn-pca', color='primary'),
    ], width=4),
    dbc.Col([
        daq.GraduatedBar(id='grad-bar-pca',
            label='Cumulative Variance (%)',
            min=0, max=100, value=0,
            showCurrentValue=True, size=200,
            color={'gradient':True,
                'ranges':{'red':[0,60],
                    'yellow':[60,80],
                    'green':[80,100]}}),
        html.Br(),
        dbc.Alert(id='pca-cond-alert', color='secondary',
            children='Condition number will appear here.'),
    ], width=5),
], className='mb-3'),
    dbc.Row([
        dbc.Col(dcc.Graph(id='pca-scree'), width=6),
        dbc.Col(dcc.Graph(id='pca-biplot'), width=6),
    ]),
    dbc.Row([
        dbc.Col(dcc.Graph(id='pca-cumvar'), width=12),

```

```

        min=-5, max=5, value=0,
        showCurrentValue=True,
        height=120, width=20,
        color='#3498db'), width=4),
    dbc.Col(daq.Thermometer(id='thermo-kurt',
        label='Kurtosis',
        min=-3, max=10, value=0,
        showCurrentValue=True,
        height=120, width=20,
        color='#e74c3c'), width=4),
    dbc.Col([
        daq.BooleanSwitch(id='norm-switch',
            label='Normal?',
            on=False,
            color='#2ecc71'),
    ], width=4),
    ],
    ], width=4),
    dbc.Col([
        daq.Knob(id='knob-pval',
            label='p-value',
            min=0, max=1, value=0,
            size=120,
            color={'gradient':True,
                'ranges':{'green':[0,0.05],
                    'yellow':[0.05,0.1],
                    'red':[0.1,1]}}),
    ], width=2),
    dbc.Col([
        daq.ColorPicker(id='norm-color',
            label='Plot Color',
            value=dict(hex='#3498db')),
    ], width=3),
    ], className='mb-3'),
    dbc.Row([
        dbc.Col(dcc.Graph(id='norm-hist'), width=6),
        dbc.Col(dcc.Graph(id='norm-qq'), width=6),
    ]),
    ])

```

```

    ]),
    ], fluid=True, className='py-3')
})

```

— TAB 6: Data Transformation

```

tab_transform = dbc.Tab(label='🔧 Data Transformation', tab_id='tab-transform', children=[
    dbc.Container([
        dbc.Card([
            dbc.CardHeader(html.H5('Data Transformation')),
            dbc.CardBody([
                dbc.Row([
                    dbc.Col([
                        html.Label('Feature:'),
                        dbc.Select(id='trans-feature',
                                options=[{'label':c,'value':c} for c in NUM_COLS],
                                value='Online_Spend'),
                        html.Br(),
                        html.Label('Transformation Method:'),
                        dbc.Select(id='trans-method',
                                options=[
                                    {'label':'Log (log1p)','value':'log'},
                                    {'label':'Square Root','value':'sqrt'},
                                    {'label':'Z-Score Standardization','value':'zscore'},
                                    {'label':'Min-Max Normalization','value':'minmax'},
                                    {'label':'Box-Cox','value':'boxcox'},
                                ],
                                value='log'),
                        html.Br(),
                        dbc.Button('Apply', id='btn-transform', color='success'),
                    ], width=3),
                    dbc.Col([
                        dbc.Row([
                            dbc.Col([
                                html.P('Before:', style={'fontWeight':'bold'}),
                                daq.LEDDisplay(id='led-skew-before', label='Skewness',
                                                value='0.00', size=20, color='#e74c3c'),
                                html.Br(),
                                daq.LEDDisplay(id='led-mean-before', label='Mean',

```

```

        value='0.00', size=20, color='#3498db'),
    ], width=6),
    dbc.Col([
        html.P('After:', style={'fontWeight':'bold'}),
        daq.LEDDisplay(id='led-skew-after', label='Skewness',
            value='0.00', size=20, color='#2ecc71'),
        html.Br(),
        daq.LEDDisplay(id='led-mean-after', label='Mean',
            value='0.00', size=20, color='#9b59b6'),
    ], width=6),
    ]),
    dbc.Alert(id='transform-alert', color='info',
        children='Select a feature and transformation.'),
    ], width=4),
    dbc.Col([
        dcc.Download(id='download-transformed'),
        dbc.Button('↓ Download Transformed CSV',
            id='btn-download-transform',
            color='secondary', outline=True, className='mt-2'),
    ], width=5),
    ], className='mb-3'),
    dbc.Row([
        dbc.Col(dcc.Graph(id='trans-before'), width=6),
        dbc.Col(dcc.Graph(id='trans-after'), width=6),
    ]),
    ])
    ], fluid=True, className='py-3')
])

```

— TAB 7: Numerical Feature Plots

```

tab_num = dbc.Tab(label='☑ Numerical Plots', tab_id='tab-num', children=[
    dbc.Container([
        dbc.Card([
            dbc.CardHeader(html.H5('Numerical Feature Visualization')),
            dbc.CardBody([
                dbc.Row([
                    dbc.Col([

```

```
html.Label('X Feature:'),
dbc.Select(id='num-x',
           options=[{'label':c,'value':c} for c in NUM_COLS],
           value='Online_Spend'),
html.Br(),
html.Label('Y Feature (for 2D plots):'),
dbc.Select(id='num-y',
           options=[{'label':c,'value':c} for c in NUM_COLS],
           value='Offline_Spend'),
html.Br(),
html.Label('Color by:'),
dbc.Select(id='num-hue',
           options=[{'label':'None','value':'None'}] +
                 [{'label':c,'value':c} for c in CAT_COLS],
           value='Gender'),
], width=3),
dbc.Col([
    html.Label('Plot Type:'),
    dbc DropdownMenu(
        label='Select Plot Type',
        id='num-plot-dropdown',
        children=[
            dbc DropdownMenuItem('Histogram + KDE', id='dd-hist'),
            dbc DropdownMenuItem('KDE Plot', id='dd-kde'),
            dbc DropdownMenuItem('Box Plot', id='dd-box'),
            dbc DropdownMenuItem('Violin Plot', id='dd-violin'),
            dbc DropdownMenuItem('Scatter + Regression', id='dd-scatter'),
            dbc DropdownMenuItem('QQ Plot', id='dd-qq'),
            dbc DropdownMenuItem('Dist Plot', id='dd-dist'),
            dbc DropdownMenuItem('Area Plot', id='dd-area'),
            dbc DropdownMenuItem('Rug Plot', id='dd-rug'),
            dbc DropdownMenuItem('Hexbin / Density', id='dd-hexbin'),
            dbc DropdownMenuItem('3D Scatter', id='dd-scatter3d'),
            dbc DropdownMenuItem('Contour', id='dd-contour'),
            dbc DropdownMenuItem('Joint Plot', id='dd-joint'),
        ],
        color='secondary', className='mb-2',
    ),
    dbc.Tooltip('Choose a plot type from the dropdown menu.',
```

```

        target='num-plot-dropdown'),
    dcc.Store(id='num-plot-type', data='hist'),
    html.Br(),
    html.Label('Bin Range (for histogram):'),
    dcc.RangeSlider(id='num-bins', min=10, max=100, step=10,
        value=[10, 50],
        marks={10:'10', 30:'30', 50:'50',
            70:'70', 100:'100'}),
    dbc.Tooltip('Drag both handles to set the min and max bin count.',
        target='num-bins'),
    html.Br(),
    dbc.Button('Plot', id='btn-num-plot', color='primary'),
    dbc.Tooltip('Click to generate the selected plot.',
        target='btn-num-plot'),
], width=4),
dbc.Col([
    dbc.Alert(id='num-stats-alert', color='light',
        children='Stats will appear here.'),
    html.Label('Observations / Notes:'),
    dcc.Textarea(id='num-textarea',
        placeholder='Write your observations about this plot here...',
        style={'width':'100%', 'height':'120px',
            'fontSize':'13px'}),
    dbc.Tooltip('Use this area to note your observations about the plot.',
        target='num-textarea'),
], width=5),
], className='mb-3'),
dbc.Spinner(dcc.Graph(id='num-graph', style={'height':'550px'})),
])
]),
], fluid=True, className='py-3')
])

```

— TAB 8: Categorical Feature Plots

```

tab_cat = dbc.Tab(label='Categorical Plots', tab_id='tab-cat', children=[
    dbc.Container([
        dbc.Card([
            dbc.CardHeader(html.H5('Categorical Feature Visualization')),
            dbc.CardBody([

```

```
dbc.Row([
  dbc.Col([
    html.Label('Categorical Feature:'),
    dbc.Select(id='cat-x',
      options=[{'label':c,'value':c} for c in CAT_COLS],
      value='Product_Category'),
    html.Br(),
    html.Label('Value Feature (Y-axis):'),
    dbc.Select(id='cat-y',
      options=[{'label':c,'value':c} for c in NUM_COLS],
      value='Revenue'),
    html.Br(),
    html.Label('Hue (Group by):'),
    dbc.Select(id='cat-hue',
      options=[{'label':'None','value':'None'} +
        [{'label':c,'value':c} for c in CAT_COLS],
      value='Gender'),
  ], width=3),
  dbc.Col([
    html.Label('Plot Type:'),
    dbc.Select(id='cat-plot-type',
      options=[
        {'label':'Bar Plot (sorted)','value':'bar'},
        {'label':'Stacked Bar','value':'bar_stack'},
        {'label':'Grouped Bar','value':'bar_group'},
        {'label':'Count Plot','value':'count'},
        {'label':'Pie Chart','value':'pie'},
        {'label':'Box Plot','value':'box'},
        {'label':'Violin Plot','value':'violin'},
        {'label':'Strip Plot','value':'strip'},
        {'label':'Swarm Plot','value':'swarm'},
        {'label':'Boxen Plot','value':'boxen'},
      ],
      value='bar'),
    html.Br(),
    html.Label('Aggregation (for bar):'),
    dbc.RadioItems(id='cat-agg',
      options=[
        {'label':'Mean','value':'mean'},
```

```

        {'label':'Sum','value':'sum'},
        {'label':'Count','value':'count'},
        {'label':'Median','value':'median'},
    ],
    value='mean', inline=True),
    html.Br(),
    dbc.Button('Plot', id='btn-cat-plot', color='success'),
], width=4),
dbc.Col([
    dbc.Alert(id='cat-info-alert', color='light',
        children='Category info will appear here.'),
], width=5),
], className='mb-3'),
dbc.Spinner(dcc.Graph(id='cat-graph', style={'height':'550px'})),
])
)),
], fluid=True, className='py-3')
])

```

— TAB 9: Statistics

```

tab_stats = dbc.Tab(label='📊 Statistics', tab_id='tab-stats', children=[
    dbc.Container([
        dbc.Row([
            dbc.Col([
                dbc.Card([
                    dbc.CardHeader(html.H5('Heatmap & Correlation')),
                    dbc.CardBody([
                        dbc.Row([
                            dbc.Col([
                                html.Label('Features:'),
                                dbc.Checklist(
                                    id='stats-heatmap-features',
                                    options=[{'label':c,'value':c} for c in NUM_COLS],
                                    value=NUM_COLS[:6],
                                    inline=False,
                                ),
                            ], width=3),
                            dbc.Col([

```

```

        html.Label('Color Scale:'),
        dbc.Select(id='stats-cmap',
                   options=[
                       {'label':'RdBu','value':'RdBu'},
                       {'label':'Viridis','value':'Viridis'},
                       {'label':'Plasma','value':'Plasma'},
                       {'label':'Coolwarm','value':'RdYlBu'},
                   ],
                   value='RdBu'),
        html.Br(),
        dbc.Button('Update', id='btn-stats-heat', color='primary'),
    ], width=3),
    dbc.Col([
        dbc.Alert(id='stats-corr-alert', color='light',
                  children='Highest correlation will appear here.'),
    ], width=6),
], className='mb-2'),
dcc.Graph(id='stats-heatmap'),
])
]),
], width=6),
dbc.Col([
    dbc.Card([
        dbc.CardHeader(html.H5('Scatter Matrix (Pair Plot)'),
        dbc.CardBody([
            dbc.Row([
                dbc.Col([
                    html.Label('Select features (max 5):'),
                    dbc.Checklist(
                        id='stats-pair-features',
                        options=[{'label':c,'value':c} for c in NUM_COLS],
                        value=['Avg_Price','Online_Spend',
                            'Offline_Spend','Revenue'],
                        inline=False,
                    ),
                ], width=5),
                dbc.Col([
                    html.Label('Color by:'),
                    dbc.Select(id='stats-pair-hue',

```

```

        options=[{'label':c,'value':c}
                  for c in CAT_COLS],
        value='Gender'),
        html.Br(),
        dbc.Button('Update', id='btn-stats-pair', color='success'),
    ], width=7),
], className='mb-2'),
dbc.Spinner(dcc.Graph(id='stats-pair')),
])
]),
], width=6),
], className='mb-3'),
dbc.Row([
    dbc.Col([
        dbc.Card([
            dbc.CardHeader(html.H5('Multivariate KDE & Statistics Table')),
            dbc.CardBody([
                dbc.Row([
                    dbc.Col([
                        html.Label('X Feature:'),
                        dbc.Select(id='stats-kde-x',
                                  options=[{'label':c,'value':c}
                                             for c in NUM_COLS],
                                  value='Online_Spend'),
                        html.Label('Y Feature:'),
                        dbc.Select(id='stats-kde-y',
                                  options=[{'label':c,'value':c}
                                             for c in NUM_COLS],
                                  value='Offline_Spend'),
                        html.Br(),
                        dbc.Button('Update KDE', id='btn-stats-kde',
                                  color='info'),
                    ], width=3),
                    dbc.Col(dcc.Graph(id='stats-kde'), width=9),
                ]),
            ])
        ], width=7),
        dbc.Col([

```

```

        dbc.Card([
            dbc.CardHeader(html.H5('Descriptive Statistics Table')),
            dbc.CardBody([
                html.Div(id='stats-table-div'),
            ])
        ]),
    ], width=5),
]),
], fluid=True, className='py-3')
])

# =====
# MAIN LAYOUT
# =====
app.layout = dbc.Container([
    navbar,
    html.Br(),
    dbc.Tabs([
        tab_load,
        tab_clean,
        tab_outlier,
        tab_pca,
        tab_normality,
        tab_transform,
        tab_num,
        tab_cat,
        tab_stats,
    ], id='main-tabs', active_tab='tab-load'),
], fluid=True)

# =====
# CALLBACKS — All tabs
# =====
from dashboard import (app, df, df_raw, NUM_COLS, CAT_COLS,
                       PALETTE, C_F, C_M, TEMPLATE, apply_layout)

import numpy as np
import pandas as pd
import plotly.express as px

```

```
import plotly.graph_objects as go
import plotly.figure_factory as ff
from plotly.subplots import make_subplots
from scipy import stats
from scipy.stats import shapiro, normaltest, zscore, skew, kurtosis
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.decomposition import PCA
import statsmodels.api as sm
```

```
from dash import Input, Output, State, dash_table, dcc
import dash_bootstrap_components as dbc
import warnings
warnings.filterwarnings('ignore')
```

```
#
```

```
# TAB 1: Load Data
```

```
#
```

```
@app.callback(
    Output('load-table-div', 'children'),
    Output('load-stats-div', 'children'),
    Output('load-status', 'children'),
    Input('btn-load', 'n_clicks'),
    State('load-rows', 'value'),
    State('load-col-filter', 'value'),
    prevent_initial_call=False,
)
def load_data(n, rows, col_filter):
    rows = int(rows) if rows else 10
    display = df.head(rows)
    if col_filter and col_filter != 'All':
        display = display[[col_filter]]

    table = dash_table.DataTable(
        data=display.astype(str).to_dict('records'),
```

```
columns=[{'name': c, 'id': c} for c in display.columns],
style_table={'overflowX': 'auto'},
style_cell={'textAlign': 'left', 'padding': '6px',
            'fontFamily': 'Arial', 'fontSize': '13px'},
style_header={'backgroundColor': '#2c3e50', 'color': 'white',
              'fontWeight': 'bold'},
style_data_conditional=[
    {'if': {'row_index': 'odd'}, 'backgroundColor': '#f8f9fa'}],
page_size=rows,
)
```

```
desc = df[NUM_COLS].describe().round(2).reset_index()
stats_table = dash_table.DataTable(
    data=desc.to_dict('records'),
    columns=[{'name': c, 'id': c} for c in desc.columns],
    style_table={'overflowX': 'auto'},
    style_cell={'textAlign': 'center', 'padding': '5px', 'fontSize': '12px'},
    style_header={'backgroundColor': '#34495e', 'color': 'white',
                  'fontWeight': 'bold'},
    style_data_conditional=[
        {'if': {'row_index': 'odd'}, 'backgroundColor': '#ecf0f1'}],
)
```

```
status = dbc.Alert('✅ Data loaded successfully!', color='success',
                  duration=3000)
return table, stats_table, status
```

```
#
```

```
# TAB 2: Data Cleaning
```

```
#
```

```
@app.callback(
    Output('clean-graph', 'figure'),
    Output('clean-alert', 'children'),
```

```

Output('clean-alert', 'color'),
Output('led-missing-before', 'value'),
Output('led-missing-after', 'value'),
Input('btn-clean', 'n_clicks'),
State('clean-method', 'value'),
State('clean-feature', 'value'),
prevent_initial_call=False,
)
def clean_data(n, method, feature):
    missing_before = int(df_raw[feature].isnull().sum()) if feature in df_raw.columns else 0

    tmp = df_raw[[feature]].copy()
    if method == 'drop':
        tmp = tmp.dropna()
    elif method == 'mean':
        tmp[feature] = tmp[feature].fillna(tmp[feature].mean())
    elif method == 'median':
        tmp[feature] = tmp[feature].fillna(tmp[feature].median())
    elif method == 'mode':
        tmp[feature] = tmp[feature].fillna(tmp[feature].mode()[0])
    elif method == 'ffill':
        tmp[feature] = tmp[feature].ffill()

    missing_after = int(tmp[feature].isnull().sum())

    fig = make_subplots(rows=1, cols=2,
                        subplot_titles=['Before Cleaning', 'After Cleaning'])
    fig.add_trace(go.Histogram(x=df_raw[feature], name='Before',
                              marker_color=C_F, opacity=0.7,
                              nbinsx=40), row=1, col=1)
    fig.add_trace(go.Histogram(x=tmp[feature], name='After',
                              marker_color=C_M, opacity=0.7,
                              nbinsx=40), row=1, col=2)
    fig.update_layout(title=f'Data Cleaning: {feature} — Method: {method.title()}',
                      template=TEMPLATE, showlegend=True,
                      font=dict(family='Arial', size=13))

    method_desc = {
        'drop': 'Rows with missing values dropped.',

```

```

    'mean': f'Missing values filled with mean ({df_raw[feature].mean():.2f}).',
    'median': f'Missing values filled with median ({df_raw[feature].median():.2f}).',
    'mode': f'Missing values filled with mode.',
    'ffill': 'Missing values forward-filled from previous row.',
}
msg = f'Method: {method.title()} | {method_desc.get(method,"")} | Missing before: {missing_before}
→ after: {missing_after}'
color = 'success' if missing_after == 0 else 'warning'

return fig, msg, color, str(missing_before), str(missing_after)

```

```
#
```

```
# TAB 3: Outlier Detection
```

```
#
```

```

@app.callback(
    Output('outlier-box', 'figure'),
    Output('outlier-hist', 'figure'),
    Output('outlier-alert', 'children'),
    Output('outlier-alert', 'color'),
    Output('gauge-outlier-pct', 'value'),
    Output('led-outlier-n', 'value'),
    Output('led-clean-n', 'value'),
    Input('btn-outlier', 'n_clicks'),
    State('outlier-method', 'value'),
    State('outlier-feature', 'value'),
    State('outlier-k', 'value'),
    prevent_initial_call=False,
)
def detect_outliers(n, method, feature, k):
    series = df[feature].dropna()

    if method == 'iqr':
        Q1, Q3 = series.quantile(0.25), series.quantile(0.75)

```

```

IQR = Q3 - Q1
mask = (series < Q1 - k*IQR) | (series > Q3 + k*IQR)
bounds = (Q1 - k*IQR, Q3 + k*IQR)
elif method == 'zscore':
    z = np.abs(zscore(series))
    mask = z > 3
    bounds = (series.mean() - 3*series.std(), series.mean() + 3*series.std())
else: # mod_z
    med = series.median()
    mad = np.median(np.abs(series - med))
    mod_z = 0.6745 * np.abs(series - med) / (mad + 1e-9)
    mask = mod_z > 3.5
    bounds = None

n_out = int(mask.sum())
n_clean = len(series) - n_out
pct = round(100 * n_out / len(series), 2)

# Box plot
fig_box = go.Figure()
fig_box.add_trace(go.Box(y=series[~mask], name='Normal',
                        marker_color=C_M, boxmean=True))
fig_box.add_trace(go.Box(y=series[mask], name='Outlier',
                        marker_color=C_F, boxmean=True))
apply_layout(fig_box, f'Box Plot: {feature} (outliers highlighted)',
             ", feature)

# Histogram with outlier shading
fig_hist = go.Figure()
fig_hist.add_trace(go.Histogram(x=series[~mask], name='Normal',
                              marker_color=C_M, opacity=0.7, nbinsx=50))
fig_hist.add_trace(go.Histogram(x=series[mask], name='Outlier',
                              marker_color=C_F, opacity=0.7, nbinsx=50))
if bounds:
    fig_hist.add_vline(x=bounds[0], line_dash='dash', line_color='red',
                      annotation_text=f'Lower: {bounds[0]:.2f}')
    fig_hist.add_vline(x=bounds[1], line_dash='dash', line_color='red',
                      annotation_text=f'Upper: {bounds[1]:.2f}')
apply_layout(fig_hist, f'Distribution: {feature} — Outlier Detection',

```

```

        feature, 'Count')
fig_hist.update_layout(barmode='overlay')

msg = (f'Method: {method.upper()} | Feature: {feature} | '
       f'Outliers: {n_out:}, ({pct:.2f}%) | Cleaned rows: {n_clean:,}')
color = 'danger' if pct > 10 else ('warning' if pct > 5 else 'success')

return fig_box, fig_hist, msg, color, pct, str(n_out), str(n_clean)

```

```
#
```

```
# TAB 4: PCA
```

```
#
```

```

@app.callback(
    Output('pca-scree', 'figure'),
    Output('pca-biplot', 'figure'),
    Output('pca-cumvar', 'figure'),
    Output('grad-bar-pca', 'value'),
    Output('pca-cond-alert', 'children'),
    Input('btn-pca', 'n_clicks'),
    State('pca-features', 'value'),
    State('pca-n', 'value'),
    State('pca-color', 'value'),
    prevent_initial_call=False,
)
def run_pca(n, features, n_comp, color_by):
    features = features or NUM_COLS
    X = df[features].dropna()
    scaler = StandardScaler()
    X_sc = scaler.fit_transform(X)
    pca = PCA()
    pca.fit(X_sc)

    evr = pca.explained_variance_ratio_ * 100

```

```

cum_var = np.cumsum(evr)
cond_n = np.linalg.cond(X_sc)

# Scree
fig_scre = go.Figure()
fig_scre.add_trace(go.Bar(x=[f'PC{i+1}' for i in range(len(evr))],
                        y=evr, marker_color=PALETTE[0],
                        text=[f'{v:.1f}%' for v in evr],
                        textposition='outside', name='Var %'))
apply_layout(fig_scre, 'Scree Plot — Explained Variance per PC',
              'Principal Component', 'Explained Variance (%)')

# Biplot
pca2 = PCA(n_components=2)
X_2d = pca2.fit_transform(X_sc)
samp = np.random.choice(len(X_2d), min(2000, len(X_2d)), replace=False)
color_vals = df[color_by].iloc[X.index].iloc[samp].values if color_by else None

fig_bi = px.scatter(x=X_2d[samp, 0], y=X_2d[samp, 1],
                   color=color_vals,
                   labels={'x': f'PC1 ({evr[0]:.1f}%)',
                           'y': f'PC2 ({evr[1]:.1f}%)',
                           'color': color_by},
                   opacity=0.5, template=TEMPLATE,
                   color_discrete_sequence=PALETTE)

# Loadings arrows
scale = 3
for i, feat in enumerate(features):
    fig_bi.add_annotation(
        x=pca2.components_[0, i]*scale,
        y=pca2.components_[1, i]*scale,
        ax=0, ay=0,
        xanchor='center', yanchor='bottom',
        text=feat, font=dict(size=10, color='red'),
        arrowcolor='red', arrowwidth=1.5, arrowhead=2,
    )
apply_layout(fig_bi, 'PCA Biplot (PC1 vs PC2)',
              f'PC1 ({evr[0]:.1f}%)', f'PC2 ({evr[1]:.1f}%)')

```

```
# Cumulative variance
fig_cum = go.Figure()
fig_cum.add_trace(go.Scatter(x=list(range(1, len(cum_var)+1)), y=cum_var,
                             mode='lines+markers+text',
                             text=[f'{v:.1f}%' for v in cum_var],
                             textposition='top center',
                             marker_color=PALETTE[1], line_width=2.5,
                             name='Cumulative Var'))
fig_cum.add_hline(y=95, line_dash='dash', line_color='red',
                  annotation_text='95% threshold')
fig_cum.add_hline(y=80, line_dash='dash', line_color='green',
                  annotation_text='80% threshold')
apply_layout(fig_cum, 'Cumulative Explained Variance',
              'Number of Components', 'Cumulative Variance (%)')

n_for_95 = int(np.argmax(cum_var >= 95)) + 1
cond_msg = (f'Condition Number: {cond_n:.2f} | '
            f'PCs needed for 95% variance: {n_for_95} | '
            f'Singular values range: {pca.singular_values_.min():.2f} – {pca.singular_values_.max():.2f}')

grad_val = round(float(cum_var[min(int(n_comp)-1, len(cum_var)-1)]), 1)
return fig_scee, fig_bi, fig_cum, grad_val, cond_msg
```

#

TAB 5: Normality Test

#

```
@app.callback(
    Output('norm-hist', 'figure'),
    Output('norm-qq', 'figure'),
    Output('norm-result', 'children'),
    Output('norm-result', 'color'),
    Output('thermo-skew', 'value'),
    Output('thermo-kurt', 'value'),
```

```

Output('norm-switch', 'on'),
Output('knob-pval', 'value'),
Input('btn-norm', 'n_clicks'),
State('norm-feature', 'value'),
State('norm-method', 'value'),
State('norm-color', 'value'),
prevent_initial_call=False,
)
def run_normality(n, feature, method, color_val):
    series = df[feature].dropna()
    hex_col = color_val.get('hex', '#3498db') if isinstance(color_val, dict) else '#3498db'
    sk_val = float(skew(series))
    kt_val = float(kurtosis(series))

    # Histogram
    fig_hist = go.Figure()
    fig_hist.add_trace(go.Histogram(x=series, nbinsx=50,
                                   marker_color=hex_col, opacity=0.7,
                                   histnorm='probability density', name='Data'))
    x_range = np.linspace(series.min(), series.max(), 200)
    mu, sig = series.mean(), series.std()
    fig_hist.add_trace(go.Scatter(x=x_range,
                                   y=stats.norm.pdf(x_range, mu, sig),
                                   mode='lines', line=dict(color='red', width=2.5),
                                   name='Normal PDF'))
    fig_hist.add_vline(x=mu, line_dash='dash', line_color='orange',
                       annotation_text=f'Mean: {mu:.2f}')
    apply_layout(fig_hist, f'Distribution of {feature}', feature, 'Density')

    # QQ Plot
    qq = sm.ProbPlot(series)
    fig_qq = go.Figure()
    fig_qq.add_trace(go.Scatter(x=qq.theoretical_quantiles,
                                y=qq.sample_quantiles,
                                mode='markers',
                                marker=dict(color=hex_col, size=3, opacity=0.5),
                                name='Sample'))
    lim = max(abs(qq.theoretical_quantiles.min()),
              abs(qq.theoretical_quantiles.max()))

```

```
fig_qq.add_trace(go.Scatter(x=[-lim, lim], y=[-lim, lim],
                           mode='lines', line=dict(color='red', width=2),
                           name='Normal line'))
apply_layout(fig_qq, f'QQ Plot: {feature}',
              'Theoretical Quantiles', 'Sample Quantiles')

# Tests
samp_n = series.sample(min(5000, len(series)), random_state=42)
results = []
min_p = 1.0

if method in ['shapiro', 'both']:
    sw_s, sw_p = shapiro(samp_n.sample(min(5000, len(samp_n)), random_state=42))
    results.append(f'Shapiro-Wilk: W={sw_s:.4f}, p={sw_p:.4f} '
                  f'→ {"Normal  " if sw_p>0.05 else "Not Normal  "}')
    min_p = min(min_p, sw_p)

if method in ['dagostino', 'both']:
    da_s, da_p = normaltest(samp_n)
    results.append(f'D'Agostino K²: stat={da_s:.4f}, p={da_p:.4f} '
                  f'→ {"Normal  " if da_p>0.05 else "Not Normal  "}')
    min_p = min(min_p, da_p)

results.append(f'Skewness: {sk_val:.2f} | Kurtosis: {kt_val:.2f}')
is_normal = min_p > 0.05
msg_color = 'success' if is_normal else 'danger'

sk_clamped = float(np.clip(sk_val, -5, 5))
kt_clamped = float(np.clip(kt_val, -3, 10))
p_display = float(np.clip(round(min_p, 4), 0, 1))

return (fig_hist, fig_qq, ' | '.join(results), msg_color,
        sk_clamped, kt_clamped, is_normal, p_display)
```

#

TAB 6: Data Transformation

#

```
@app.callback(
    Output('trans-before', 'figure'),
    Output('trans-after', 'figure'),
    Output('transform-alert', 'children'),
    Output('led-skew-before', 'value'),
    Output('led-skew-after', 'value'),
    Output('led-mean-before', 'value'),
    Output('led-mean-after', 'value'),
    Input('btn-transform', 'n_clicks'),
    State('trans-feature', 'value'),
    State('trans-method', 'value'),
    prevent_initial_call=False,
)

def transform_data(n, feature, method):
    series = df[feature].dropna()

    if method == 'log':
        transformed = np.log1p(series)
        label = 'Log (log1p)'
    elif method == 'sqrt':
        transformed = np.sqrt(np.abs(series))
        label = 'Square Root'
    elif method == 'zscore':
        transformed = pd.Series(zscore(series))
        label = 'Z-Score'
    elif method == 'minmax':
        mn, mx = series.min(), series.max()
        transformed = (series - mn) / (mx - mn + 1e-9)
        label = 'Min-Max'
    elif method == 'boxcox':
        shifted = series - series.min() + 1
        transformed, _ = stats.boxcox(shifted)
        transformed = pd.Series(transformed)
        label = 'Box-Cox'
    else:
```

```

transformed = series
label = 'Original'

sk_before = round(float(skew(series)), 2)
sk_after = round(float(skew(transformed)), 2)
mu_before = round(float(series.mean()), 2)
mu_after = round(float(transformed.mean()), 2)

x_b = np.linspace(series.min(), series.max(), 200)
x_a = np.linspace(transformed.min(), transformed.max(), 200)

fig_b = go.Figure()
fig_b.add_trace(go.Histogram(x=series, nbinsx=50,
                             marker_color=C_F, opacity=0.7,
                             histnorm='probability density', name='Original'))
fig_b.add_trace(go.Scatter(x=x_b,
                           y=stats.norm.pdf(x_b, series.mean(), series.std()),
                           mode='lines', line=dict(color='black', dash='dash'),
                           name='Normal PDF'))
apply_layout(fig_b, f'Before: {feature} (Skew={sk_before:.2f})',
              feature, 'Density')

fig_a = go.Figure()
fig_a.add_trace(go.Histogram(x=transformed, nbinsx=50,
                             marker_color=C_M, opacity=0.7,
                             histnorm='probability density', name=label))
fig_a.add_trace(go.Scatter(x=x_a,
                           y=stats.norm.pdf(x_a, transformed.mean(), transformed.std()),
                           mode='lines', line=dict(color='black', dash='dash'),
                           name='Normal PDF'))
apply_layout(fig_a, f'After: {label} (Skew={sk_after:.2f})',
              f'{feature} ({label})', 'Density')

msg = (f'Transformation: {label} | '
       f'Skewness: {sk_before:.2f} → {sk_after:.2f} | '
       f'Mean: {mu_before:.2f} → {mu_after:.2f}')

return (fig_b, fig_a, msg,

```

```
f'{sk_before:.2f}', f'{sk_after:.2f}',  
f'{mu_before:.2f}', f'{mu_after:.2f}')
```

```
@app.callback(  
    Output('download-transformed', 'data'),  
    Input('btn-download-transform', 'n_clicks'),  
    State('trans-feature', 'value'),  
    State('trans-method', 'value'),  
    prevent_initial_call=True,  
)  
def download_transformed(n, feature, method):  
    series = df[feature].dropna()  
    if method == 'log':  
        out = np.log1p(series)  
    elif method == 'sqrt':  
        out = np.sqrt(np.abs(series))  
    elif method == 'zscore':  
        out = pd.Series(zscore(series), index=series.index)  
    elif method == 'minmax':  
        mn, mx = series.min(), series.max()  
        out = (series - mn) / (mx - mn + 1e-9)  
    elif method == 'boxcox':  
        shifted = series - series.min() + 1  
        out_arr, _ = stats.boxcox(shifted)  
        out = pd.Series(out_arr, index=series.index)  
    else:  
        out = series  
    export = df.copy()  
    export[f'{feature}_transformed'] = out  
    return dcc.send_data_frame(export.to_csv, f'{feature}_{method}_transformed.csv',  
                               index=False)
```

```
#
```

```
# TAB 7: Numerical Plots
```

#

—— DropdownMenu: single callback with all 13 inputs ——

```
@app.callback(
    Output('num-plot-type', 'data'),
    Input('dd-hist', 'n_clicks'),
    Input('dd-kde', 'n_clicks'),
    Input('dd-box', 'n_clicks'),
    Input('dd-violin', 'n_clicks'),
    Input('dd-scatter', 'n_clicks'),
    Input('dd-qq', 'n_clicks'),
    Input('dd-dist', 'n_clicks'),
    Input('dd-area', 'n_clicks'),
    Input('dd-rug', 'n_clicks'),
    Input('dd-hexbin', 'n_clicks'),
    Input('dd-scatter3d', 'n_clicks'),
    Input('dd-contour', 'n_clicks'),
    Input('dd-joint', 'n_clicks'),
    prevent_initial_call=True,
)
def update_plot_type(*args):
    from dash import ctx
    triggered = ctx.triggered_id
    mapping = {
        'dd-hist': 'hist', 'dd-kde': 'kde', 'dd-box': 'box',
        'dd-violin': 'violin', 'dd-scatter': 'scatter', 'dd-qq': 'qq',
        'dd-dist': 'dist', 'dd-area': 'area', 'dd-rug': 'rug',
        'dd-hexbin': 'hexbin', 'dd-scatter3d': 'scatter3d',
        'dd-contour': 'contour', 'dd-joint': 'joint',
    }
    return mapping.get(triggered, 'hist')
```

```
@app.callback(
    Output('num-graph', 'figure'),
    Output('num-stats-alert', 'children'),
    Input('btn-num-plot', 'n_clicks'),
    State('num-x', 'value'),
```

```

    State('num-y', 'value'),
    State('num-hue', 'value'),
    State('num-plot-type', 'data'),
    State('num-bins', 'value'),
    prevent_initial_call=False,
)

def num_plot(n, x, y, hue, ptype, bins_range):
    hue_col = None if (not hue or hue == 'None') else hue
    samp = df.sample(min(5000, len(df)), random_state=42)
    # RangeSlider returns [min, max] — use max value as bin count
    bins = bins_range[1] if isinstance(bins_range, list) else (bins_range or 40)

    # helper: build kwargs only when hue_col is set
    def hkw():
        return {'color': hue_col, 'color_discrete_sequence': PALETTE} if hue_col else {}

    if ptype == 'hist':
        fig = go.Figure()
        series_x = df[x].dropna()
        if hue_col:
            for i, (cat, grp) in enumerate(df.groupby(hue_col)):
                fig.add_trace(go.Histogram(
                    x=grp[x].dropna(), name=str(cat),
                    nbinsx=bins, opacity=0.6,
                    marker_color=PALETTE[i % len(PALETTE)]))
        else:
            fig.add_trace(go.Histogram(
                x=series_x, nbinsx=bins,
                opacity=0.7, marker_color=C_M, name=x))
        # KDE overlay
        kde_fn = stats.gaussian_kde(series_x)
        xs = np.linspace(series_x.min(), series_x.max(), 300)
        scale = len(series_x) * (series_x.max() - series_x.min()) / bins
        fig.add_trace(go.Scatter(
            x=xs, y=kde_fn(xs) * scale,
            mode='lines', line=dict(color='red', width=2.5),
            name='KDE'))
        fig.update_layout(barmode='overlay', template=TEMPLATE)
        apply_layout(fig, f'Histogram + KDE: {x}', x, 'Count')

```

```

elif ptype == 'kde':
    fig = go.Figure()
    if hue_col:
        for cat, grp in df.groupby(hue_col):
            kde = stats.gaussian_kde(grp[x].dropna())
            xs = np.linspace(grp[x].min(), grp[x].max(), 300)
            fig.add_trace(go.Scatter(x=xs, y=kde(xs), mode='lines',
                                    fill='tozeroy', opacity=0.5,
                                    name=str(cat)))
    else:
        kde = stats.gaussian_kde(df[x].dropna())
        xs = np.linspace(df[x].min(), df[x].max(), 300)
        fig.add_trace(go.Scatter(x=xs, y=kde(xs), mode='lines',
                                fill='tozeroy', line_color=C_M))
    apply_layout(fig, f'KDE Plot: {x}', x, 'Density')

elif ptype == 'box':
    fig = px.box(df, y=x, template=TEMPLATE, points='outliers', **hkw())
    if not hue_col:
        fig.update_traces(marker_color=C_M)
    apply_layout(fig, f'Box Plot: {x}', '', x)

elif ptype == 'violin':
    fig = px.violin(df, y=x, box=True, template=TEMPLATE, **hkw())
    if not hue_col:
        fig.update_traces(marker_color=C_M)
    apply_layout(fig, f'Violin Plot: {x}', '', x)

elif ptype == 'scatter':
    fig = px.scatter(samp, x=x, y=y, trendline='ols', opacity=0.5,
                    template=TEMPLATE, **hkw())
    if not hue_col:
        fig.update_traces(marker_color=C_M, selector=dict(mode='markers'))
    r = df[[x, y]].corr().iloc[0, 1]
    apply_layout(fig, f'Scatter + Regression: {x} vs {y} (r={r:.2f})', x, y)

elif ptype == 'qq':
    qq = sm.ProbPlot(df[x].dropna())

```

```

lim = max(abs(qq.theoretical_quantiles).max(), abs(qq.sample_quantiles).max())
fig = go.Figure()
fig.add_trace(go.Scatter(x=qq.theoretical_quantiles,
                        y=qq.sample_quantiles,
                        mode='markers',
                        marker=dict(color=C_M, size=3, opacity=0.5),
                        name='Sample'))
fig.add_trace(go.Scatter(x=[-lim, lim], y=[-lim, lim],
                        mode='lines', line=dict(color='red', width=2),
                        name='Normal line'))
apply_layout(fig, f'QQ Plot: {x}',
             'Theoretical Quantiles', 'Sample Quantiles')

elif ptype == 'dist':
    hist_data = [df[x].dropna().values]
    fig = ff.create_distplot(hist_data, [x], colors=[C_M],
                            bin_size=(df[x].max()-df[x].min())/bins)
    apply_layout(fig, f'Dist Plot: {x}', x, 'Density')

elif ptype == 'area':
    monthly = df.groupby('Month')[x].mean().reset_index()
    fig = px.area(monthly, x='Month', y=x,
                  color_discrete_sequence=PALETTE, template=TEMPLATE)
    apply_layout(fig, f'Area Plot: Monthly Avg {x}', 'Month', f'Avg {x}')

elif ptype == 'rug':
    fig = go.Figure()
    if hue_col:
        for i, (cat, grp) in enumerate(df.groupby(hue_col)):
            samp_rug = grp[x].sample(min(2000, len(grp)), random_state=42)
            kde = stats.gaussian_kde(grp[x].dropna())
            xs = np.linspace(grp[x].min(), grp[x].max(), 300)
            fig.add_trace(go.Scatter(x=xs, y=kde(xs), mode='lines',
                                    name=str(cat),
                                    line_color=PALETTE[i % len(PALETTE)]))
    fig.add_trace(go.Scatter(
        x=samp_rug, y=[-0.002*(i+1)] * len(samp_rug),
        mode='markers', marker=dict(symbol='line-ns-open',
                                    color=PALETTE[i % len(PALETTE)]),

```

[illegible]

```

        showscale=False,
        line_width=1))
    apply_layout(fig, f'Contour Plot: {x} vs {y}', x, y)

elif ptype == 'joint':
    fig = make_subplots(rows=2, cols=2,
                        column_widths=[0.8, 0.2],
                        row_heights=[0.2, 0.8])
    samp2 = df[[x, y]].sample(min(3000, len(df)), random_state=42)
    fig.add_trace(go.Histogram(x=samp2[x], marker_color=C_M,
                              opacity=0.7, showlegend=False), row=1, col=1)
    fig.add_trace(go.Scatter(x=samp2[x], y=samp2[y], mode='markers',
                             marker=dict(color=C_M, size=3, opacity=0.4),
                             name='Data'), row=2, col=1)
    fig.add_trace(go.Histogram2dContour(x=samp2[x], y=samp2[y],
                                       colorscale='Blues',
                                       showscale=False,
                                       line_width=1), row=2, col=1)
    fig.add_trace(go.Histogram(y=samp2[y], marker_color=C_F,
                              opacity=0.7, showlegend=False), row=2, col=2)
    fig.update_layout(title=f'Joint Plot: {x} vs {y}',
                      template=TEMPLATE, font=dict(family='Arial', size=13))

else:
    fig = go.Figure()
    apply_layout(fig, 'Select a plot type')

stats_msg = (f'{x}: Mean={df[x].mean():.2f}, '
             f'Std={df[x].std():.2f}, '
             f'Skew={skew(df[x].dropna()):.2f}')
    return fig, stats_msg

```

#

TAB 8: Categorical Plots

#

```
@app.callback(
    Output('cat-graph', 'figure'),
    Output('cat-info-alert', 'children'),
    Input('btn-cat-plot', 'n_clicks'),
    State('cat-x', 'value'),
    State('cat-y', 'value'),
    State('cat-hue', 'value'),
    State('cat-plot-type', 'value'),
    State('cat-agg', 'value'),
    prevent_initial_call=False,
)
def cat_plot(n, x, y, hue, ptype, agg):
    hue_col = None if (not hue or hue == 'None') else hue

    def hkw():
        return {'color': hue_col, 'color_discrete_sequence': PALETTE} if hue_col else {}

    if ptype == 'bar':
        grp = df.groupby(x)[y].agg(agg).sort_values(ascending=False).reset_index()
        fig = px.bar(grp, x=x, y=y, text=y,
                     color=x, color_discrete_sequence=PALETTE,
                     template=TEMPLATE)
        fig.update_traces(texttemplate='%{text:.2f}', textposition='outside')
        apply_layout(fig, f'Bar Plot: {agg.title()} {y} by {x}', x, y)

    elif ptype == 'bar_stack':
        if hue_col:
            grp = df.groupby([x, hue_col])[y].agg(agg).reset_index()
            fig = px.bar(grp, x=x, y=y, barmode='stack',
                         template=TEMPLATE, **hkw())
        else:
            grp = df.groupby(x)[y].agg(agg).reset_index()
            fig = px.bar(grp, x=x, y=y, template=TEMPLATE,
                         color_discrete_sequence=PALETTE)
            apply_layout(fig, f'Stacked Bar: {y} by {x}', x, y)
```

```
elif ptype == 'bar_group':
    if hue_col:
        grp = df.groupby([x, hue_col])[y].agg(agg).reset_index()
        fig = px.bar(grp, x=x, y=y, barmode='group',
                     template=TEMPLATE, text=y, **hkw())
        fig.update_traces(texttemplate='%{text:.2f}', textposition='outside')
    else:
        grp = df.groupby(x)[y].agg(agg).reset_index()
        fig = px.bar(grp, x=x, y=y, template=TEMPLATE,
                     color_discrete_sequence=PALETTE)
    apply_layout(fig, f'Grouped Bar: {y} by {x} & {hue_col}', x, y)

elif ptype == 'count':
    fig = px.histogram(df, x=x, barmode='group',
                      template=TEMPLATE, text_auto=True, **hkw())
    apply_layout(fig, f'Count Plot: {x}', x, 'Count')

elif ptype == 'pie':
    vc = df[x].value_counts().reset_index()
    fig = px.pie(vc, names=x, values='count',
                color_discrete_sequence=PALETTE, template=TEMPLATE,
                hole=0.3)
    fig.update_traces(textposition='inside',
                      texttemplate='%{label}<br>%{percent:.2%}')
    fig.update_layout(title=f'Pie Chart: {x} Distribution',
                      font=dict(family='Arial', size=13))

elif ptype == 'box':
    fig = px.box(df, x=x, y=y, points='outliers',
                template=TEMPLATE, **hkw())
    apply_layout(fig, f'Box Plot: {y} by {x}', x, y)

elif ptype == 'violin':
    fig = px.violin(df, x=x, y=y, box=True,
                   template=TEMPLATE, **hkw())
    apply_layout(fig, f'Violin Plot: {y} by {x}', x, y)

elif ptype == 'strip':
    samp = df.sample(min(3000, len(df)), random_state=42)
```

```

fig = px.strip(samp, x=x, y=y, template=TEMPLATE,
               stripmode='overlay', **hkw())
apply_layout(fig, f'Strip Plot: {y} by {x}', x, y)

elif ptype == 'swarm':
    samp = df.sample(min(800, len(df)), random_state=42)
    fig = px.strip(samp, x=x, y=y, template=TEMPLATE, **hkw())
    apply_layout(fig, f'Swarm Plot: {y} by {x}', x, y)

elif ptype == 'boxen':
    fig = px.box(df, x=x, y=y, points=False,
                 notched=True, template=TEMPLATE, **hkw())
    apply_layout(fig, f'Boxen Plot: {y} by {x}', x, y)

else:
    fig = go.Figure()

n_cats = df[x].nunique()
info = (f'{x}: {n_cats} unique categories | '
        f'Most common: {df[x].value_counts().index[0]} '
        f'{df[x].value_counts().iloc[0:,:]}')
return fig, info

```

#

TAB 9: Statistics

#

```

@app.callback(
    Output('stats-heatmap', 'figure'),
    Output('stats-corr-alert', 'children'),
    Input('btn-stats-heat', 'n_clicks'),
    State('stats-heatmap-features', 'value'),
    State('stats-cmap', 'value'),
    prevent_initial_call=False,

```

```

)
def stats_heatmap(n, features, cmap):
    features = features or NUM_COLS[:6]
    corr = df[features].corr().round(2)
    fig = px.imshow(corr, text_auto='.2f', color_continuous_scale=cmap,
                    zmin=-1, zmax=1, template=TEMPLATE,
                    aspect='auto')
    apply_layout(fig, 'Pearson Correlation Heatmap', "", "")

    # Find highest absolute correlation (off-diagonal)
    corr_abs = corr.abs().mask(np.eye(len(corr), dtype=bool), 0)
    idx = corr_abs.stack().idxmax()
    val = corr.loc[idx[0], idx[1]]

    msg = (f'Strongest correlation: {idx[0]} ↔ {idx[1]} '
           f'(r = {val:.2f})')
    return fig, msg

@app.callback(
    Output('stats-pair', 'figure'),
    Input('btn-stats-pair', 'n_clicks'),
    State('stats-pair-features', 'value'),
    State('stats-pair-hue', 'value'),
    prevent_initial_call=False,
)
def stats_pair(n, features, hue):
    features = (features or NUM_COLS[:4])[:5]
    samp = df[features + [hue]].sample(min(2000, len(df)), random_state=42)
    fig = px.scatter_matrix(samp, dimensions=features, color=hue,
                           color_discrete_sequence=PALETTE,
                           opacity=0.4, template=TEMPLATE)
    fig.update_traces(diagonal_visible=True, showupperhalf=False)
    apply_layout(fig, f'Scatter Matrix (Pair Plot) — color: {hue}', "", "")
    return fig

@app.callback(
    Output('stats-kde', 'figure'),
    Input('btn-stats-kde', 'n_clicks'),

```

```

    State('stats-kde-x', 'value'),
    State('stats-kde-y', 'value'),
    prevent_initial_call=False,
)
def stats_kde(n, x, y):
    samp = df[[x, y]].sample(min(3000, len(df)), random_state=42)
    fig = go.Figure()
    fig.add_trace(go.Histogram2dContour(
        x=samp[x], y=samp[y],
        colorscale='Viridis',
        contours_coloring='fill',
        showscale=True,
        colorbar=dict(title='Density')
    ))
    fig.add_trace(go.Histogram2dContour(
        x=samp[x], y=samp[y],
        colorscale='Viridis',
        contours_coloring='lines',
        showscale=False,
        line_width=0.8,
        line_color='white'
    ))
    apply_layout(fig, f'Multivariate KDE: {x} vs {y}', x, y)
    return fig

```

```

@app.callback(
    Output('stats-table-div', 'children'),
    Input('main-tabs', 'active_tab'),
)
def stats_table(tab):
    desc = df[NUM_COLS].describe().round(2)
    extra = pd.DataFrame({
        'skewness': [skew(df[c].dropna()) for c in NUM_COLS],
        'kurtosis': [kurtosis(df[c].dropna()) for c in NUM_COLS],
    }, index=NUM_COLS).round(2).T
    desc = pd.concat([desc, extra])
    desc = desc.reset_index().rename(columns={'index': 'stat'})
    return dash_table.DataTable(

```

```
data=desc.to_dict('records'),
columns=[{'name': c, 'id': c} for c in desc.columns],
style_table={'overflowX': 'auto'},
style_cell={'textAlign': 'center', 'padding': '4px', 'fontSize': '11px'},
style_header={'backgroundColor': '#2c3e50', 'color': 'white',
              'fontWeight': 'bold'},
style_data_conditional=[
    {'if': {'row_index': 'odd'}, 'backgroundColor': '#f0f4f8'}],
page_size=12,
)

# =====
# DATS6401 Final Term Project — Phase II: Interactive Dashboard
# Dataset: Online Shopping | Author: Jeongwon Yoo
# =====

import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import plotly.figure_factory as ff
from plotly.subplots import make_subplots
from scipy import stats
from scipy.stats import shapiro, normaltest, zscore, skew, kurtosis
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.decomposition import PCA
import statsmodels.api as sm
import warnings
warnings.filterwarnings('ignore')

import dash
from dash import dcc, html, Input, Output, State, dash_table
import dash_bootstrap_components as dbc
import dash_daq as daq

# =====
# DATA LOADING & PREPROCESSING
# =====
```

```
DATA_PATH = '/Users/jeongwonyoo/Desktop/2026_Spring/Data  
Visualization/online_shopping/online_shopping.csv'
```

```
df_raw = pd.read_csv(DATA_PATH)  
df_raw = df_raw.drop(columns=['Unnamed: 0'], errors='ignore')
```

```
col_required = ['CustomerID', 'Gender', 'Location', 'Tenure_Months', 'Transaction_ID',  
               'Quantity', 'Avg_Price', 'Delivery_Charges', 'Coupon_Status',  
               'GST', 'Offline_Spend', 'Online_Spend']  
df = df_raw.dropna(subset=col_required).copy()  
df['Transaction_Date'] = pd.to_datetime(df['Transaction_Date'])  
df['CustomerID']      = df['CustomerID'].astype(int)  
df['Transaction_ID']  = df['Transaction_ID'].astype(int)  
df['Total_Spend']     = df['Offline_Spend'] + df['Online_Spend']  
df['Revenue']         = df['Quantity'] * df['Avg_Price']  
df['Month']           = df['Transaction_Date'].dt.month  
df['Price_Bin']       = pd.cut(df['Avg_Price'],  
                               bins=[0,25,50,100,200,400],  
                               labels=['$0-25', '$25-50', '$50-100', '$100-200', '$200+'])
```

```
CAT_COLS = ['Gender', 'Location', 'Product_Category', 'Coupon_Status']  
NUM_COLS = ['Tenure_Months', 'Quantity', 'Avg_Price', 'Delivery_Charges',  
            'GST', 'Offline_Spend', 'Online_Spend', 'Discount_pct', 'Total_Spend', 'Revenue']
```

```
PALETTE = px.colors.qualitative.Set2  
C_F     = '#e74c3c'  
C_M     = '#3498db'  
TEMPLATE = 'plotly_white'
```

```
# =====  
# HELPER: standard figure layout  
# =====  
def apply_layout(fig, title="", xlab="", ylab="):  
    fig.update_layout(  
        title=dict(text=title, font=dict(size=18, color='#2c3e50'), x=0.5),  
        xaxis_title=xlab, yaxis_title=ylab,  
        template=TEMPLATE, margin=dict(t=60, b=50, l=60, r=30),  
        legend=dict(bgcolor='rgba(255,255,255,0.8)', bordercolor='#ccc', borderwidth=1),  
        font=dict(family='Arial', size=13)
```

```
)
return fig

# =====
# APP INIT
# =====
app = dash.Dash(__name__,
                 external_stylesheets=[dbc.themes.FLATLY],
                 suppress_callback_exceptions=True)
app.title = 'Online Shopping Dashboard — DATS6401'

# =====
# NAVBAR
# =====
navbar = dbc.NavbarSimple(
    brand='🛒 Online Shopping Analytics Dashboard',
    brand_href='#',
    color='primary',
    dark=True,
    children=[
        dbc.NavItem(dbc.NavLink('DATS6401 | Jeongwon Yoo', href='#')),
    ]
)

# =====
# TAB CONTENTS
# =====

# ——— TAB 1: Load Data

tab_load = dbc.Tab(label='📁 Load Data', tab_id='tab-load', children=[
    dbc.Container([
        dbc.Row([
            dbc.Col([
                dbc.Card([
                    dbc.CardHeader(html.H5('Dataset Overview', className='mb-0')),
                    dbc.CardBody([
                        dbc.Row([
                            dbc.Col([
```

```
html.Label('Select display rows:'),
dbc.Select(id='load-rows',
           options=[{'label':f'{n} rows','value':n}
                    for n in [5,10,20,50]],
           value=10),
], width=3),
dbc.Col([
    html.Label('Filter by Column:'),
    dbc.Select(id='load-col-filter',
               options=[{'label':'All','value':'All'}] +
                       [{'label':c,'value':c} for c in df.columns],
               value='All'),
], width=3),
dbc.Col([
    dbc.Button('Load Data', id='btn-load', color='primary',
               className='mt-4'),
    dbc.Spinner(html.Div(id='load-status'), size='sm'),
], width=3),
], className='mb-3'),
html.Hr(),
dbc.Row([
    dbc.Col(dbc.Card([
        dbc.CardBody([
            html.H6('Total Rows'),
            dbc.Badge(f'{len(df):,}', color='primary', className='fs-5'),
        ])
    ]), width=2),
    dbc.Col(dbc.Card([
        dbc.CardBody([
            html.H6('Columns'),
            dbc.Badge(f'{df.shape[1]}', color='info', className='fs-5'),
        ])
    ]), width=2),
    dbc.Col(dbc.Card([
        dbc.CardBody([
            html.H6('Missing Values'),
            dbc.Badge(f'{df.isnull().sum().sum()}', color='success', className='fs-5'),
        ])
    ]), width=2),
], width=2),
```

```

        dbc.Col(dbc.Card([
            dbc.CardBody([
                html.H6('Unique Customers'),
                dbc.Badge(f'{df["CustomerID"].nunique():,}', color='warning', className='fs-5'),
            ])
        ]), width=2),
        dbc.Col(dbc.Card([
            dbc.CardBody([
                html.H6('Date Range'),
                dbc.Badge(f'{df["Transaction_Date"].min().date()} ~ {df["Transaction_Date"].max().date()}',
                    color='secondary', className='fs-6'),
            ])
        ]), width=4),
    ], className='mb-3'),
    html.Div(id='load-table-div'),
])
])
])
]),
dbc.Row([
    dbc.Col([
        dbc.Card([
            dbc.CardHeader(html.H5('Descriptive Statistics')),
            dbc.CardBody([
                html.Div(id='load-stats-div'),
            ])
        ])
    ])
]), className='mt-3'),
], fluid=True, className='py-3')
])

```

— TAB 2: Data Cleaning

```

tab_clean = dbc.Tab(label='🔧 Data Cleaning', tab_id='tab-clean', children=[
    dbc.Container([
        dbc.Card([
            dbc.CardHeader(html.H5('Data Cleaning Methods')),

```

```
dbc.CardBody([
  dbc.Row([
    dbc.Col([
      html.Label('Cleaning Method:'),
      dbc.RadioItems(
        id='clean-method',
        options=[
          {'label': 'Drop NA rows', 'value': 'drop'},
          {'label': 'Fill with Mean', 'value': 'mean'},
          {'label': 'Fill with Median', 'value': 'median'},
          {'label': 'Fill with Mode', 'value': 'mode'},
          {'label': 'Forward Fill', 'value': 'ffill'},
        ],
        value='drop', inline=False,
      ),
    ], width=3),
    dbc.Col([
      html.Label('Select Feature to Check:'),
      dbc.Select(id='clean-feature',
        options=[{'label': c, 'value': c} for c in
          ['Tenure_Months', 'Quantity', 'Avg_Price',
            'Delivery_Charges', 'GST',
            'Offline_Spend', 'Online_Spend', 'Discount_pct']],
        value='Online_Spend'),
      html.Br(),
      dbc.Button('Apply Cleaning', id='btn-clean', color='success'),
    ], width=3),
    dbc.Col([
      dbc.Alert(id='clean-alert', color='info',
        children='Select a method and click Apply.'),
      dbc.Row([
        dbc.Col(daq.LEDDisplay(id='led-missing-before',
          label='Missing Before',
          value='0', size=24,
          color='#e74c3c'), width=6),
        dbc.Col(daq.LEDDisplay(id='led-missing-after',
          label='Missing After',
          value='0', size=24,
          color='#2ecc71'), width=6),
      ])
    ]
  ])
])
```

```

    ]),
    ], width=6),
    ], className='mb-3'),
    dcc.Graph(id='clean-graph'),
  ])
]),
], fluid=True, className='py-3')
])

```

— TAB 3: Outlier Detection

```

tab_outlier = dbc.Tab(label='🔍 Outlier Detection', tab_id='tab-outlier', children=[
  dbc.Container([
    dbc.Card([
      dbc.CardHeader(html.H5('Outlier Detection & Removal')),
      dbc.CardBody([
        dbc.Row([
          dbc.Col([
            html.Label('Detection Method:'),
            dbc.RadioItems(
              id='outlier-method',
              options=[
                {'label': 'IQR (1.5×)', 'value': 'iqr'},
                {'label': 'Z-Score ( $|z| > 3$ )', 'value': 'zscore'},
                {'label': 'Modified Z-Score', 'value': 'mod_z'},
              ],
              value='iqr',
            ),
          ], width=3),
          dbc.Col([
            html.Label('Feature:'),
            dbc.Select(id='outlier-feature',
              options=[{'label': c, 'value': c} for c in NUM_COLS],
              value='Online_Spend'),
            html.Br(),
            html.Label('IQR Multiplier:'),
            dcc.Slider(id='outlier-k', min=1.0, max=3.0, step=0.5,
              value=1.5, marks={1.0:'1.0', 1.5:'1.5', 2.0:'2.0',
                2.5:'2.5', 3.0:'3.0'}),

```

```

        dbc.Button('Detect Outliers', id='btn-outlier', color='warning', className='mt-2'),
    ], width=4),
    dbc.Col([
        dbc.Row([
            dbc.Col(daq.Gauge(id='gauge-outlier-pct',
                              label='Outlier %',
                              min=0, max=30, value=0,
                              color={'gradient':True,
                                    'ranges':{'green':[0,5],
                                              'yellow':[5,15],
                                              'red':[15,30]}},
                              size=150), width=6),
            dbc.Col([
                daq.LEDDisplay(id='led-outlier-n', label='# Outliers',
                               value='0', size=24, color='#e74c3c'),
                html.Br(),
                daq.LEDDisplay(id='led-clean-n', label='Cleaned Rows',
                               value=str(len(df)), size=24, color='#2ecc71'),
            ], width=6),
        ]),
    ], width=5),
], className='mb-3'),
dbc.Row([
    dbc.Col(dcc.Graph(id='outlier-box'), width=6),
    dbc.Col(dcc.Graph(id='outlier-hist'), width=6),
]),
dbc.Alert(id='outlier-alert', color='info',
          children='Select method and click Detect.'),
])
]),
], fluid=True, className='py-3')
])

```

— TAB 4: PCA

```

tab_pca = dbc.Tab(label='PCA', tab_id='tab-pca', children=[
    dbc.Container([

```

```
dbc.Card([
  dbc.CardHeader(html.H5('Principal Component Analysis (PCA)'),
  dbc.CardBody([
    dbc.Row([
      dbc.Col([
        html.Label('Features to include:'),
        dbc.Checklist(
          id='pca-features',
          options=[{'label':c,'value':c} for c in NUM_COLS],
          value=NUM_COLS,
          inline=False,
        ),
      ], width=3),
      dbc.Col([
        html.Label('Number of Components:'),
        dcc.Slider(id='pca-n', min=2, max=10, step=1, value=5,
          marks={i:str(i) for i in range(2,11)}),
        html.Br(),
        html.Label('Color by:'),
        dbc.Select(id='pca-color',
          options=[{'label':c,'value':c} for c in CAT_COLS],
          value='Gender'),
        html.Br(),
        dbc.Button('Run PCA', id='btn-pca', color='primary'),
      ], width=4),
      dbc.Col([
        daq.GraduatedBar(id='grad-bar-pca',
          label='Cumulative Variance (%)',
          min=0, max=100, value=0,
          showCurrentValue=True, size=200,
          color={'gradient':True,
            'ranges':{'red':[0,60],
              'yellow':[60,80],
              'green':[80,100]})),
        html.Br(),
        dbc.Alert(id='pca-cond-alert', color='secondary',
          children='Condition number will appear here.'),
      ], width=5),
    ], className='mb-3'),
```

```

        dbc.Row([
            dbc.Col(dcc.Graph(id='pca-scree'), width=6),
            dbc.Col(dcc.Graph(id='pca-biplot'), width=6),
        ]),
        dbc.Row([
            dbc.Col(dcc.Graph(id='pca-cumvar'), width=12),
        ]),
    ])
    ], fluid=True, className='py-3')
])

```

— TAB 5: Normality Test

```

tab_normality = dbc.Tab(label='Normality Test', tab_id='tab-norm', children=[
    dbc.Container([
        dbc.Card([
            dbc.CardHeader(html.H5('Normality Tests')),
            dbc.CardBody([
                dbc.Row([
                    dbc.Col([
                        html.Label('Feature:'),
                        dbc.Select(id='norm-feature',
                                options=[{'label':c,'value':c} for c in NUM_COLS],
                                value='Online_Spend'),
                        html.Br(),
                        html.Label('Test Method:'),
                        dbc.RadioItems(
                            id='norm-method',
                            options=[
                                {'label': "Shapiro-Wilk", 'value': 'shapiro'},
                                {'label': "D'Agostino K2", 'value': 'dagostino'},
                                {'label': 'Both', 'value': 'both'},
                            ],
                            value='both',
                        ),
                        html.Br(),
                        dbc.Button('Run Test', id='btn-norm', color='info'),
                    ], width=3),

```

```
dbc.Col([
    dbc.Alert(id='norm-result', color='secondary',
        children='Select feature and run test.'),
    dbc.Row([
        dbc.Col(daq.Thermometer(id='thermo-skew',
            label='Skewness',
            min=-5, max=5, value=0,
            showCurrentValue=True,
            height=120, width=20,
            color='#3498db'), width=4),
        dbc.Col(daq.Thermometer(id='thermo-kurt',
            label='Kurtosis',
            min=-3, max=10, value=0,
            showCurrentValue=True,
            height=120, width=20,
            color='#e74c3c'), width=4),
        dbc.Col([
            daq.BooleanSwitch(id='norm-switch',
                label='Normal?',
                on=False,
                color='#2ecc71'),
            ], width=4),
    ]),
], width=4),
dbc.Col([
    daq.Knob(id='knob-pval',
        label='p-value',
        min=0, max=1, value=0,
        size=120,
        color={'gradient':True,
            'ranges':{'green':[0,0.05],
                'yellow':[0.05,0.1],
                'red':[0.1,1]}}),
    ], width=2),
dbc.Col([
    daq.ColorPicker(id='norm-color',
        label='Plot Color',
        value=dict(hex='#3498db')),
    ], width=3),
```

```

    ], className='mb-3'),
    dbc.Row([
        dbc.Col(dcc.Graph(id='norm-hist'), width=6),
        dbc.Col(dcc.Graph(id='norm-qq'), width=6),
    ]),
    ])
    ]),
    ], fluid=True, className='py-3')
])

```

— TAB 6: Data Transformation

```

tab_transform = dbc.Tab(label='📊 Data Transformation', tab_id='tab-transform', children=[
    dbc.Container([
        dbc.Card([
            dbc.CardHeader(html.H5('Data Transformation')),
            dbc.CardBody([
                dbc.Row([
                    dbc.Col([
                        html.Label('Feature:'),
                        dbc.Select(id='trans-feature',
                                options=[{'label':c,'value':c} for c in NUM_COLS],
                                value='Online_Spend'),
                        html.Br(),
                        html.Label('Transformation Method:'),
                        dbc.Select(id='trans-method',
                                options=[
                                    {'label':'Log (log1p)','value':'log'},
                                    {'label':'Square Root','value':'sqrt'},
                                    {'label':'Z-Score Standardization','value':'zscore'},
                                    {'label':'Min-Max Normalization','value':'minmax'},
                                    {'label':'Box-Cox','value':'boxcox'},
                                ],
                                value='log'),
                        html.Br(),
                        dbc.Button('Apply', id='btn-transform', color='success'),
                    ], width=3),
                    dbc.Col([
                        dbc.Row([

```

```

        dbc.Col([
            html.P('Before:', style={'fontWeight':'bold'}),
            daq.LEDDisplay(id='led-skew-before', label='Skewness',
                           value='0.00', size=20, color='#e74c3c'),
            html.Br(),
            daq.LEDDisplay(id='led-mean-before', label='Mean',
                           value='0.00', size=20, color='#3498db'),
        ], width=6),
        dbc.Col([
            html.P('After:', style={'fontWeight':'bold'}),
            daq.LEDDisplay(id='led-skew-after', label='Skewness',
                           value='0.00', size=20, color='#2ecc71'),
            html.Br(),
            daq.LEDDisplay(id='led-mean-after', label='Mean',
                           value='0.00', size=20, color='#9b59b6'),
        ], width=6),
    ]),
    dbc.Alert(id='transform-alert', color='info',
              children='Select a feature and transformation.'),
], width=4),
dbc.Col([
    dcc.Download(id='download-transformed'),
    dbc.Button('↓ Download Transformed CSV',
               id='btn-download-transform',
               color='secondary', outline=True, className='mt-2'),
], width=5),
], className='mb-3'),
dbc.Row([
    dbc.Col(dcc.Graph(id='trans-before'), width=6),
    dbc.Col(dcc.Graph(id='trans-after'), width=6),
]),
])
]),
], fluid=True, className='py-3')
])

```

— TAB 7: Numerical Feature Plots

```

tab_num = dbc.Tab(label='☑ Numerical Plots', tab_id='tab-num', children=[

```

```
dbc.Container([
    dbc.Card([
        dbc.CardHeader(html.H5('Numerical Feature Visualization')),
        dbc.CardBody([
            dbc.Row([
                dbc.Col([
                    html.Label('X Feature:'),
                    dbc.Select(id='num-x',
                               options=[{'label':c,'value':c} for c in NUM_COLS],
                               value='Online_Spend'),
                    html.Br(),
                    html.Label('Y Feature (for 2D plots):'),
                    dbc.Select(id='num-y',
                               options=[{'label':c,'value':c} for c in NUM_COLS],
                               value='Offline_Spend'),
                    html.Br(),
                    html.Label('Color by:'),
                    dbc.Select(id='num-hue',
                               options=[{'label':'None','value':'None'}] +
                                     [{'label':c,'value':c} for c in CAT_COLS],
                               value='Gender'),
                ], width=3),
            dbc.Col([
                html.Label('Plot Type:'),
                dbc.DropdownMenu(
                    label='Select Plot Type',
                    id='num-plot-dropdown',
                    children=[
                        dbc.DropdownMenuItem('Histogram + KDE', id='dd-hist'),
                        dbc.DropdownMenuItem('KDE Plot', id='dd-kde'),
                        dbc.DropdownMenuItem('Box Plot', id='dd-box'),
                        dbc.DropdownMenuItem('Violin Plot', id='dd-violin'),
                        dbc.DropdownMenuItem('Scatter + Regression', id='dd-scatter'),
                        dbc.DropdownMenuItem('QQ Plot', id='dd-qq'),
                        dbc.DropdownMenuItem('Dist Plot', id='dd-dist'),
                        dbc.DropdownMenuItem('Area Plot', id='dd-area'),
                        dbc.DropdownMenuItem('Rug Plot', id='dd-rug'),
                        dbc.DropdownMenuItem('Hexbin / Density', id='dd-hexbin'),
                        dbc.DropdownMenuItem('3D Scatter', id='dd-scatter3d'),
```

```

        dbc.DropdownMenuItem('Contour',      id='dd-contour'),
        dbc.DropdownMenuItem('Joint Plot',   id='dd-joint'),
    ],
    color='secondary', className='mb-2',
),
dbc.Tooltip('Choose a plot type from the dropdown menu.',
            target='num-plot-dropdown'),
dcc.Store(id='num-plot-type', data='hist'),
html.Br(),
html.Label('Bin Range (for histogram):'),
dcc.RangeSlider(id='num-bins', min=10, max=100, step=10,
                value=[10, 50],
                marks={10:'10', 30:'30', 50:'50',
                       70:'70', 100:'100'}),
dbc.Tooltip('Drag both handles to set the min and max bin count.',
            target='num-bins'),
html.Br(),
dbc.Button('Plot', id='btn-num-plot', color='primary'),
dbc.Tooltip('Click to generate the selected plot.',
            target='btn-num-plot'),
], width=4),
dbc.Col([
    dbc.Alert(id='num-stats-alert', color='light',
              children='Stats will appear here.'),
    html.Label('Observations / Notes:'),
    dcc.Textarea(id='num-textarea',
                 placeholder='Write your observations about this plot here...',
                 style={'width':'100%', 'height':'120px',
                        'fontSize':'13px'}),
    dbc.Tooltip('Use this area to note your observations about the plot.',
                target='num-textarea'),
], width=5),
], className='mb-3'),
dbc.Spinner(dcc.Graph(id='num-graph', style={'height':'550px'})),
])
]),
], fluid=True, className='py-3')
])

```

— TAB 8: Categorical Feature Plots —

```
tab_cat = dbc.Tab(label='Categorical Plots', tab_id='tab-cat', children=[
    dbc.Container([
        dbc.Card([
            dbc.CardHeader(html.H5('Categorical Feature Visualization')),
            dbc.CardBody([
                dbc.Row([
                    dbc.Col([
                        html.Label('Categorical Feature:'),
                        dbc.Select(id='cat-x',
                                options=[{'label':c,'value':c} for c in CAT_COLS],
                                value='Product_Category'),
                        html.Br(),
                        html.Label('Value Feature (Y-axis):'),
                        dbc.Select(id='cat-y',
                                options=[{'label':c,'value':c} for c in NUM_COLS],
                                value='Revenue'),
                        html.Br(),
                        html.Label('Hue (Group by):'),
                        dbc.Select(id='cat-hue',
                                options=[{'label':'None','value':'None'}] +
                                      [{'label':c,'value':c} for c in CAT_COLS],
                                value='Gender'),
                    ], width=3),
                    dbc.Col([
                        html.Label('Plot Type:'),
                        dbc.Select(id='cat-plot-type',
                                options=[
                                    {'label':'Bar Plot (sorted)','value':'bar'},
                                    {'label':'Stacked Bar','value':'bar_stack'},
                                    {'label':'Grouped Bar','value':'bar_group'},
                                    {'label':'Count Plot','value':'count'},
                                    {'label':'Pie Chart','value':'pie'},
                                    {'label':'Box Plot','value':'box'},
                                    {'label':'Violin Plot','value':'violin'},
                                    {'label':'Strip Plot','value':'strip'},
                                    {'label':'Swarm Plot','value':'swarm'},
                                    {'label':'Boxen Plot','value':'boxen'},
                                ],
                                value='Bar Plot (sorted)'),
                    ], width=3),
                ],
            ),
        ],
    ),
])
```

```
        inline=False,
    ),
], width=5),
dbc.Col([
    html.Label('Color by:'),
    dbc.Select(id='stats-pair-hue',
               options=[{'label':c,'value':c}
                        for c in CAT_COLS],
               value='Gender'),
    html.Br(),
    dbc.Button('Update', id='btn-stats-pair', color='success'),
], width=7),
], className='mb-2'),
dbc.Spinner(dcc.Graph(id='stats-pair')),
])
]),
], width=6),
], className='mb-3'),
dbc.Row([
    dbc.Col([
        dbc.Card([
            dbc.CardHeader(html.H5('Multivariate KDE & Statistics Table')),
            dbc.CardBody([
                dbc.Row([
                    dbc.Col([
                        html.Label('X Feature:'),
                        dbc.Select(id='stats-kde-x',
                                 options=[{'label':c,'value':c}
                                            for c in NUM_COLS],
                                 value='Online_Spend'),
                        html.Label('Y Feature:'),
                        dbc.Select(id='stats-kde-y',
                                 options=[{'label':c,'value':c}
                                            for c in NUM_COLS],
                                 value='Offline_Spend'),
                        html.Br(),
                        dbc.Button('Update KDE', id='btn-stats-kde',
                                  color='info'),
                    ], width=3),
```

```

        dbc.Col(dcc.Graph(id='stats-kde'), width=9),
    ),
),
], width=7),
dbc.Col([
    dbc.Card([
        dbc.CardHeader(html.H5('Descriptive Statistics Table')),
        dbc.CardBody([
            html.Div(id='stats-table-div'),
        ])
    ]),
], width=5),
]),
], fluid=True, className='py-3')
])

# =====
# MAIN LAYOUT
# =====
app.layout = dbc.Container([
    navbar,
    html.Br(),
    dbc.Tabs([
        tab_load,
        tab_clean,
        tab_outlier,
        tab_pca,
        tab_normality,
        tab_transform,
        tab_num,
        tab_cat,
        tab_stats,
    ], id='main-tabs', active_tab='tab-load'),
], fluid=True)

```
